TRADEGENIUS

System Architecture

v6.14.0

May 2026

Repo: valira3/stock-spike-monitor
Service: tradegenius.up.railway.app

Contents

Strategy Compliance — v5.13.0 (Tiger Sovereign)

- v5.13.0 PR sequence
- Rule-ID inventory
- Production posture during the series

Module map (post-v5.11.0)

- telegram_ui/ (post-v5.11.1)
- broker/ (post-v5.11.2)
- executors/ (post-v5.12.0)

v5.10.0 — Project Eye of the Tiger (algorithm summary)

1. High-level overview

2. Repo layout

3. Process startup

4. Ticker universe

5. Scheduler & scan loop

- 5.1 Timed jobs (`scheduler_thread`)
- 5.2 Scan loop (`scan_loop`)

6. Trading algorithm — Tiger/Buffalo (v5.0.0, unchanged through v5.1.2)

- 6.1 States
- 6.2 Long protocol — "The Tiger Hunts the Bison"
- 6.3 Short protocol — "The Wounded Buffalo / Gravity Trade"
- 6.4 Sizing — preserved from v4

7. Risk: v5 stop model + portfolio brakes

- 7.1 v5 per-trade stop model
- 7.2 Hard exits (L-P4-R3 / S-P4-R3..R4)
- 7.3 Re-Hunt budget (L-P5-R3 / S-P5-R3)
- 7.4 Portfolio-level brakes — preserved from v4

8. Signal bus & executors

- 8.1 The bus
- 8.2 Executors: TradeGeniusVal & TradeGeniusGene
- 8.3 Live mode flip
- 8.4 Alpaca order idempotency (v5.2.1 H1)

8.5 Alpaca client URL hygiene

9. Dashboard

9.1 Backend (`dashboard_server.py`)

9.2 Index ticker (v4.10 → v4.13.0)

9.3 Frontend (`dashboard_static/`)

9.4 Shadow strategy P&L (v5.2.0)

9.5 Shadow tab (v5.3.0)

9.6 Shadow market-data credentials (v5.5.3)

10. Environment variables

10.1 Required

10.2 Owner whitelist

10.3 Market data

10.3b Data root (v6.9.1)

10.3c Sweep env helper (v6.9.4)

10.4 Persistence (Railway Volume)

10.5 Risk / strategy

10.6 Dashboard

10.7 Executors (Val / Gene)

10.8 Forensic Volume Filter (v5.1.0+)

10.9 Testing

10.10 Do not set

11. Known gotchas

11.1 Dockerfile per-file COPY whitelist

11.2 Yahoo "real-time" is not real-time

11.3 `marketState` is `None`

11.4 Dashboard is read-only

11.5 Don't import `trade_genius` from helper threads

11.6 Executor trade-confirmation DM (v5.0.3 auto-learn)

11.7 Paper and live Alpaca keys are independent (v5.0.4 revert)

12. Persistence

12.1 SQLite store (v5.1.8+)

12.2 Legacy JSON shape (still relevant for the import path)

13. Testing

13.1 Local smoke (`smoke_test.py`)

13.2 Synthetic harness (`synthetic_harness/`)

13.3 CI guards

14. Deployment

14.1 Railway

14.2 Local development

14.3 Logging

15. Maintenance

15.1 Regenerating `trade_genius_algo.pdf`

15.2 Bumping `BOT_VERSION`

15.3 Adding a new top-level Python module

15.4 Reading the live state without restarting

16. v5 strategy quick reference

16.1 State machine

16.2 Rule grid

16.3 Where the v5 logic lives

16.4 Migration from v4

16.5 v5.0.x → v5.4.x — what changed (and what didn't)

17. Forensic Volume Filter (Anaplan logic) — v5.1.0 → v5.4.1

17.1 Volume Library — parameters

17.2 V-P1 grid (legacy fixed-threshold evaluator)

17.3 Env-driven active config + helper (v5.1.1+)

17.4 `SHADOW_CONFIGS` — fixed analysis configs (v5.1.1 → v5.1.9)

17.4a Bucket-Fill Velocity (v5.1.6)

17.5 Shadow log lines

17.6 Failure modes

17.7 Rollout plan (current)

18. Forensic Capture (v5.1.2)

18.1 Bar archive (`bar_archive.py` — Tier-1.1)

18.2 Per-minute volume log (`[V510-MINUTE]` — Tier-1.2)

18.3 Skipped-candidate log (`[V510-CAND]` — Tier-1.3)

18.4 Entry log enrichment (`[V510-ENTRY]` — Tier-1.4)

18.5 FSM transition log (`[V510-FSM]` — Tier-2.1)

18.6 Pre-trade indicator snapshots (`indicators.py` — Tier-2.2)

18.7 Out of scope (deferred)

18a. Always-On Algo Plus Ingest (v6.5.0)

18a.1 Component overview

18a.2 ConnectionHealth state machine

18a.3 Bar flow sequence — normal boot and live path

18a.4 Gap detection contract
18a.5 REST backfill rate-limit budget
18a.6 BAR_SCHEMA_FIELDS — `feed_source` addition (M-4)
18a.7 `shadow_data_status` contract
18a.8 `/api/state` additions
18a.9 Extended REST window and SIP feed promotion
18a.10 Boot wiring

19. Permission gates — v5.9.0 QQQ Regime Shield + ticker AVWAP + OR

19.1 Three-gate set (v5.9.0+: G1 swapped to EMA cross)
19.1.1 Data-collection log schema (v5.6.1)
19.2 Volume-profile G4 shadow grid (legacy v5.1.0+ surface, unchanged)

20. Offline backtest CLI (v5.4.0)

20.1 How to run
20.2 What `--validate` does
20.3 CSV ledger columns
20.4 Reused logic

21. Shadow tab charts (v5.4.1)

21.1 Endpoint — `GET /api/shadow_charts`
21.2 Frontend — three chart groups on the Shadow tab
21.3 Polling
21.4 Interactivity (v5.5.1)

22. v5.7.0 — Unlimited Titan Strikes (Apr 2026)

23. v5.7.1 — Bison & Buffalo exit FSM (Apr 2026)

24. v5.8.0 — Developer Velocity Bundle (Apr 2026)

25. v5.8.1 — Ops scripts (Infra-B post-deploy smoke + checks library)

Saturday weekly report (v5.8.4)

v6.1.0 — P&L recovery bundle (May 2026)

Why
#1 ATR-scaled trailing stop with profit-protect ratchet
#2 Two-bar EMA-cross confirmation + lunch-chop suppression
#3 ATR-normalized OR-break entry gate + late-OR window
Validation
Rollback playbook

v6.2.0 — Entry-gate loosening (May 2026)

Why

#A Local override OR-break leg
#B Time-conditional 1-bar boundary hold pre-10:30 ET
#D Generic ENTRY_1 DI threshold 25 → 22
Dashboard surfacing
Rollback playbook

§18b. v6.6.0 — Ingest Hardening (May 2026)

Overview
New Files
Modified Files
Storage
Locked Decisions
Rollout

System Health Test (v6.7.0)

15 Checks
Orchestrator Design
RTH Window
Thresholds (locked, PRODUCT_SPEC.md)
Sample Passing Output

§19. v6.9.0 -- Backtest Cache Layer

L1 -- Parquet bar cache (`backtest/bar_cache.py`)
L2 -- Indicator precompute cache (`backtest/indicator_cache.py`)
Engine module annotations

§20. v6.11.0 -- C25: SPY Regime-B Short Amplification

Regime classification (`spy_regime.py`)
Env-var contract (`eye_of_tiger.py`)
Sizing helper (`broker/orders.py`)
Wire points
Dashboard fields (`/api/state`)
Validation
Rollback

§21. Pre-market Readiness Check (v6.11.1)

Cron schedule
Script: `scripts/premarket_check.py`
14 checks performed (in order)
Output artifact
Exit codes
Future hook

Callable Python API

TradeGenius — System Architecture

Version: v6.14.0 · May 2026 -- Volume gate fix end-to-end + cancel-first entry guard.

v6.14.0 volume gate plumbing (issue #354): The Section II.1 55-day Institutional Oomph baseline never populated correctly from production SIP feed bars because three structural data bugs compounded: (a) `ingest/algo_plus.py` hardcoded `et_bucket=None` on both REST backfill and websocket `_on_bar` paths so every persisted bar lacked the per-minute key the baseline reader keys on; (b) the bar archive persisted only `iex_volume` (~3% of total US equity volume; SIP-sourced bars carry zero in this field) and never persisted the SIP-aggregate `volume`; (c) only six days of bar history existed on disk vs. the 55-day lookback window. Fix shipped in three coordinated patches: `ingest/algo_plus.py` adds `_compute_et_bucket(ts)` helper (UTC -> RTH `"HHMM"` ET, returns None outside 09:30-16:00 ET) and populates both `et_bucket` and `total_volume` on every bar written to the archive; `bar_archive.py` adds `total_volume` to `BAR_SCHEMA_FIELDS` between `close` and `iex_volume`; `volume_bucket.py` reads `bar.get("total_volume")` first with `iex_volume` fallback (so legacy IEX-era bars on disk still contribute) and skips zero-volume entries entirely. The one-shot `tools/backfill_bar_archive.py` populates `/data/bars` from the canonical 84-day SIP archive at `/home/user/workspace/canonical_backtest_data/84day_2026_sip/replay_layout/` with the v6.14.0 field shape (`volume` -> `total_volume`, recomputed `et_bucket`, `n` -> `trade_count`, `vw` -> `bar_vwap`, `_feed` -> `feed_source`); idempotent on `ts` against existing JSONL. Rollout sequence: merge -> auto-deploy -> ssh-run backfill -> wait one RTH session -> verify `bb.days_available > 30` for all 12 prod tickers -> THEN flip `VOLUME_GATE_ENABLED=true`. The data plumbing fix and the on/off toggle are deliberately shipped in separate releases. End-to-end coverage in `tests/test_v6_14_0_volume_gate_e2e.py` (6 tests) asserts the chain produces a numeric `ratio_to_55bar_avg` (the v15-spec field name the 10:00 ET conditional path keys on), `total_volume` wins over `iex_volume` when both are present, and zero-volume bars do not poison the rolling mean.

v6.14.0 cancel-first entry guard (issue #357): On 2026-05-04 13:56:54 UTC, an Alpaca paper order for GOOG was rejected with `40310000 "potential wash trade detected"` referencing existing order `d93911e6`. The bot had covered a short at 13:56:52.443, then submitted a new short entry at 13:56:53.658 -- the covering order was still working in Alpaca's book when the new opposite-side entry hit. v6.14.0 adds a `_cancel_first_guard(ticker, side)` helper in `broker/orders.py` invoked at the top of `execute_breakout()` immediately after the v6.11.13 `POST_EXIT_SAME_TICKER_COOLDOWN_SEC` check. Flow per entry: query Alpaca via `tc.get_orders(filter=GetOrdersRequest(status=OPEN, symbols=[ticker]))`; identify any open order whose side opposes the new entry (open SELL when entering LONG; open BUY when entering SHORT); cancel each via `tc.cancel_order_by_id`; poll `tc.get_order_by_id` every 100 ms until each id reaches a terminal status (`canceled`, `filled`, `expired`, `rejected`, `replaced`, `done_for_day`) or `CANCEL_ACK_TIMEOUT_MS` (default 1500 ms via `eye_of_tiger.CANCEL_ACK_TIMEOUT_MS`) elapses. On timeout the entry is skipped with a `[V6140-CANCEL-FIRST]` warning rather than fired into a known wash-trade race. Any unexpected broker error (`get_orders` raises, SDK missing, etc.) is logged and the helper fails open -- the v6.11.13 cooldown remains as defense-in-depth so the entry path is never bricked by a transient hiccup. The helper is a no-op (returns True) when no Alpaca creds are configured (replay, smoke tests, missing env), giving byte-identical replay output and a verified $0.00 P/L delta on three sample dates of the 84-day SIP corpus. The v6.13.0 cancel-first spec at `specs/v6_13_0_cancel_first_entry.md` proposes an `_open_broker_orders` registry maintained via Alpaca trade-updates websocket; v6.14.0 takes the simpler synchronous-poll path because no `TradingStream` wiring exists in the codebase today. Direct round-trip is 30-150 ms; for the 12-ticker prod universe firing roughly one entry per minute the additional API load is negligible. Coverage in `tests/test_v6_14_0_cancel_first.py` (7 tests) asserts no-op when no broker, no-cancel when no opposing orders, same-side orders left alone, opposing orders cancelled and polled until terminal, ack-timeout returns False so caller skips, multi-order cancel waits for all acks, and broker errors fail open. Once cancel-first is verified live for one full RTH session with zero

40310000 rejects, `POST_EXIT_SAME_TICKER_COOLDOWN_SEC=0` can be set via Railway env to recover the + $13 K backtest delta without a code change.

v6.11.0 baseline (still active for short-side amplification): C25 SPY Regime-B Short Amplification ships unchanged from v6.11.0 through v6.14.0. See section below.

Backtest cache layer (v6.9.2): L1: Per-day Parquet bar cache (`backtest/bar_cache.py`) -- layout `.cache_v2/<TICKER>/<YYYY-MM-DD>.parquet` ; one file per (ticker, date); ZSTD-3; SHA-256 cache key over (path, mtime_ns, size); `get_bars()` is a drop-in for `load_day_bars` . v6.9.0 used a single file per ticker containing all 84 days in one row group, which forced a 2.1 MB full-file scan for every 1-day request (12-15x regression). v6.9.2 opens exactly one ~167 KB file per request. Cache version bumped v1 \u2192 v2; prior `.cache_v1/` files are automatically invalidated and rebuilt on first access. In-process LRU (`functools.lru_cache(maxsize=4096)`) on `_lru_read_bars` and `_lru_read_indicators` : after the first disk read per (ticker, date), all subsequent calls within the same process return in ~5 \u00b5s. A process-level `_CACHE_VERIFIED` set skips repeated `os.stat()` freshness checks after the first validation, eliminating a 2.5 ms overhead per call. L2: Indicator precompute cache (`backtest/indicator_cache.py`) -- per-day layout `.indcache_v2/<TICKER>__<params_hash>/<YYYY-MM-DD>.parquet` ; covers ATR14/20, EMA9/20/50, VWAP, OR-5m/OR-30m, premarket H/L/range, session boundary. Warm LRU: 91x faster than JSONL on 5dx5t. Warm replay: 192x per-pass on 84dx25t sweep. Wave 2 (11-run C4+C5+C7+C10 sweep) target wall: <=15 min.

Wave 2 results (May 2026): C5 12:00 cutoff and C10 60-min short cooldown shipped. C4 ATR-OR-break filter no-ship (negative lift across all tested K). C7 post-gate damping deferred (feature was speced but never implemented; requires re-design before sweep).

v6.0.x release timeline (backfill, May 2026): the v6.0.x line is a stability series after the v5.x algorithm consolidation. Each release in this series is small, focused, and ships hot to prod. The full per-release WHY/WHAT/TESTS narrative lives in `CHANGELOG.md` — this document is the runtime architecture surface.

- **v6.0.0** (2026-04-28) — dual-broker (Val + Gene) executor split. `executors/base.py` introduces `BaseExecutor` (broker-agnostic reconciliation, position recording, idempotent close, and the periodic + boot reconcile loops). `executors/val.py` and `executors/gene.py` thin-wrap the per-broker config. Live engine ticks fan out to every enabled executor; persistence keys by `(executor_name, ticker)` .
- **v6.0.1** (2026-04-29) — QBTS removed from `TICKERS_DEFAULT` . Outstanding: prior open QBTS positions on Val/Gene executors continue to be managed until they exit; new entries are blocked.
- **v6.0.2** (2026-04-29) — SMA stack persistence rehydration: `bars["sma_stack"]` re-warmed from the daily-bars cache on boot so Phase 1 Sovereign doesn't see `None` for the first 5 minutes after a deploy. (Outstanding: an unrelated SMA-stack-None edge case is on the carry-forward list; Val deferred the hotfix.)
- **v6.0.3** (2026-04-30) — sentinel forensic-emission hotfix. `[SENTINEL]` lines now include `pos_qty` / `pos_side` so Alarm A/B/C audits don't have to cross-reference `executor_positions` to reconstruct the sign of the position.
- **v6.0.4** (2026-05-01) — sentinel persistence rehydration hotfix. `pos["sentinel_state"]` was lost across boot, causing Alarm F's `bars_seen` counter to reset to 0 every redeploy. Fix: serialize `TrailState` into `executor_positions.trail_state_json` on every stop write; rehydrate on `_load_positions_from_db` boot.
- **v6.0.5** (2026-05-01) — Yahoo trailing-None hotfix + Alpaca-IEX promoted to primary 1m bars source. `trade_genius.fetch_1min_bars` reordered: Alpaca primary, Yahoo fallback, dual-source-failure `[SENTINEL][CRITICAL]` + Telegram. `broker/positions.py:_run_sentinel` walks back over trailing `None` to find the most recent finite `last_1m_close` so Alarm F's gate (`last_1m_close is not None`) stops silently skipping.

- **v6.0.6** (2026-05-01) — Dashboard fix: TRAIL badge now fires on Alarm-F chandelier. `dashboard_server.py` exposes `chandelier_stage` (0=INACTIVE, 1=BREAKEVEN, 2=CHANDELIER_WIDE, 3=CHANDELIER_TIGHT) on every position row; `dashboard_static/app.js` ORs `p.trail_active` with `chandelier_stage >= 1` for the badge.
- **v6.0.7** (2026-05-01) — post-action reconcile race fix + iPhone Pro Max (430 px) mobile UI overflow fix + stale-test cleanup. See "Reconcile state machine — v6.0.7" subsection below for the new `expect=` parameter and the `_get_open_position_settled` polling helper.
- **v6.0.8** (2026-05-01, this release) — session-state SQLite persistence + mobile expanded-permit-row width fix. Apr 30 had 9 Railway redeploys during RTH; the in-memory `_v570_session_hod` / `_v570_session_lod` / `_v570_strike_counts` / `_v570_daily_realized_pnl` / `_v570_kill_switch_latched` reset to empty on every redeploy, causing NVDA strike 2/3 to fire off shallow LODs (real session LOD 197.38; post-redeploy prints at 198.57 and 198.20 registered as fresh `lod_break` s vs `prev_lod=None`). Two new SQLite tables `session_state` (per ticker per ET-date) and `session_globals` (scalars) in `persistence.py` mirror the dicts on every mutation; `_v570_rehydrate_from_disk(today)` in `trade_genius.py` seeds the dicts on the first call to `_v570_reset_if_new_session()` per ET date in this process. Idempotent across the day via `_v570_rehydrated_for_date` . ET-date keying so a stale row from yesterday is naturally pruned on the day-rollover transition. Mobile fix: `dashboard_static/app.css` adds an `@media (max-width: 480px)` block that constrains `.pmtx-detail-row td` with `max-width: 100vw` + `box-sizing: border-box` + `overflow-wrap: anywhere` , so the colspan'd td no longer renders 653 px wide on a 453 px viewport and component-card descriptions wrap inside the visible area. See "Session-state persistence — v6.0.8" subsection below.
- **v6.0.8.1** (2026-05-01) — mobile expanded permit row width hotfix. v6.0.8's `@media (max-width: 480px)` rule set `max-width: 100vw` on `.pmtx-detail-row td` , but `max-width` is ignored by the table layout algorithm. Switched the expanded-row `<td>` to `display: block` + `position: sticky`
- `width: calc(100vw - 24px)` scoped to `tr.pmtx-detail-row.pmtx-detail-open > td` . CSS-only.
- **v6.1.0** (2026-05-01, this release) — P&L recovery bundle: three algo upgrades shipped together after the 5/1 TRUE backtest closed at −$8.32 against ~$143/share of available swing across the universe. (1) **ATR-scaled trailing stop with profit-protect ratchet** in `engine/sentinel.py:check_alarm_a_stop_price` — replaces fixed-cents trail with 3-stage ATR distance (1.0× → 1.5× → 50% of peak open profit) plus a 0.3× ATR floor; new `atr5_1m(bars)` helper in `indicators.py` . (2) **Two-bar EMA-cross confirmation + lunch-chop suppression** in `check_alarm_b` — per-position counter `_ema_cross_pending` , requires two consecutive cross-against bars before firing, blocks exit between 11:30–13:00 ET. (3) **ATR-normalized OR-break entry gate + late-OR window** in `trade_genius.py` / `broker/orders.py` — break threshold = `0.6 × pre_market_range_atr` (replaces fixed cents); new late-OR window 11:00–12:00 ET fires only if standard 9:30–10:30 OR-break never triggered. All three behind feature flags (`_V610_ATR_TRAIL_ENABLED` , `_V610_EMA_CONFIRM_ENABLED` , `_V610_LUNCH_SUPPRESSION_ENABLED` , `_V610_ATR_OR_BREAK_ENABLED` , `V610_LATE_OR_ENABLED`) for instant rollback. 655 tests pass. Validate against weekly shadow data before flipping any flag to default-off. See "v6.1.0 — P&L recovery bundle" subsection below.
- **v6.1.1** (2026-05-01) — dashboard-only patch: surface v6.1.0 strategy in expanded matrix cards. Alarm F card shows active ATR multiple + dollar width when WIDE/TIGHT is armed; Phase 2 Boundary card shows OR-only / k×ATR / late-OR; Phase 3 Authority card shows EMA 2-bar / lunch suppression. New `/api/state.v610_flags` block. Per minor-release rule, no ARCHITECTURE / PDF update on this patch.
- **v6.2.0** (2026-05-01) — entry-gate loosening: three forensics-backed relaxations of the entry matrix to recover trades that the 7,113-row entry-gate forensics sweep showed would have profited. (A) **Local override OR-break leg** in `engine/local_weather.py` — `evaluate_local_override` now admits a 4th leg: if the

ticker has cleared OR by `k×ATR_pm` (k = `V620_LOCAL_OR_BREAK_K = 0.25`), the divergence override fires even when the QQQ permit closes. Plumbed via `or_high`, `or_low`, `atr_pm` kwargs from `broker/orders.py`. Reason `"open_or_break"` distinguishes the new path. (B) **Time-conditional 1-bar boundary hold pre-10:30 ET** in `v5_10_1_integration.py:evaluate_boundary_hold_gate` — when wall-clock ET < `V620_FAST_BOUNDARY_CUTOFF_HHMM_ET = "12:00"`, the gate uses `required_closes=1`; post-cutoff reverts to the spec 2-bar (`eot.BOUNDARY_HOLD_REQUIRED_CLOSES`). The spec constant is unchanged — only the gate caller relaxes — so `tests/test_spec_v15_conformance.py:96` (`== 2` assertion) still passes. (D) **Generic ENTRY_1 DI threshold 25 → 22** in `eye_of_tiger.py:30` — not symbol-specific; based on TSLA 5/1 (44 di_5m rejections, 100% would-have-profited) and the same gate blocking AVGO/GOOG/AMZN. Dashboard surfaces all three via a new `/api/state.v620_flags` block + card-text suffixes on Local Weather (`· OR+0.25×ATR leg`), Boundary (`· 1-bar pre-10:30` / `· 2-bar hold`), and Momentum (`· DI≥22`) tiles — mirrors the v6.1.1 v610_flags pattern (zero new DOM, zero new endpoints). All three loosenings behind module-level `V620_*` flags for instant rollback. See "v6.2.0 — Entry-gate loosening" subsection below.

- **v6.3.0** (2026-05-01, this release) — Sentinel B noise-cross filter. The Apr 27 – May 1 weekly backtest split MtM-adjusted P&L by sentinel: A (per-position $ stop / 1m flash) **+$258.97 / 70% wins** vs B (5m close-vs-9-EMA cross) **−$277.52 / 6% wins**. Sixteen of 17 B losers closed at adverse −0.05% to −0.37% — all within typical 1m noise. Fix: gate the v6.1.0 2-bar EMA-cross exit on a minimum adverse move of `V630_NOISE_CROSS_ATR_K × last_1m_atr` (k = 0.10) from entry. New constants in `engine/sentinel.py`: `V630_NOISE_CROSS_FILTER_ENABLED = True`, `V630_NOISE_CROSS_ATR_K = 0.10`. `check_alarm_b` signature gained `entry_price`, `current_price`, `last_1m_atr` kwargs; when all three are present and atr > 0, the v6.1.0 stateful path sits out (does **NOT** reset the counter) when adverse < k×ATR. The position waits for either price to confirm the move or the cross condition to flip naturally. `evaluate_sentinel` threads the new params; `broker/positions.py` already passed them so the fix flows to prod with no caller change required. Dashboard surface: new `/api/state.v630_flags` block (`{noise_cross_filter_enabled, noise_cross_atr_k}`); Local Weather card and the B Trend Death sentinel-strip cell append a `· noise≥0.10×ATR` suffix. See "v6.3.0 — Sentinel B noise- cross filter" subsection below.
- **v6.3.1** (2026-05-01) — wire `position_id + now_et` into `evaluate_sentinel` so v6.1.0 stateful counter actually keys per-position. Patch only — no architecture changes.
- **v6.3.2** (2026-05-01) — three small backtest infra fixes that were warping every backtest run since v5.0: (1) add `v5_lock_all_tracks` (was referenced by EOD/daily-breaker but never defined; calls were silently swallowed by try/except); (2) move `after_close` from 15:55 to 16:00 ET so the engine manages the full final 5-minute bucket; (3) derive `now_ts` from harness clock `_tg()._now_et()` instead of `_time.time()` so Alarm A velocity is deterministic in backtest, also fixes a v6.3.1 typo (`_tg().now_et()` → `_tg()._now_et()`) leaving v6.1.0 lunch suppression dead. Patch only — no architecture changes.
- **v6.4.0** (2026-05-01, this release) — disable Sentinel B by default + tighten Alarm F Chandelier multipliers. Apr 27 – May 1 replay-only sweep across 6 configurations × 5 days replaced Sentinel B with progressively tighter F-trail variants. Winning config (`disable_b_tight_F`): **+$1,049.43 vs +$831.50 baseline (+ $217.93/wk, +26%), 60 pairs vs 73, win rate 45.2% → 61.7%.** The win mechanism is *not* extra F-EXIT firings (only one F-EXIT fired all week); it is letting trades survive to the per-position $ stop (Alarm A) without B's EMA-cross prematurely cutting them. Avg trade $11→$17, avg hold 41m→62m, AVGO swung +$140 (worst baseline → positive). Two-knob change in `engine/`: (1) `ALARM_B_ENABLED: bool = False` constant in `engine/sentinel.py` — `evaluate_sentinel` now skips the `check_alarm_b` call entirely when False (no log lines, no state mutation); the v6.1.0 stateful counter and v6.3.0 noise-cross filter remain in code, zero-cost when B is off and ready if a future release flips the flag back on. (2) Alarm F `WIDE_MULT: 2.0 → 1.5` and `TIGHT_MULT: 1.0 → 0.7` in `engine/alarm_f_trail.py`. Stage 2 arms sooner once B no longer cuts trades early; `MIN_BARS_BEFORE_ARM=3` still guards the first three bars from entry-bar noise.

Dashboard surface: new `/api/state.v640_flags` block (`{alarm_b_enabled, chandelier_wide_mult, chandelier_tight_mult}`); B Trend Death sentinel-strip cell renders DISABLED state when B is off; Local Weather card suffix swaps `· noise≥0.10×ATR` for `· chand 1.5/0.7` so operators see the active trail config instead of a stale noise threshold. EOD harness fix (`backtest/replay_v511_full.py`) ports up the `eod_close@15:49 ET` invocation that the smoke-test scheduler suppresses, so replay tail-positions no longer persist as `stuck-EOD` . See "v6.4.0 — Disable Sentinel B + tighten Chandelier" subsection below.

v6.4.0 — Disable Sentinel B + tighten Chandelier

Why. A 6-config × 5-day replay sweep (Apr 27 – May 1) using the v6.3.2 baseline showed Sentinel B continued to bleed P&L even after the v6.3.0 noise-cross filter:

Config	Week P&L	Pairs	WR	Δ vs baseline
baseline (v6.3.2)	+$831.50	73	45.2%	—
disable_b	+$998.09	60	61.7%	+$166.59
b_3bar / b_5bar	+$831.50	73	45.2%	$0 (no-op)
disable_b_wide_F (2.5/1.5)	+$897.75	59	59.3%	+$66.25
disable_b_tight_F (1.5/0.7)	**+$1,049.43**	**60**	**61.7%**	**+$217.93**

The `b_3bar` / `b_5bar` rows were byte-identical to baseline because the v6.1.0 stateful EMA-cross path hardcodes `count >= 2` and ignores the `confirm_bars` argument; the legacy path treats any value `>= 2` identically. The only meaningful lever is a full B disable.

What changes.

1. `engine/sentinel.py` — new module-level constant `ALARM_B_ENABLED: bool = False` , mirroring the existing `ALARM_C_ENABLED` / `ALARM_D_ENABLED` pattern. The Alarm B block in `evaluate_sentinel` now reads:

```
python if ALARM_B_ENABLED: b_fired = check_alarm_b(...) result.alarms.extend(b_fired)
```

No log lines, no `_ema_cross_pending` state mutation, no forensic-emission overhead when disabled. `check_alarm_b` , the noise-cross filter, and `_ema_cross_pending` all remain in the code so a future release can flip the flag back on without a rewrite. Tests can monkeypatch the constant per case.

1. `engine/alarm_f_trail.py` — `WIDE_MULT: 2.0 → 1.5` and `TIGHT_MULT: 1.0 → 0.7` . Same Stage-2/Stage-3 transition logic; only the multipliers change. `MIN_BARS_BEFORE_ARM=3` keeps the first three bars protected.

2. `dashboard_server.py` `/api/state` — new `v640_flags` block next to `v610_flags` / `v620_flags` / `v630_flags` :

```
json {"alarm_b_enabled": false, "chandelier_wide_mult": 1.5, "chandelier_tight_mult": 0.7}
```

`feature_flags_block` also gains `alarm_b_enabled` so the KPI pill row can show the new toggle alongside C/D/E/F.

1. `dashboard_static/app.js` — `v640Flags` pickup at the same site as `v630Flags`. The B Trend Death sentinel-strip cell renders the existing `idle` (dim) theme with a clear `DISABLED · ALARM_B_ENABLED=false` value when off, instead of stale close/EMA9 deltas. The Local Weather card suffix swaps: `· noise≥0.10×ATR` (when B was on with the v6.3.0 filter) becomes `· chand 1.5/0.7` (when B is off and the Chandelier trail is the new primary B-side exit signal).
2. `backtest/replay_v511_full.py` — explicit `eod_close()` invocation when sim-clock crosses `15:49 ET`. Production fires this via the scheduler thread; `SSM_SMOKE_TEST=1` disables the scheduler, so the harness loop must call it directly. Without this, any position still open at `end_dt` was left in `tg.positions` and persisted as `stuck-EOD`, distorting every replay's tail. Aligns with `broker.lifecycle.EOD_FLUSH_ET`.

Why not noisier signals? The sweep tested two `confirm_bars` variants (3-bar, 5-bar) which were no-ops because of the dead `count >= 2` path; the only working `b` lever is a full disable. Tightening F further (`disable_b_wide_F` = 2.5/1.5) underperformed `1.5/0.7` by **−$152/wk**, indicating the Stage-2 chandelier benefits from being closer to price rather than further from it once Sentinel B is no longer chopping winners.

Rollback. Flip `ALARM_B_ENABLED = True` and revert `WIDE_MULT/TIGHT_MULT` to `2.0/1.0` in two file edits, no migration. Dashboard degrades gracefully: pre-v6.4.0 deploys render as legacy because every JS reader uses `(d.v640Flags && ...)` guards.

v6.3.0 — Sentinel B noise-cross filter

Why. The Apr 27 – May 1 weekly backtest (`/home/user/workspace/v620_week_backtest/report.md`) split realised P&L by sentinel and found Sentinel B was the bleed:

Sentinel	Trades	P&L	Win rate
A (per-position $ stop / 1m flash)	20	**+$258.97**	**70%**
B (5m close vs 9-EMA cross)	17	**−$277.52**	**6%**

Forensic review of the 16 EMA-cross losers found every one closed at adverse moves of −0.05% to −0.37% — entirely within 1m noise. The 5m close ticked under the 5m 9-EMA by a hair, the cross fired the exit, and price recovered the next minute or two.

What. Filter logic is in `engine/sentinel.py`:

`V630_NOISE_CROSS_FILTER_ENABLED: bool = True V630_NOISE_CROSS_ATR_K: float = 0.10`

Inside the v6.1.0 stateful path of `check_alarm_b`, after the 2-bar count check passes, the function now requires:

- LONG: `(entry_price - current_price) ≥ k × last_1m_atr`
- SHORT: `(current_price - entry_price) ≥ k × last_1m_atr`

When the threshold is not met the function returns `[]` **without** resetting `_ema_cross_pending[position_id]`. This is the load- bearing behaviour: the position waits, and the cross still fires the moment price either confirms the move or reverts past the cross threshold (which then resets the

counter via the existing False- branch). The filter is bypassed when any of `entry_price`, `current_price`, or `last_1m_atr` is None or when ATR ≤ 0 — this keeps cold-start / stale-data paths safe.

k = 0.10 sizing. The 16 losers averaged 0.19% adverse at exit; a typical 1m ATR on the watched megacaps is 0.30–0.50%. 0.10 × ATR ≈ 0.03–0.05% adverse → admits roughly the bottom 80% of noise- crosses while the structural conviction crosses still fire normally.

Plumbing. `evaluate_sentinel` was updated to thread `entry_price`, `current_price`, `last_1m_atr` through to `check_alarm_b`. `broker/positions.py:_run_sentinel` already calls `evaluate_sentinel(entry_price=entry_p, current_price=current_price, last_1m_atr=last_1m_atr, ...)` so the fix flows to production code automatically.

Dashboard surface (mirrors the v6.1.1 / v6.2.0 patterns):

- `dashboard_server.py:/api/state` payload gained a `v630_flags` block alongside `v620_flags`. Reads the constants via `getattr(engine.sentinel, ...)` with safe defaults so older deploys / partial syncs degrade to legacy text.
- `dashboard_static/app.js` (Local Weather card, ~line 956): appends `· noise≥0.10×ATR` to the value text alongside the existing v6.2.0 OR-break leg suffix.
- `dashboard_static/app.js` (B Trend Death sentinel cell, ~line 3076): appends the same suffix to the `close=… / ema=… / Δ=…` value, so on every expanded titan card the operator can see the active threshold without diving into /api/state.

Tests: `tests/test_sentinel_v630_noise_cross.py` (10 cases — LONG/SHORT, above/below threshold, None/zero ATR bypass paths, counter-no-reset semantics across consecutive blocked bars).

Reconcile state machine — v6.0.7

`executors/base.py:_reconcile_position_with_broker(client, ticker, ...)` takes a new keyword `expect: str = "any"`:

expect	Caller	Behaviour on mismatch
`"any"`	Periodic reconcile (~30 s), boot reconcile	Single-shot, legacy v5.25 behaviour preserved.
`"present"`	`_on_signal` post- `ENTRY_LONG` / `ENTRY_SHORT`	Retry if broker returns 40410000 inside the 4 s grace; if still flat after grace, leave the local row in place (do NOT delete).
`"flat"`	`_on_signal` post- `EXIT_LONG` / `EXIT_SHORT`	Retry if broker still has the position inside the 4 s grace; if still present after grace, leave it untracked (do NOT graft a phantom POST_RECONCILE row).

The grace window is per-ticker, stamped on every successful ENTRY (`_record_position`) or EXIT (`_close_position_idempotent`) via `self._last_action_ts[ticker] = time.monotonic()`. Module constants: `RECONCILE_GRACE_SECONDS = 4.0`, `RECONCILE_RETRY_SLEEP = 0.6`. The polling helper `_get_open_position_settled` retries `get_open_position` while the response disagrees with `expect` AND the grace is open; after grace, it returns the last response so the caller can decide whether to act.

Why this matters: Alpaca REST `get_open_position` is eventually consistent for ~1–2 s after a fill. Pre-v6.0.7, `_on_signal`'s post-action reconcile mistook this window for true divergence and either (a) deleted the just-recorded local row (post-ENTRY race) or (b) grafted a phantom POST_RECONCILE row over a just-closed position (post-EXIT race). The race surfaces on the next periodic / boot reconcile as a "true divergence" Telegram with `source='RECONCILE', stop=None, trail=None`.

Session-state persistence — v6.0.8

`trade_genius.py` keeps five pieces of session-scoped state at module scope: `_v570_strike_counts` (per ticker entry counter), `_v570_session_hod` / `_v570_session_lod` (per ticker session high/low), `_v570_daily_realized_pnl` (running cumulative P&L), and `_v570_kill_switch_latched` (daily-loss floor breach flag). Pre-v6.0.8 these were Python dicts/floats living only in the running process. Every Railway redeploy started a fresh process and the values all came up empty, regardless of whether the real ET session had already established HOD/LOD or fired strike 1.

Apr 30 2026 had 9 redeploys during RTH and the failure mode showed itself on NVDA: real session LOD = 197.38 set at 14:18 ET; redeploy at 14:32 ET wiped `_v570_session_lod["NVDA"]`; print at 14:35 ET = 198.57 hit `_v570_update_session_hod_lod` with `prev_lod = None`, registered as a fresh `lod_break` (the gate is `prev_lod is not None and px < prev_lod` so when prev is None we seed but don't break — but the strike-2 entry path was reading the *post-update* state machine and treating the seed itself as a break trigger via the surrounding gate logic). Strike 2 fired at a price 1.19 above the real LOD; strike 3 followed off 198.20 four minutes later off the same redeploy-blank state. Both should have been blocked.

v6.0.8 adds two SQLite tables in `persistence.py`: `session_state(ticker, et_date, session_hod, session_lod, strike_count, last_updated_utc)` keyed on `(ticker, et_date)`, and `session_globals(key, et_date, value_real, value_int, last_updated_utc)` keyed on `(key, et_date)`. ET-date keying means a stale row from yesterday is naturally ignored on rehydrate when today != stored et_date.

Six new failure-tolerant helpers expose UPSERT-with-COALESCE writes plus per-date reads and prunes: `save_session_state`, `load_session_state_for_date`, `prune_session_state`, `save_session_global`, `load_session_globals_for_date`, `prune_session_globals`. Every write helper wraps the sqlite3 call in try/except and logs+swallows on failure — a corrupt or unwritable state.db can never raise into the trading path.

`trade_genius.py` adds `_v570_rehydrate_from_disk(today)` which reads both tables for `today` and seeds the in-memory v570 dicts. `_v570_reset_if_new_session()` calls it on the FIRST entry per ET date in this process; subsequent calls within the same ET date short-circuit via `_v570_rehydrated_for_date == today`. On the day-rollover transition (`_v570_rehydrated_for_date != today and not empty`), yesterday's rows are pruned via `persistence.prune_session_state(today)` + `prune_session_globals(today)`.

Three call sites mirror state TO disk: - `_v570_record_entry()` after every increment (`save_session_state(strike_count=new_n)`), - `_v570_update_session_hod_lod()` only when HOD or LOD actually moved (avoids one disk write per quote on quiet prints), - `_v570_record_trade_close()` after every cumulative-P&L update (writes both `daily_realized_pnl` and the `kill_switch_latched` int to `session_globals`).

Net effect on the Apr 30 NVDA scenario: redeploy at 14:32 ET -> fresh process boots -> dicts come up empty -> first call into `_v570_reset_if_new_session()` hits `_v570_rehydrate_from_disk("2026-04-30")` which

seeds `_v570_session_lod["NVDA"] = 197.38` from the row the 14:18 ET update wrote -> the 14:35 ET 198.57 print is correctly NOT a fresh `lod_break` (`prev_lod=197.38, px=198.57`). Strike 2/3 stays blocked.

Mobile expanded-permit-row width fix (separate workstream in the same release): the `<td>` of `.pmtx-detail-row` uses colspan across the full Permit Matrix table and previously rendered 653 px wide on a 453 px iPhone 13 viewport, clipping every component-card description at the right edge. The new `@media (max-width: 480px)` block in `dashboard_static/app.css` constrains the td with `max-width: 100vw` , `box-sizing: border-box` , and `overflow-wrap: anywhere` , and tightens its descendants (`min-width: 0` , `max-width: 100%`) so block-level children inside cannot push the row past the visible viewport.

---

v5.14.0 retirement (READ FIRST): the shadow strategy and its supporting surfaces were removed in v5.14.0. Specifically:

The remainder of this document still describes the v5.14.0 baseline. The v6.0.x deltas above are layered on top; for any conflict, the sections above and `CHANGELOG.md` are authoritative.

Version (legacy header preserved for reference): v5.14.0 · April 2026 — **Shadow retirement Repo:** `valira3/stock-spike-monitor` · **Service:** `tradegenius.up.railway.app`

v5.14.0 retirement (READ FIRST): the shadow strategy and its supporting surfaces were removed in v5.14.0. Specifically: - `shadow_pnl.py` module (deleted) — the in-process `ShadowPnL` tracker is gone - `volume_profile.SHADOW_CONFIGS` tuple + `evaluate_g4_config` (deleted) — only the live-engine `evaluate_g4` path remains - `[V510-SHADOW][CFG=...]` , `[V520-SHADOW-PNL]` , and the `[SHADOW DISABLED]` warning (renamed to `[VOLFEED DISABLED]`) - `shadow_positions` SQLite table (`DROP TABLE IF EXISTS` runs on boot) - Dashboard Shadow tab + `/api/shadow_charts` endpoint + `shadow_pnl` key in `/api/state` (the `shadow_data_status` field was renamed to `volume_feed_status`) - Saturday weekly cron `873854a1` + `scripts/saturday_weekly_report.py` - `backtest/replay.py` (the SHADOW_CONFIGS replay engine); `backtest/replay_v511_full.py` remains

The forensic capture log emitters `[V510-CAND]` , `[V510-FSM]` , `[V510-MINUTE]` , `[V510-VEL]` , `[V510-DI]` , `[V510-VOLBUCKET]` , `[V510-BAR]` , and the `/data/bars/YYYY-MM-DD/<TICKER>.jsonl` bar archive ALL remain live so future backtests can be rebuilt against `trade_log.jsonl` (live entries) and `executor_positions` (open positions). The trading-logic sections below still reference shadow surfaces — treat those references as historical context describing the v5.0–v5.13 era.

Source of truth (v5.11.0): `specs/canonical/eye_of_the_tiger_gene_2026-04-28c.md` (Gene's authoritative rev 28c spec — the 28c spec is unchanged from v5.10.0; v5.11.0 is a refactor release), `eye_of_tiger.py` (pure-function spec gates, Sections I–VI), `engine/` (per-tick orchestration extracted from `trade_genius.py` in v5.11.0), `volume_bucket.py` (Section II.1 Institutional Oomph baseline), `tests/test_eye_of_tiger.py` , `tests/test_volume_bucket.py` , `tests/golden/` (byte-equal harness gating engine moves), `trade_genius.py` (bot lifecycle: WebSocket / broker / Telegram / persistence / dashboard). Legacy implementation surfaces (`tiger_buffalo_v5.py` , `volume_profile.py` , `bar_archive.py` , `persistence.py` , `dashboard_server.py` , `dashboard_static/{app.js,app.css,index.html}` , `backtest/`) remain for infrastructure/observability; their *trading-logic* references below describe the v5.0–v5.9 algorithms that v5.10.0 superseded.

Strategy Compliance — v5.13.0 (Tiger Sovereign)

The v5.13.0 series adopts the Tiger Sovereign trading specification (`STRATEGY.md` , `tiger_sovereign_spec_v2026-04-28h`). The previous v5 strategy doc is archived at `docs/spec_archive/v5_strategy_pre_tiger_sovereign.md` .

Adoption is staged across an integration branch (`v5_13_0_integration`) so prod stays on **v5.12.0** until **PR 6** merges to `main` with the `BOT_VERSION` bump. PRs 1–5 land on the integration branch only.

v5.13.0 PR sequence

PR	Branch	Delivers	Behavior change
PR 1	`v5_13_0/pr1-spec-and-tests`	Adopt STRATEGY.md, archive v5 spec, add `tests/test_tiger_sovereign_spec.py` per-rule scaffold + `spec_gap` marker + `spec_gap_report.py` . Wire compliant subset into smoke.	**None.**
PR 2	`v5_13_0/pr2-sentinel-alarms`	`engine/sentinel.py` Alarm A (-$500 OR -1%/min) and Alarm B (5m close vs 9-EMA), wired into per-tick loop for long+short.	YES
PR 3	`v5_13_0/pr3-titan-grip-ratchet`	Titan Grip Harvest ratchet: OR_High +0.93% Stage 1 (sell 25%, stop to +0.40%); +0.25% micro-ratchet; OR_High +1.88% Stage 3 (sell 25%); 50% runner Stage 4. Mirror short side.	YES
PR 4	`v5_13_0/pr4-phase-2-gates`	`L-P2-S3` / `S-P2-S3` 55-day rolling per-minute volume baseline ≥100% gate; `L-P2-S4` / `S-P2-S4` two-consecutive-1m-close-above-OR-High (or below OR-Low).	YES
PR 5	`v5_13_0/pr5-timing-rules`	`SHARED-CUTOFF` 15:44:59 ET new-position cutoff; `SHARED-EOD` 15:49:59 ET EOD flush; `SHARED-HUNT` unlimited hunting until cutoff.	YES
PR 6	`v5_13_0/pr6-order-types-and-ship`	Order-type audit (LIMIT for harvest, STOP MARKET for defensive stops). `BOT_VERSION` 5.12.0 → 5.13.0. CHANGELOG. Merges integration → main.	YES

Rule-ID inventory

PR 1 introduces `tests/test_tiger_sovereign_spec.py` . Each rule ID below is asserted; rules already complied with by v5.12.0 PASS, rules not yet implemented FAIL with a `@pytest.mark.spec_gap("PR-N", "rule-id")` marker so the failure becomes the precise to-do list for PRs 2–5.

Rules already compliant in v5.12.0 (passing in PR 1)

- `L-P1-S1` — QQQ 5m price > 9-EMA (long permit).
- `L-P1-S2` — QQQ price > 9:30 anchor VWAP (long permit).
- `L-P3-S5` — 5m DI+ > 25 AND 1m DI+ > 25 (DI threshold matches `TIGER_V2_DI_THRESHOLD = 25`).
- `S-P1-S1` — QQQ 5m price < 9-EMA (short permit, mirror).
- `S-P1-S2` — QQQ price < 9:30 anchor VWAP (short permit, mirror).
- `S-P3-S5` — 5m DI- > 25 AND 1m DI- > 25 (DI threshold).

- `SHARED-CB` — Daily circuit breaker = -$1,500 (`DAILY_LOSS_LIMIT_DOLLARS = -1500.0`).

PR 1 NOT_IMPLEMENTED gaps (closed by PRs 2–5)

Rule ID	Gap	Closed by
`L-P2-S3` / `S-P2-S3`	Volume ≥ 100% 55-day rolling per-minute average	**PR 4**
`L-P2-S4` / `S-P2-S4`	Two consecutive 1m candles closed above OR_High (or below OR_Low)	**PR 4**
`L-P3-S6` / `S-P3-S6`	1m DI threshold > 30 AND fresh NHOD/NLOD for Entry 2	**PR 4** (verified during gates work)
`L-P4-A` / `S-P4-A`	-$500 OR -1%/min single-minute alarm → exit 100%	**PR 2**
`L-P4-B` / `S-P4-B`	5m candle CLOSE vs 5m 9-EMA → exit 100%	**PR 2**
`L-P4-C-S1` / `S-P4-C-S1`	OR_High +0.93% / OR_Low -0.93% Stage 1 harvest + stop to ±0.40%	**PR 3**
`L-P4-C-S2` / `S-P4-C-S2`	±0.25% micro-ratchet steps	**PR 3**
`L-P4-C-S3` / `S-P4-C-S3`	OR_High +1.88% / OR_Low -1.88% Stage 3 second harvest	**PR 3**
`L-P4-C-S4` / `S-P4-C-S4`	50% runner with continued ±0.25% ratchet	**PR 3**
`SHARED-CUTOFF`	New-position cutoff at 15:44:59 ET (currently 15:30 ET)	**PR 5**
`SHARED-EOD`	EOD flush at 15:49:59 ET (currently 15:59:50 ET)	**PR 5**
`SHARED-HUNT`	Unlimited hunting until 15:44:59 cutoff	**PR 5**
`SHARED-ORDER-PROFIT`	Profit-taking via LIMIT orders	**PR 6**
`SHARED-ORDER-STOP`	Defensive stops via STOP MARKET orders	**PR 6**

Production posture during the series

- `main` HEAD remains at v5.12.0 (`c562876`) — Railway deploys nothing until PR 6 merges.
- The integration branch is not deployed.
- `BOT_VERSION` is unchanged in PRs 1–5; the `version-bump-check` workflow only enforces the bump on PRs to `main` , so integration PRs use the `[skip-version]` token in the title for clarity.
- Smoke and synthetic harnesses run on each integration PR; PR 1 has zero behavior change so harness baselines hold.

Module map (post-v5.11.0)

v5.11.0 split the trading-rules engine out of `trade_genius.py` into a dedicated `engine/` package. The split is mechanical — **no algorithm change** — and was gated by the byte-equal `tests/golden/` harness on every PR. Spec source: see the v5.11.0 extraction spec carried in the project's spec set (mirrors `specs/v5_11_0_engine_extraction.md`).

File	Responsibility
`engine/__init__.py`	Re-exports the public engine surface so callers do `from engine import …`. Boot log line: `[ENGINE] modules loaded: bars, seeders, phase_machine, callbacks, scan`.
`engine/bars.py`	5-minute OHLC + EMA(9) aggregation. `compute_5m_ohlc_and_ema9` (formerly `_v5105_compute_5m_ohlc_and_ema9` in `trade_genius.py`) plus 1m/5m bar normalization helpers shared between prod and replay. Pure compute, no I/O.
`engine/seeders.py`	Pre-market warmup as a unit. `qqq_regime_seed_once` (was `_v590_qqq_regime_seed_once`), `qqq_regime_tick`, `seed_di_buffer` / `seed_di_all`, `seed_opening_range` / `seed_opening_range_all`. Touches Alpaca + bar-archive readers via the same env-flag toggles as before; the seed paths (archive → Alpaca → prior-session) are unchanged.
`engine/phase_machine.py`	Phase A → B-Layered → B-Locked → C-Extraction state machine. `phase_machine_tick(ticker, side, pos, bars)` is a pure decision function over `eye_of_tiger.evaluate_*` (the spec gates) — no broker/Telegram/db calls inside. Position state mutators that drive the lock-step transitions live here.
`engine/callbacks.py`	`EngineCallbacks` Protocol (`execute_entry`, `execute_exit`, `get_position`, `alert`, `now_et`, …). Names every side effect the engine performs so the engine itself can stay free of `trade_genius` imports for I/O. Prod passes a wrapper around the existing `trade_genius` functions; the replay/test harness passes a record-only mock that just appends to a trade list.
`engine/scan.py`	Per-minute `scan_loop()` over the universe — the orchestration body that was inlined inside `trade_genius.scan_loop`. Drives `engine.bars`, the seeders, the phase machine, and the spec gates against an injected `EngineCallbacks`. The `trade_genius.scan_loop` retained at the call site is now a thin shim that builds the callback impl and delegates.

Unchanged / canonical homes:

- `eye_of_tiger.py` — spec gates §I–§VI (Global Permit, Volume Bucket, Boundary Hold, DI/NHOD entry triggers, tick overrides, Triple-Lock stops, machine rules). Pure functions; v5.11.0 did not touch this file.
- `indicators.py` — DI(15), EMA, RSI, ATR, VWAP, spread math. Pure compute, called by `engine/seeders.py` and `engine/phase_machine.py`.
- `qqq_regime.py` — QQQ Index Shield logic (§I); the `engine/seeders.py` warmup wraps this module's seeding helpers but does not re-implement them.
- `volume_bucket.py` — §II.1 Institutional Oomph rolling 55-day per-(ticker, minute-of-day) baseline. Untouched by v5.11.0.

The `EngineCallbacks` Protocol seam. The point of the Protocol in `engine/callbacks.py` is to keep the per-tick decision path I/O-free inside `engine/`. Prod ships an `EngineCallbacks` impl that wraps the existing `trade_genius` broker / Telegram / position-store / clock helpers; replay and unit tests ship a `RecordOnlyCallbacks` impl that appends emitted entries/exits to a list and uses a synthetic clock. This is what makes the byte-equal harness in `tests/golden/` possible without ever touching network or db.

Migration shims (v5.11.0 only). The version-prefixed private names that callers in `trade_genius.py` historically used (e.g. `_v5105_compute_5m_ohlc_and_ema9`, `_v590_qqq_regime_seed_once`, `_v5105_phase_machine_tick`) are retained for one release as aliases that point at the public `engine.*` names. They will be deleted in v5.12.0; new code should `from engine import …` directly.

telegram_ui/ (post-v5.11.1)

v5.11.1 split the Telegram UI surface out of `trade_genius.py` into a dedicated `telegram_ui/` package. Pure presentation; no trading logic. `send_telegram` remains in `trade_genius.py` because it's the broker-side notification entry used by paper_state, error_state, and the scheduler.

File	Responsibility
`telegram_ui/__init__.py`	Re-exports the public UI surface. Boot log: `[TELEGRAM-UI] modules loaded: charts, commands, menu, runtime`.
`telegram_ui/charts.py`	Matplotlib builders for dayreport, equity curve, portfolio pie. Dayreport row formatters.
`telegram_ui/commands.py`	Synchronous command builders (`_log_sync`, `_replay_sync`, `_perf_compute`, `_price_sync`, `_proximity_sync`, `_orb_sync`, etc.). Construct text/keyboards; do not send.
`telegram_ui/menu.py`	Keyboards, callback handlers (`positions_callback`, `proximity_callback`, `monitoring_callback`), `_CallbackUpdateShim`, and `menu_callback` dispatch.
`telegram_ui/runtime.py`	Bot lifecycle: `_auth_guard`, `_set_bot_commands`, `send_startup_message`, `run_telegram_bot`.

broker/ (post-v5.11.2)

v5.11.2 split the broker / position-management surface out of `trade_genius.py`. This is real-money path code; the synthetic harness (`synthetic_harness/scenarios/`) exercises every function via `m.<name>(...)` dispatch. The v5.11.x deprecation aliases that mirrored this surface back into `trade_genius`'s namespace were purged in v5.12.0; callers (synthetic_harness, smoke_test, telegram_commands) now import the broker names directly from `broker.{stops, orders, positions, lifecycle}`. Strict layering: stops → orders → positions → lifecycle.

File	Responsibility
`broker/__init__.py`	Re-exports the public broker surface. Boot log: `[BROKER] modules loaded: stops, orders, positions, lifecycle`.
`broker/stops.py`	Stop-management helpers: breakeven, capped, ladder, retighten, `retighten_all_stops`.
`broker/orders.py`	Order execution: `check_breakout`, `paper_shares_for`, `execute_breakout`, `close_breakout`.
`broker/positions.py`	Per-tick position management: `manage_positions`, `manage_short_positions`, `_v5104_maybe_fire_entry_2`.
`broker/lifecycle.py`	Entry/exit dispatchers and EOD close: `check_entry`, `execute_entry`, `close_position`, `eod_close`, plus short-side counterparts.

executors/ (post-v5.12.0)

v5.12.0 split the per-executor (Val / Gene) shadow-book runtime out of `trade_genius.py` into a dedicated `executors/` package. The base class owns the Alpaca-paper-keys handshake, signal-bus subscription, and per- executor error rings; the subclasses just set `NAME` / `ENV_PREFIX`. The v5.11.x deprecation aliases that bridged extracted helpers back into `trade_genius`'s namespace have been removed in v5.12.0 — callers (synthetic_harness, smoke_test, telegram_commands) now import from the canonical packages directly.

File	Responsibility
`executors/__init__.py`	Re-exports `TradeGeniusBase`, `TradeGeniusVal`, `TradeGeniusGene`. Boot log: `[EXEC] modules loaded: base, val, gene, bootstrap`.
`executors/base.py`	`TradeGeniusBase` — env-prefixed Alpaca paper key resolution, signal-bus subscription, error ring, dispatch hooks.
`executors/val.py`	`TradeGeniusVal(TradeGeniusBase)` — `NAME = "Val"`, `ENV_PREFIX = "VAL_"`.
`executors/gene.py`	`TradeGeniusGene(TradeGeniusBase)` — `NAME = "Gene"`, `ENV_PREFIX = "GENE_"`.
`executors/bootstrap.py`	`build_val_executor()`, `build_gene_executor()`, `install_globals()` — invoked by trade_genius's `__main__` block to publish the executor instances into both `trade_genius` and `telegram_commands` namespaces.

---

v5.10.0 — Project Eye of the Tiger (algorithm summary)

The v5.10.0 algorithm is organized into six numbered sections that match the canonical spec rev 28c. Pure-function evaluators live in `eye_of_tiger.py`; the rolling per-(ticker, minute-of-day) volume baseline lives in `volume_bucket.py`. `trade_genius.py` orchestrates state, time, data, and the signal bus around them.

Section I — Global Permit (QQQ Index Shield). Long permitted only when QQQ 5-minute close > 5m EMA(9) **and** QQQ current price > 9:30 ET session AVWAP; short is the strict mirror. Either anchor missing closes the gate (`reason: data_missing` or `anchor_misaligned`).

Section II — Ticker-Specific Permits (Entry-1 only). 1. **Volume Bucket (NEW):** rolling 55-trading-day average volume per `(ticker, HH:MM ET)` minute-of-day key, threshold ratio **1.00** (`current_minute_volume >= baseline` ⇒ PASS). With < 55 days of archive available, `gate=COLDSTART` and the ratio test is bypassed (PASS-THROUGH); a rate-limited `VOLBUCKET-COLDSTART` warning is emitted once per ticker until enough data accrues. 2. **Boundary Hold (NEW):** require **two consecutive 1-minute closes strictly outside** the 5-minute opening-range boundary (long: above `OR_High` ; short: below `OR_Low`). Earliest possible satisfaction is **09:36:00 ET**. Equality at the boundary breaks the hold.

Section III — Entry triggers and sizing. - **Entry 1:** fires when Section I open + Volume Bucket OK + Boundary Hold OK + DI(15) on 5m **> 25** + DI(15) on 1m **> 25** + a print at NHOD (long) or NLOD (short). Strict `>` everywhere; equality fails. Sizing: 50% of intended position. - **Entry 2 (NEW gating):** requires Entry 1 active **and** Section I still open at trigger time **and** a **fresh** NHOD/NLOD print (strictly beyond the highest/lowest mark since Entry 1) **and** a **crossing edge** on DI(15) 1m: `prev <= 30` then `now > 30` . Sustained DI > 30 without a fresh crossing does **not** re-trigger. Fires once per Entry-1 lifecycle. Sizing: remaining 50%.

Section IV — Tick overrides (highest priority). - **Sovereign Brake:** unrealized P&L `<= -$500` ⇒ flush position. - **Velocity Fuse:** intra-bar adverse move `> 1.0%` from the current 1-minute open (long: price below `open*(1 - 0.01)` ; short: mirror) ⇒ flush. Strict `>` so exactly 1.0% does not fire.

Section V — Triple-Lock stops (in priority order after IV). - **Maffei 1-2-3 Recursive INSIDE-OR Gate:** when a 1-minute candle opens *outside* the OR boundary and closes *back inside*, exit if the current 1m extreme is worse than the prior 1m extreme (long: `low < prior_low` ; short: `high > prior_high`). Equality holds the position. - **Layered Shield (Phase B step 1):** activates on Entry 2; stop moves to the Entry-1 fill price. - **Two-Bar Lock (Phase B step 2):** two consecutive favorable 5-minute closes (long: `close > open` ; short: `close < open`) lock the stop to the average entry price. Any unfavorable close resets the counter. - **The Leash / EMA Trail (Phase C Extraction):** once locked, exit on any 5-minute close through the 5m EMA(9) (long: close < EMA9; short: close > EMA9).

Section VI — Machine rules. - **Unlimited Hunting:** no per-day trade cap; positions are allowed whenever Section I + II permit. - **Daily Circuit Breaker:** cumulative realized P&L `<= -$1500` disables new entries for the rest of the session. - **EOD Flush:** all positions closed at **15:59:50 ET** (was 15:55).

Phase machine. Each open position carries `phase ∈ {SURVIVAL, NEUT_LAYERED, NEUT_LOCKED, EXTRACTION}` . Phase A on Entry 1 → step into B-Layered on Entry 2 → step into B-Locked on the second favorable 5m close → C-Extraction once the EMA trail is the binding stop.

`exit_reason` enum (v5.10.0). `sovereign_brake` , `velocity_fuse` , `forensic_stop` (Maffei INSIDE-OR), `be_stop` (Layered Shield / Two-Bar Lock break-even), `ema_trail` (The Leash), `daily_circuit_breaker` , `eod` , `manual` . Legacy values `LORDS_LEFT` , `BULL_VACUUM` , `HARD_EJECT_TIGER` , `V572-ABORT` , and the dual-PDC sovereign-eject codes are retired and **must not** be emitted by v5.10.0 logic.

Locked defaults. Volume Bucket lookback = 55 trading days, threshold ratio = 1.00, cold-start = pass-through with rate-limited log; Boundary Hold = 2 consecutive 1m closes strictly outside the 5m OR; DI period = **15** (Gene-canonical, not Wilder 14); all DI/NHOD comparisons strict `>` ; daily circuit breaker = `-$1500` ; sovereign brake = `-$500` (uses `<=`); velocity fuse = `> 1.0%` strict; EOD flush = 15:59:50 ET; post-loss cooldown: LONG = 0 min (env `POST_LOSS_COOLDOWN_MIN_LONG`), SHORT = **30 min** (env `POST_LOSS_COOLDOWN_MIN_SHORT` ; v6.10.0 raised this 30 -> 60 per the 30d Wave 2 C10 sweep, but v6.10.2 reverted to 30 after the 84d v6.10.1 SIP validation showed S=30 and S=60 produce bit-identical PnL on the current 10-ticker universe -- the cooldown

gate has zero same-ticker short re-entry events to act on, so the original 30d lift was sweep noise; reverting to 30 keeps long/short symmetric).

Cold-start caveat. Until the `/data/bars/<YYYY-MM-DD>/<TICKER>.jsonl` archive carries 55 trading days for a given ticker, that ticker's Volume Bucket gate returns `COLDSTART` and is treated as PASS — Entry 1 is gated only by Section I + Boundary Hold + DI + NHOD. Backtest and shadow-replay results during the cold-start window are not directly comparable to steady-state v5.10.0 production.

Rollback. No feature flag; v5.10.0 fully overwrites v5.9.4. Rollback path is `git revert` of the v5.10.0 PR followed by Railway redeploy.

> **Legacy algorithm sections below (v5.0–v5.9).** The text from here through the end of this document describes the v5.0 Tiger/Buffalo state machine and its v5.1–v5.9 layers. Those algorithms have been **superseded** by Project Eye of the Tiger above. The legacy text is retained for historical context and because the *infrastructure* surfaces it describes (signal bus, persistence, dashboard, backtest harness, shadow P&L, executor mirrors) are still in use under the v5.10.0 algorithm. Where the legacy text describes entry/exit logic, exit reasons, daily limits, or EOD timing, the v5.10.0 summary above is authoritative.

> **Legacy preamble (v5.5.10 · April 2026): Source of truth (legacy):** `STRATEGY.md` (canonical trading-logic spec for v5.0–v5.9), `trade_genius.py`, `tiger_buffalo_v5.py`, `volume_profile.py`, `indicators.py`, `bar_archive.py`, `shadow_pnl.py`, `persistence.py`, `dashboard_server.py`, `dashboard_static/{app.js,app.css,index.html}`, `backtest/{loader,ledger,replay,__main__}.py`

TradeGenius is a Python Telegram-driven trading bot with a paper book, two Alpaca-backed executor mirrors, and a live web dashboard. As of **v5.0.0** it runs the **Tiger/Buffalo two-stage state machine** for both long ("The Tiger Hunts the Bison") and short ("The Wounded Buffalo / Gravity Trade") trades on a small, hand-curated ticker universe; emits ENTRY/EXIT signals on an in-process bus; and (when configured) replicates those signals against Alpaca paper or live via two independent executor bots. **v5.1.x** layered an observation-only **Forensic Volume Filter** (`volume_profile.py`) plus **Forensic Capture** modules (`bar_archive.py` , `indicators.py`) on top of the v5.0.0 algorithm. **v5.1.8** moved persistence from atomic JSON to a SQLite database on the Railway Volume (`persistence.py` , `/data/state.db`). **v5.2.0** added a real-time shadow-strategy P&L tracker (`shadow_pnl.py`); **v5.2.1** hardened Alpaca submits with deterministic `client_order_id` idempotency plus boot-time broker reconcile; **v5.3.0** moved the shadow panel to its own dedicated dashboard tab with click-to-expand per-config detail. **v5.4.0** added the `backtest/` offline replay package and a `python -m backtest.replay` CLI with replay-vs-prod validation. **v5.11.0 PR 6** ports the canonical full-day replay harness into the repo at `backtest/replay_v511_full.py` and rewires it to consume `engine.scan.scan_loop` directly via a `RecordOnlyCallbacks` impl of the `EngineCallbacks` Protocol — backtests now drive the same scan body the bot runs in prod, eliminating the parallel re-implementation of seeding/phase logic that the workspace-only `replay_v510_full_v4.py` carried. **v5.4.1** layered three Chart.js visualizations onto the Shadow tab (equity curves, day-P&L heatmap, rolling win-rate sparklines) backed by a new `/api/shadow_charts` endpoint with a 30 s server-side cache. The v5.0.0 Tiger/Buffalo entry/exit decisions are unchanged through v5.4.1 — every v5.1.x / v5.2.x / v5.3.x / v5.4.x add is observational, infrastructure, or display-only.

The canonical specification of the trading logic lives in `STRATEGY.md` at the repo root. Sections 6 and 7 below summarize the v5 algorithm and risk model; for any disagreement between this file and `STRATEGY.md`, **`STRATEGY.md` is authoritative**.

App was branded **Stock Spike Monitor** through v3.5.0; renamed to **TradeGenius** in v3.5.1. The pre-v3.5.0 TradersPost mirror portfolio, Robinhood surface, and Gmail/IMAP intake have all been removed and are not part of the current system.

---

1. High-level overview

```
                    ┌──────────────────────────────────────┐
                    │   Railway container (single process)    │
                    │                                      │
Telegram ──────────►│  trade_genius.py                     │
(main bot,          │   ├─ Scheduler thread                │
 Val bot,           │   │   • scan_loop() every 60s        │
 Gene bot)          │   │   • timed jobs (09:30/35/55, ...)   │
                    │   │                                   │
                    │   ├─ In-memory state                  │
                    │   │   positions, short_positions,     │
                    │   │   paper_cash, or_high/low, pdc,   │
                    │   │   _trading_halted, etc.           │
                    │   │                                   │
                    │   ├─ Signal bus  ──────► Val / Gene ──► Alpaca
                    │   │   _emit_signal(event) async fan-out  │
                    │   │                                   │
                    │   └─ paper_state.py persistence       │
                    │       paper_state.json on Railway Volume │
                    │                                      |
                    │  dashboard_server.py (aiohttp)        │
                    │   ├─ /login, /api/state, /api/indices,   │
                    │   │  /api/executor/{val,gene},        │
                    │   │  /api/errors/{exec}, /stream (SSE)   │
                    │   ├─ /api/version (unauthenticated)      |
                    │   └─ static UI (dashboard_static/)       |
                    └──────────────┬──────────────────────┘
                                   │
                    ┌──────────────┼──────────────┐
                    ▼              ▼              ▼
Alpaca Markets      Yahoo v8/chart      Telegram API
(equity bars,       (cash indices       (commands +
 paper + live       ^GSPC/^IXIC/^DJI/    notifications,
 executors,         ^RUT/^VIX, futures   per-bot tokens)
 Val + Gene)        ES/NQ/YM/RTY)
```

Everything runs in a **single Python process** on Railway. Concurrency model:

- **Main thread** — Telegram async polling (`python-telegram-bot==21.11.1`).
- **`scheduler_thread`** — daemon thread, owns the 60 s scan cadence and the daily timed jobs (`reset_daily_state` , `collect_or` , `eod_close` , …).

- **`dashboard-http`** — daemon thread with its own `asyncio` event loop running an `aiohttp` server on `DASHBOARD_PORT` (default 8080).
- **Per-executor Telegram apps** — Val and Gene each run their own `telegram.ext.Application` event loop (started by `TradeGeniusBase.start()`).
- **Signal listeners** — each `_emit_signal(event)` fans out to listeners in **fresh daemon threads** so a slow Alpaca round-trip on one executor cannot stall the scanner.

---

2. Repo layout

```
trade_genius.py            # bot lifecycle (post-v5.11.0): WebSocket, broker,
                           # Telegram, persistence, dashboard, scheduler;
                           # BOT_VERSION + CURRENT_MAIN_NOTE live here
engine/                    # v5.11.0 — extracted trading-rules engine
    __init__.py            # re-exports the public engine surface
    bars.py                # 5m OHLC + EMA9 aggregation
    seeders.py             # pre-market QQQ regime / DI / OR seeders
    phase_machine.py       # Phase A/B/C state machine (pure)
    callbacks.py           # EngineCallbacks Protocol (I/O seam)
    scan.py                # per-minute scan_loop over the universe
telegram_ui/               # v5.11.1 — Telegram presentation layer
    __init__.py            # re-exports public UI surface
    charts.py              # matplotlib dayreport / equity / portfolio
    commands.py            # synchronous command builders (sync helpers)
    menu.py                # keyboards + callback handlers
    runtime.py             # bot lifecycle (auth, startup, run loop)
broker/                    # v5.11.2 — broker / position-management
    __init__.py            # re-exports public broker surface
    stops.py               # breakeven, capped, ladder, retighten
    orders.py              # check_breakout, execute_breakout, close_breakout
    positions.py           # manage_positions / manage_short_positions
    lifecycle.py           # check_entry, execute_entry, close_position, eod_close
executors/                 # v5.12.0 — Val / Gene shadow-book runtime
    __init__.py            # re-exports TradeGeniusBase / Val / Gene
    base.py                # TradeGeniusBase: paper-keys + signal subscription
    val.py                 # TradeGeniusVal(NAME="Val", ENV_PREFIX="VAL_")
    gene.py                # TradeGeniusGene(NAME="Gene", ENV_PREFIX="GENE_")
    bootstrap.py           # build_val_executor, build_gene_executor, install_globals
eye_of_tiger.py            # spec gates §I–§VI (pure functions, unchanged)
qqq_regime.py              # QQQ Index Shield helpers (§I)
volume_bucket.py           # §II.1 rolling 55-day volume baseline
dashboard_server.py        # aiohttp dashboard backend (~1.9 kLOC)
dashboard_static/
    index.html             # single-page dashboard shell
    app.js                 # two IIFEs: main tab + Val/Gene tab + index ticker
    app.css                # tokens + responsive @media bands
side.py                    # Side enum + SideConfig table (long/short collapse)
paper_state.py             # paper book persistence (extracted in v4.6.0)
error_state.py             # per-executor error rings + dedup gate (v4.11.0)
tiger_buffalo_v5.py        # v5.0.0 Tiger/Buffalo state-machine helpers (legacy)
volume_profile.py          # v5.1.0 Forensic Volume Filter (shadow only)
indicators.py              # v5.1.2 pure indicator math (rsi/ema/atr/vwap/spread)
bar_archive.py             # v5.1.2 1m bar JSONL persistence (/data/bars/)
persistence.py             # v5.1.8 SQLite-backed state store (/data/state.db)
shadow_pnl.py              # v5.2.0 real-time shadow-strategy P&L tracker
telegram_commands.py       # slash-command handlers
smoke_test.py              # local + prod smoke tests
synthetic_harness/         # 50-scenario byte-equal replay harness
    runner.py, recorder.py, market.py, clock.py, scenarios/, goldens/
tests/golden/              # v5.11.0 byte-equal harness gating engine/* moves
    record_session.py, verify.py, v5_10_7_session_2026-04-28.jsonl
scripts/
    build_algo_pdf.py      # regenerates trade_genius_algo.pdf from this file
Dockerfile                 # explicit per-file COPY whitelist (see §11 Gotchas)
requirements.txt
railway.json, nixpacks.toml
```

```
.github/workflows/
    version-bump-check.yml # blocks PRs without BOT_VERSION + CHANGELOG bump
    post-deploy-smoke.yml  # runs smoke against tradegenius.up.railway.app
CHANGELOG.md               # full version history
COMMANDS.md                # full Telegram command reference
README.md
trade_genius_algo.pdf      # rendered architecture/algo doc
ARCHITECTURE.md            # this file
```

---

3. Process startup

`python trade_genius.py` (the `CMD` in the Dockerfile) does the following at module load:

1. Read environment variables (see §10).
2. `_init_tickers()` reads `TICKERS_FILE` (default `tickers.json`); falls back to `TICKERS_DEFAULT` if the file is missing or malformed. `TICKERS_PINNED = ("SPY", "QQQ")` is force-merged in.
3. `_validate_side_config_attrs()` asserts that every `*_attr` field on `SideConfig` (long + short) resolves to a real module-level global. Fail-fast import guard added in v4.9.2 — a renamed dict raises at load time instead of at first entry mid-session.
4. Optional: `dashboard_server.start_in_thread()` if `DASHBOARD_PASSWORD` is set (and ≥ 8 chars).
5. Optional: instantiate `TradeGeniusVal` and `TradeGeniusGene` if their `*_ENABLED` env vars are truthy and paper keys are present; each subscribes to the signal bus via `register_signal_listener`.
6. Start `scheduler_thread` as a daemon.
7. Boot the main Telegram `Application` and call `run_polling()`.

If `SSM_SMOKE_TEST=1` is set the file imports cleanly but skips the network-touching startup (used by `smoke_test.py` and the synthetic harness so they can `import trade_genius` without Telegram credentials).

---

4. Ticker universe

`TICKERS_DEFAULT` is **11 tickers**:

```
AAPL  MSFT  NVDA  TSLA  META  GOOG  AMZN  AVGO  QBTS  SPY  QQQ
```

- **9 spike candidates** — entries fire here.
- **2 pinned filter tickers** — `SPY` and `QQQ` are `TICKERS_PINNED`; they are read on every scan to gate entries by index polarity but are **never opened as positions** and cannot be removed via `/ticker remove`.

Edits go through `add_ticker()` / `remove_ticker()` (Telegram `/ticker add|remove`) which mutate the in-place `TICKERS` list and persist via `_save_tickers_file()` (atomic `.tmp` + `os.replace`). `TRADE_TICKERS` is rebuilt to mirror `TICKERS - TICKERS_PINNED` after every mutation. `TICKERS_MAX = 40` caps the universe so the per-cycle Yahoo budget stays bounded.

Symbol regex: `^[A-Z][A-Z0-9.\-]{0,7}$` (uppercase, US-equity-style; classes, ETFs, and common preferred-stock notations are accepted).

---

5. Scheduler & scan loop

5.1 Timed jobs (`scheduler_thread`)

All times in **America/New_York** (ET); display in the UI is also ET. The trader views the ET clock as the trading clock; an internal display helper renders some Telegram cards in CDT to match the trader's home tz.

Time	Day(s)	Job
09:20	Weekdays	System self-test card
09:30	Weekdays	`reset_daily_state()` — clear OR, daily counts
09:31	Weekdays	System self-test card
09:35	Weekdays	`collect_or()` (in its own thread)
09:36	Weekdays	`send_or_notification()` — OR card to Telegram
15:55	Weekdays	`eod_close()` — force-close all open positions
15:58	Weekdays	`send_eod_report()`
18:00	Sunday	`send_weekly_digest()`

Idempotency: each fire is keyed by `YYYY-MM-DD-HH:MM-day-HH:MM`, kept in a `fired` set, and pruned to today's keys when it grows beyond 200 entries. The fire set is in-memory; a process restart re-fires any job whose minute the new instance lands inside.

5.2 Scan loop (`scan_loop`)

`scheduler_thread` calls `scan_loop()` every `SCAN_INTERVAL = 60` s. **Post-v5.11.0** the body of `scan_loop` lives in `engine/scan.py`; the function in `trade_genius.py` is a thin shim that builds an `EngineCallbacks` impl (broker calls, position lookup, Telegram alerts, clock) and delegates. The loop's structure (unchanged):

1. `_refresh_market_mode()` — refresh the OPEN/CHOP/POWER/DEFENSIVE/CLOSED classifier *before* any early-return so the dashboard banner stays correct in after-hours too.
2. Skip if weekend, or before 09:35 ET, or after 15:55 ET (sets `_scan_idle_hours = True` for the dashboard's GATE pill).
3. `_clear_cycle_bar_cache()` — drop the per-cycle 1-minute bar cache.
4. **Regime alert** — re-evaluate `(SPY_cur > SPY_PDC) and (QQQ_cur > QQQ_PDC)`; on transition, fire a Telegram regime card.
5. `manage_positions()` — long stop chain + Red Candle + Lords Left.
6. `manage_short_positions()` — short stop chain + Bull Vacuum + Polarity Shift.
7. `_tiger_hard_eject_check()` — DI-decay hard-eject for open positions.

8. If `_scan_paused` is true (manual `/monitoring pause`), skip new entries; **position management still runs**.
9. For each `t` in `TRADE_TICKERS`: `_update_gate_snapshot(t)` → `check_entry(t)` → `execute_entry(t, px)` → `check_short_entry(t)` → `execute_short_entry(t, px)`.
10. Every exception inside steps 5–9 is wrapped in `report_error(...)` so it shows up in the per-executor health pill (§9.2) and the matching Telegram channel.

`save_paper_state()` runs **every 5 minutes** in its own daemon thread (driven by `last_state_save` in `scheduler_thread`).

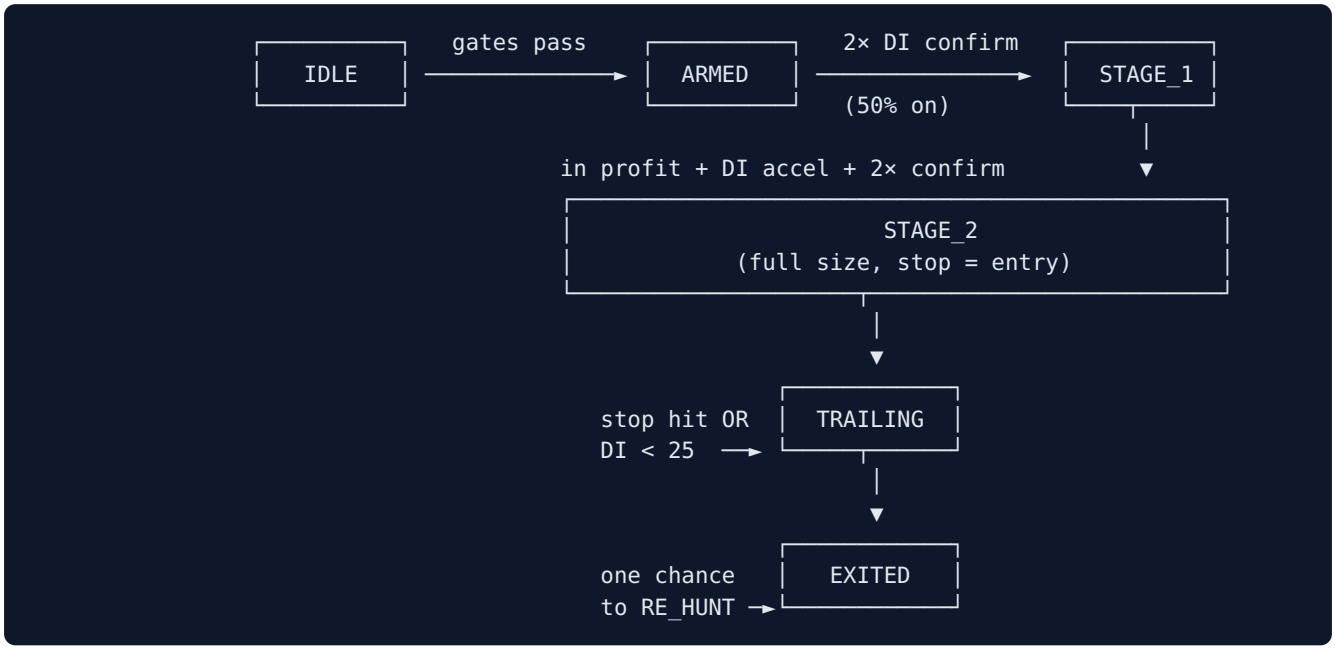
---

6. Trading algorithm — Tiger/Buffalo (v5.0.0, unchanged through v5.1.2)

> **Canonical spec:** `STRATEGY.md`. For any rule disagreement, the spec wins. Every code-level decision in `tiger_buffalo_v5.py` cites a rule ID (e.g. `L-P2-R3`) that maps 1:1 to the spec; smoke-test docstrings reference the same IDs.
>
> **v5.1.x is observation only.** v5.1.0 added the Forensic Volume Filter (§17) and v5.1.2 added the Forensic Capture layer (§18), but **neither one changes any Tiger/Buffalo entry, exit, sizing, or stop decision**. `VOL_GATE_ENFORCE` defaults to `0` and stays at `0` through v5.1.2. The synthetic harness 50/50 byte-equal goldens are still byte-equal under v5.1.2.

v5 reframes both long and short trades as a **two-stage entry, structural-ratchet exit** state machine. Per-ticker per-direction state lives in `trade_genius.v5_long_tracks` / `trade_genius.v5_short_tracks` (persisted in `paper_state.json`).

6.1 States



State enum: `IDLE`, `ARMED`, `STAGE_1`, `STAGE_2`, `TRAILING`, `EXITED`, `RE_HUNT_PENDING`, `LOCKED_FOR_DAY` (helpers in `tiger_buffalo_v5.py`).

6.2 Long protocol — "The Tiger Hunts the Bison"

Phase	Rule IDs	Summary
L-P1	G1..G4	Permission gates: QQQ > QQQ-PDC, SPY > SPY-PDC, ticker > ticker-PDC, ticker > first-hour high (09:30–10:30 ET). All four must be true to leave IDLE.
L-P2	R1..R5	Stage-1 (50% on): `DI+(1m) > 25 AND DI+(5m) > 25` confirmed across **two consecutive closed 1m candles**. Initial stop = low of prior closed 5m candle (hard, does not move during STAGE_1). Record `original_entry_price`.
L-P3	R1..R5	Stage-2 (full size): `DI+(1m) > 30` confirmed across two more consecutive closed 1m candles AND ticker last > original entry ("Winning Rule"). Add remaining 50%. **Safety Lock:** stop on entire 100% position moves to `original_entry_price` ("House Money").
L-P4	R1..R4	TRAILING: every 5m close, ratchet stop UP-ONLY to most recent Higher Low. Hard exits: `ticker.last < current_stop` OR `DI+(1m) < 25` on a closed 1m candle — either flattens 100%.
L-P5	R1..R3	Re-Hunt (one shot): after exit, dormant until ticker > original entry ("reclamation"). On reclamation re-arm with FRESH values. After a second exit → `LOCKED_FOR_DAY`.

6.3 Short protocol — "The Wounded Buffalo / Gravity Trade"

Structural mirror of the long side. The **priority-1 rule (S-P4-R3) inverts** vs. the long side: `DI−(1m) < 25` is checked BEFORE the structural-stop hit. Rationale (per spec): "fear moves faster than greed" — short-side momentum decay typically precedes a squeeze, so the bot covers on DI failure ahead of a stop trigger.

Phase	Rule IDs	Summary
S-P1	G1..G4	Gates: QQQ < PDC, SPY < PDC, ticker < PDC, ticker < opening-range-low (09:30–09:35 ET). If indices are GREEN, shorts are forbidden regardless of ticker weakness.
S-P2	R1..R5	Stage-1 short (50% on): `DI−(1m) > 25 AND DI−(5m) > 25` confirmed × 2. Initial stop = high of prior closed 5m candle.
S-P3	R1..R5	Stage-2: `DI−(1m) > 30` × 2 closed candles AND ticker last < original entry. Add remaining 50%. Safety Lock moves stop to `original_entry_price`.
S-P4	R1..R5	TRAILING: ratchet stop DOWN-ONLY to most recent Lower High on each 5m close. **Priority-1 hard eject:** `DI−(1m) < 25` ⇒ immediate flatten BEFORE structural-stop check. **Priority-2:** `ticker.last > current_stop` ⇒ flatten.
S-P5	R1..R3	Re-Hunt (one shot) on price reclaiming below original entry. Second exit ⇒ `LOCKED_FOR_DAY`.

6.4 Sizing — preserved from v4

v5 changes how the position is **staged in** (50% then 50%), not how the unit is computed.

Parameter	Value
Starting paper capital	$100,000.00 (`PAPER_STARTING_CAPITAL`)
Dollars per entry (paper)	`$10,000` (`PAPER_DOLLARS_PER_ENTRY` env) — this is "100% of unit"
Stage-1 fill size	50% of unit
Stage-2 fill size	remaining 50%
Shares per entry	`paper_shares_for(price)` (unchanged)
Order type	Limit at current market price

Executor sizing is independent and per-executor: `VAL_DOLLARS_PER_ENTRY` and `GENE_DOLLARS_PER_ENTRY` (default $10,000 each). As of **v5.1.4** the live executor `_shares_for(price, ticker=...)` is **equity-aware**: on every entry it calls `client.get_account()` and computes

```
effective_dollars = min(
    dollars_per_entry,
    equity * MAX_PCT_PER_ENTRY/100,
    cash - MIN_RESERVE_CASH,
)
qty = max(1, int(effective_dollars // price))
```

Defaults: `MAX_PCT_PER_ENTRY = 10.0` (i.e. ≤10% of current Alpaca equity per entry) and `MIN_RESERVE_CASH = 500` USD. Worked example: a $30k account with `DOLLARS_PER_ENTRY=10000` will scale each entry down to ~$3,000 and emit `[SIZE_CAPPED]` at INFO. If `effective_dollars < price` the executor returns `qty=0` and logs `[INSUFFICIENT_EQUITY]` instead of submitting an order Alpaca will reject. If `get_account()` itself fails (network blip), the executor logs `[SIZING_FALLBACK]` and falls back to the legacy `int(dollars_per_entry // price)` path so live trading never hard-fails on a transient API error. Paper book sizing (`paper_shares_for` / `PAPER_DOLLARS_PER_ENTRY`) is **unchanged** and remains byte-equal under the synthetic-harness goldens. Per spec C-R7, SPY/QQQ are pinned filter rows on the dashboard and serve as L-P1-G1/G2 / S-P1-G1/G2 inputs; they are NEVER traded directly.

---

7. Risk: v5 stop model + portfolio brakes

> The v4 4-layer stop chain (structural baseline / 0.75% cap / breakeven ratchet / peak-anchored profit-lock ladder) has been **replaced** in v5 with the spec's two-stage stop model below. The v4 ladder math is preserved in source for the synthetic harness baselines but is no longer authoritative.

7.1 v5 per-trade stop model

The stop is a single value per track, evolved through three regimes:

1. **Stage-1 stop (L-P2-R4 / S-P2-R4) — "Emergency Exit."** Set to the low of the prior closed 5m candle (long) or the high of the prior closed 5m candle (short). Hard stop. Does NOT move while the track is in `STAGE_1`.

2. **Stage-2 safety lock (L-P3-R5 / S-P3-R5) — "House Money."** On the Stage-2 fill, the stop on the entire 100% position is moved to `original_entry_price`. The trade is now risk-free vs. its own cost basis.
3. **TRAILING ratchet (L-P4-R1..R2 / S-P4-R1..R2).** On each 5m close after Stage 2, compute the most recent Higher Low (long) or Lower High (short). Ratchet in the favorable direction only — the stop NEVER moves down on a long or up on a short. Implementation: `tiger_buffalo_v5.ratchet_long_higher_low` / `ratchet_short_lower_high`.

7.2 Hard exits (L-P4-R3 / S-P4-R3..R4)

Side	Priority-1 (closed 1m candle)	Priority-2 (every tick)
Long	`DI+(1m) < 25` ⇒ flatten	`ticker.last < current_stop`
Short	`DI−(1m) < 25` ⇒ flatten	`ticker.last > current_stop`

Short side inverts ordering per S-P4-R3: the DI<25 hard eject is priority-1 over the structural-stop check. This is the single most important behavioral asymmetry between the two protocols and is covered by a dedicated smoke test (`v5 S-P4-R3: short DI<25 hard eject fires PRIORITY-1 over structural stop`). Long side: either trigger flattens, order is observability-only.

7.3 Re-Hunt budget (L-P5-R3 / S-P5-R3)

After an exit the track lands in `EXITED` with the re-hunt budget unspent. On reclamation (price climbs back above original entry for longs, below for shorts), the state machine returns to `ARMED` with all stage counters reset and `original_entry_price = None`. After a second exit the track is forced to `LOCKED_FOR_DAY` regardless of subsequent reclamations.

7.4 Portfolio-level brakes — preserved from v4

Spec section C documents the cross-cutting rules; v5 wires each into the existing v4 helpers so the brakes still fire even when the v5 state machine is mid-trade.

Spec	Brake	Implementation
C-R4	Daily-loss-limit (incl. v4.7.0 short-side cap)	`_check_daily_loss_limit()` flips `_trading_halted=True` AND calls `v5_lock_all_tracks("daily_loss_limit")` so every track moves to `LOCKED_FOR_DAY`.
C-R5	EOD force-close at 15:55 ET	`eod_close()` walks `positions` + `short_positions` flattening every open paper position, then calls `v5_lock_all_tracks("eod")` so the next session starts fresh rather than resuming.
C-R6	Sovereign Regime Shield (Eye of the Tiger)	**Fully retired by v5.9.3.** The PDC-based dual-index eject is gone in three steps: v5.9.1 removed `_sovereign_regime_eject()` (`LORDS_LEFT` / `BULL_VACUUM`); v5.9.2 stripped the dual-PDC half from `_tiger_hard_eject_check()` , leaving Tiger as a pure DI-decay eject (`HARD_EJECT_TIGER` fires on `DI < TIGER_V2_DI_THRESHOLD` only); v5.9.3 eradicated the residual `LORDS_LEFT*` / `BULL_VACUUM*` keys from `REASON_LABELS` / `_SHORT_REASON` and the lingering Telegram strategy-help text. v5.9.0 already swapped the entry-side index regime to the 5m EMA compass (QQQ Regime Shield). Per-ticker RED_CANDLE / POLARITY_SHIFT exits remain.
C-R7	9-ticker spike universe + SPY/QQQ pinned	`TRADE_TICKERS` is the 9-name spike list (excludes SPY/QQQ). `check_breakout` reads SPY/QQQ as polarity inputs only.

`DAILY_LOSS_LIMIT` (default −$500, env-tunable). Once today's realized P&L crosses the floor, `/api/state` exposes `_trading_halted_reason` so the dashboard's GATE pill paints `HALTED`. Both long and short honor the limit (the v4.7.0 fix is preserved).

---

8. Signal bus & executors

8.1 The bus

`_signal_listeners: list[Callable]` plus `_signal_listeners_lock` (a `threading.Lock`). `register_signal_listener(fn)` is idempotent — a re-registration is a no-op so a supervisor restart of an executor cannot double-fire entries against Alpaca.

`_emit_signal(event)` snapshots the listener list under the lock, spawns a fresh daemon thread per listener, and returns immediately. Per-listener exceptions are caught and logged but never propagated.

Event schema:

```
{
    "kind":         "ENTRY_LONG" | "ENTRY_SHORT" | "EXIT_LONG" |
                    "EXIT_SHORT" | "EOD_CLOSE_ALL",
    "ticker":       "AAPL",        # absent on EOD_CLOSE_ALL
    "price":        175.42,        # main's reference price
    "reason":       "BREAKOUT" | "STOP" | "TRAIL" | "RED_CANDLE" |
                    "POLARITY_SHIFT" | "EOD" | "HARD_EJECT_TIGER" | ... ,
                    # PDC-based Sovereign Regime Shield fully retired:
                    # v5.9.1 removed LORDS_LEFT/BULL_VACUUM emission,
                    # v5.9.2 dropped the dual-PDC HARD_EJECT_TIGER half,
                    # v5.9.3 eradicated the residual labels & comments.
    "timestamp_utc":"2026-04-24T13:45:12Z",
    "main_shares":  57,            # audit: shares the paper book traded
}
```

8.2 Executors: TradeGeniusVal & TradeGeniusGene

Both subclass a shared `TradeGeniusBase`. Behavior lives in the base; the subclasses only set `NAME` and `ENV_PREFIX`.

Executor	Name	Env prefix
`TradeGeniusVal`	Val	`VAL_`
`TradeGeniusGene`	Gene	`GENE_`

Per-executor knobs (read at instance construction):

- `*_ALPACA_PAPER_KEY` / `*_ALPACA_PAPER_SECRET`
- `*_ALPACA_LIVE_KEY` / `*_ALPACA_LIVE_SECRET`
- `*_TELEGRAM_TG` (token) / `*_TELEGRAM_CHAT_ID`
- `*_DOLLARS_PER_ENTRY` (default `$10,000`)
- `*_ENABLED` (default on; set to `0` to skip startup)

The owner whitelist is **shared with main** via `TRADEGENIUS_OWNER_IDS` — there are no per-executor `VAL_TELEGRAM_OWNER_IDS` or `GENE_TELEGRAM_OWNER_IDS` (those names appear in `.env.example` for historical readers but the code uses the unified set). Default value: `5165570192,167005578` (Val + Gene).

Routing (clarified in v5.0.3). Each executor's *own* Telegram bot fans trade confirmations out to every learned owner-id → chat-id pair. In other words: Val's trades land on Val's bot, Gene's trades land on Gene's bot, but **every owner sees every executor's confirmations on that executor's bot** because the owner whitelist is shared. Each owner must DM their executor bot once (any message) so the auto-learn hook records the chat_id; the map is persisted at `/data/executor_chats_{name}.json` and survives Railway redeploys (see §11.6). No per-executor `<PREFIX>TELEGRAM_CHAT_ID` env var is required — it is accepted only as a back-compat seed.

Paper/live Alpaca key independence (v5.0.4). Each executor reads its paper credentials from **only** `<PREFIX>ALPACA_PAPER_KEY` / `<PREFIX>ALPACA_PAPER_SECRET` and its live credentials from **only** `<PREFIX>ALPACA_LIVE_KEY` / `<PREFIX>ALPACA_LIVE_SECRET`. There is no fallback from paper to live (or vice versa). v5.0.3 briefly added a fallback from `<PREFIX>ALPACA_PAPER_KEY` to `<PREFIX>ALPACA_KEY` (the un-

prefixed name); v5.0.4 reverted it after Val confirmed `GENE_ALPACA_KEY` on Railway is a LIVE key. Operators provisioning a new executor must mint a fresh paper key from a paper Alpaca account and set the paper-prefixed env var — never repurpose un-prefixed keys. See §11.7 for the full incident write-up.

State files (one per mode, per executor):

```
tradegenius_val_paper.json     tradegenius_val_live.json
tradegenius_gene_paper.json    tradegenius_gene_live.json
```

`_load_state()` picks the most-recently-written file so a live restart stays in live; mode flips rebuild the Alpaca client through `_build_alpaca_client(mode=...)`.

8.3 Live mode flip

`set_mode("live", confirm_token)` requires:

1. `confirm_token == "confirm"` (i.e. the operator typed `/mode val live confirm`).
2. `_live_sanity_check()` — build a temp live `TradingClient`, call `get_account()`, verify status contains `ACTIVE`. Logs account number / cash / buying power.

If either gate fails, the mode does not flip and the bot replies with the error reason. Paper flips have no token gate.

8.4 Alpaca order idempotency (v5.2.1 H1)

Every executor submit is routed through `_submit_order_idempotent`, which stamps a deterministic `client_order_id` of the form

```
f"{NAME}-{TICKER}-{utc_iso_minute}-{LONG|SHORT}"
```

(e.g. `Val-AMD-2026-04-24T13:48Z-LONG`) on the request. Alpaca rejects a second submit with the same `client_order_id` as a duplicate; the wrapper catches that rejection, refetches the existing order via `get_order_by_client_order_id`, and returns it as if the original submit succeeded. The net effect is "submit once, even after a process restart within the same minute." A startup hook, `_reconcile_broker_positions`, walks the live Alpaca positions list at boot, grafts any broker order that is missing from the in-process state back into the matching track, and DMs the executor's owners that a graft happened. Together H1 closes the v5.0–5.2 race where a Railway redeploy mid-minute could have double-submitted an entry.

8.5 Alpaca client URL hygiene

`alpaca-py 0.43.2`'s `RESTClient` builds the final URL as `base_url + "/" + api_version + path`, i.e. it always appends `/v2`. `url_override` therefore must be a **host** (e.g. `https://paper-api.alpaca.markets`) — not a host-with-`/v2`. `_build_alpaca_client` defensively strips trailing `/v2` or `/v2/` so a misconfigured `ALPACA_ENDPOINT_PAPER` or `ALPACA_ENDPOINT_TRADE` env var cannot produce double-prefixed URLs.

---

9. Dashboard

9.1 Backend (`dashboard_server.py`)

`aiohttp` server, runs in a daemon thread with its own asyncio loop.

Endpoint	Auth	Purpose
`GET /`	cookie or login page	Dashboard SPA (`dashboard_static/index.html`)
`POST /login`	rate-limited (5/min/IP)	Form-encoded password; sets HMAC token cookie
`POST /logout`	cookie	Clears `spike_session` cookie
`GET /api/state`	cookie	Aggregate snapshot: portfolio, positions, trades, gates, regime, errors
`GET /api/version`	**none**	Returns `{"version": BOT_VERSION}` — used by the post-deploy poller
`GET /api/indices`	cookie	SPY/QQQ/DIA/IWM/VIX (Alpaca) + ^GSPC/^IXIC/^DJI/^RUT/^VIX (Yahoo) + futures badges
`GET /api/executor/{name}`	cookie	Per-executor tab data; cached 15 s
`GET /api/errors/{executor}`	cookie	Health-pill expanded list (last 10)
`GET /api/trade_log`	cookie	Persistent trade-log tail; supports `limit` , `since` , `portfolio`
`GET /stream`	cookie	Server-Sent Events; pushes a state snapshot every 2 s
`GET /static/*`	none	Served from `dashboard_static/`

Auth: HMAC-SHA256 token stored in `spike_session` cookie. Token = `hex(HMAC(secret, BE-uint64-timestamp)) + ":" + ts` . The secret is generated once on first boot and persisted to `dashboard_secret.key` on the Railway Volume so re-deploys do not invalidate sessions (v3.4.29). Override via `DASHBOARD_SESSION_SECRET` for tests / forced rotation. Cookie attrs: `HttpOnly; SameSite=Lax; Secure=True; Max-Age=7d` .

`_check_auth()` validates timestamp + HMAC on every authenticated endpoint. POST `/login` enforces a per-IP in-memory rate limit (5 attempts per 60 s window).

State snapshot caching:

- `_cached_snapshot()` — TTL cache of `snapshot()` (~10 s) shared across SSE clients. v4.1.9-dash so concurrent dashboards do not each pay `O(N_positions)` Alpaca round-trips every 2 s.
- `_indices_cache` — 30 s TTL on `_fetch_indices()` (one Yahoo batch per cache miss; ETF prices come from Alpaca via the executor's data client).
- `_executor_cache` — 15 s TTL per `(executor_name)` .

9.2 Index ticker (v4.10 → v4.13.0)

The strip at the top of the dashboard now carries **two stacked sources**:

Alpaca ETF rows (5):

```
SPY  QQQ  DIA  IWM  VIX
```

`_fetch_indices()` calls `client.get_stock_snapshot(StockSnapshotRequest(...))` on the first available executor's paper keys (`_resolve_data_client`). `VIX` is requested separately and tagged with `reason="vix_no_equity_feed"` because Alpaca's equity feed does not carry the volatility index symbol — its row stays as a sentinel placeholder.

Yahoo cash + futures rows (5):

```
^GSPC (S&P 500)   [ES +0.40%]
^IXIC (Nasdaq)    [NQ +0.32%]
^DJI  (Dow)       [YM +0.18%]
^RUT  (Russell 2K)[RTY −0.05%]
^VIX  (VIX)       (no future badge — VX=F is on CFE)
```

`_fetch_yahoo_quote_one(symbol)` hits

```
https://query1.finance.yahoo.com/v8/finance/chart/<URL-ENC(symbol)>?
    interval=1m&range=1d&includePrePost=true
```

with the caret/equals URL-encoded (`^→%5E` , `=→%3D`) so `^GSPC` and `ES=F` both round-trip cleanly. `_fetch_yahoo_quotes(symbols)` fans out across a `ThreadPoolExecutor(max_workers=8)` with a per-request `timeout=6` seconds, so the 9-symbol batch (5 cash + 4 futures) completes in ≈ one request's wall-clock instead of 9 sequential.

The futures badge `[ES +0.40%]` is computed from the future's *own* percent move vs. the future's *own* previous close — not from last-cash vs. future-prev — so the badge tells the trader *where futures are pricing the next session's open*, which is the entire reason to show futures. `^VIX` has no entry in `_YAHOO_INDEX_FUTURE` so its row never paints a badge.

If the Yahoo batch returns **zero** symbols (whole-batch failure), the payload sets `yahoo_ok=false` and `yahoo_error=...` ; the frontend prepends a dim `data delayed` chip while keeping every Alpaca ETF row live ("degrade, don't disappear"). A *partial* miss (e.g. `^RUT` flickering for one cache window) silently omits the missing row.

Session classification — `_classify_session_et()` returns one of `rth | pre | post | closed` from `America/New_York` :

- Weekday `04:00–09:30 ET` → `pre`
- Weekday `09:30–16:00 ET` → `rth`
- Weekday `16:00–20:00 ET` → `post`
- Otherwise (incl. weekends) → `closed`

Used by the frontend AH layer on ETF rows: outside RTH the row paints an extra `AH +0.42 +0.06%` (or `PRE …`) chip in the AH delta's own color (independent of the regular-session change's color). The backend's `_fetch_indices()` returns `marketState=None` from Yahoo's metadata, which is why we classify session locally instead of trusting the upstream field.

9.3 Frontend (`dashboard_static/`)

Single HTML file, two independent IIFEs in `app.js`:

1. **Main dashboard IIFE** (≈ lines 1–797). Owns the `/api/state` poller, `/stream` SSE consumer, KPI rendering, position table, trade table, GATE/Regime/Session pills, `applyGateTriState()` , `__tgApplyHealthPill()` , brand-row clock, LIVE pill, recycle countdown.
2. **Tabs + executors IIFE** (≈ lines 799–1632). Owns the Main/Val/Gene tab switcher, per-executor `/api/executor/{name}` poller (15 s), and the index ticker strip `/api/indices` poller (30 s) including marquee animation.

The two IIFEs communicate via `window.__tgApplyGateTriState` and `window.__tgApplyHealthPill` — explicit named bridges added in v4.10.2 when `Fetch failed` banners surfaced from cross-IIFE access.

LIVE pill (v4.11.5). `#h-tick` always renders `♻ NN` s where `NN` is the seconds to the next scan, or `♻ --` when the backend has no schedule (weekend / scanner idle). The 1 s `streamTickTimer` interval decrements `__nextScanSec` if numeric and unconditionally calls `updateNextScanLabel()` . `#h-tick` is **never** hidden — Val's hard rule preserved across every mobile band.

Index marquee (v4.12.0). After each `renderIndices()` the JS measures `track.scrollWidth` vs `strip.clientWidth` ; on overflow it adds `.idx-marquee` to the strip and **duplicates** the inner items so the CSS `translateX(0 → -50%)` keyframe loops seamlessly. Pause is applied on `:hover` , `:focus-within` , and a tap-to-toggle `.is-paused` class. The whole thing is gated by `prefers-reduced-motion: reduce` which falls back to native `overflow-x: auto` .

Health pill (v4.11.0). Replaced the noisy log-tail card. Tap to expand the last 10 events. Three tiers driven by `error_state.snapshot()` :

- `green` — count == 0
- `warning` (amber) — count > 0 AND no error/critical events today
- `red` — any error or critical event today

Errors fan out to the matching executor's Telegram channel via `report_error()` with a per- `(executor, code)` 5-minute dedup gate (see `error_state.py`). Daily counts reset inside `reset_daily_state()` .

CSS breakpoints (`dashboard_static/app.css`): `@media (max-width: 500/400/380/360 px)` for the mobile cascade; an additional `900 / 640 px` band for sidebar/grid collapse and the index-strip "compact" mode that drops the absolute Δ$ value so 5 items fit on a 390-px iPhone horizontally.

9.4 Shadow strategy P&L (v5.2.0)

`shadow_pnl.py` (`ShadowPnL` class) is the in-process tracker for every shadow config that owns virtual positions. It is paper-portfolio sized (`PAPER_DOLLARS_PER_ENTRY`), marked-to-market on every minute close (`mark_to_market`), and persists rows in the `shadow_positions` table inside `/data/state.db` . The dashboard renders running today + cumulative P&L per config, win rate, and unrealized P&L. v5.2.1 H3 ungated

the mark-to-market path from `paper_holds` so a shadow position on a ticker the live paper book is also holding still gets marked.

9.5 Shadow tab (v5.3.0)

Through v5.2.1 the shadow P&L block sat at the bottom of the Main tab. v5.3.0 promotes shadow strategies to a dedicated tab — the dashboard header now carries **four tabs**: Main / Val / Gene / Shadow. The Shadow panel hosts the same per-config running P&L block plus a click-to-expand row per config that calls two new `ShadowPnL` methods:

- `open_positions_for(config_name)` — list of currently open virtual positions for the named config, including the entry timestamp, side, entry price, and live mark.
- `recent_closed_for(config_name, limit=10)` — the last N closed trades for the named config, ordered newest first, with realized P&L per row.

No schema migration ships with v5.3.0 — both methods read the existing `shadow_positions` table that v5.2.0 introduced. The two IIFEs in `app.js` continue to share state through the `window.__tg*` named bridges; the new Shadow tab simply registers a fourth `data-tg-tab` target alongside Main / Val / Gene.

9.6 Shadow market-data credentials (v5.5.3)

The shadow path is fed by `_start_volume_profile()` in `trade_genius.py`, which boots `volume_profile.WebsocketBarConsumer` once per process. That consumer needs Alpaca market-data credentials. Through v5.5.2 the cred lookup only checked the legacy `ALPACA_PAPER_KEY` / `ALPACA_KEY` pairs — but prod is provisioned with `VAL_ALPACA_PAPER_KEY` / `VAL_ALPACA_PAPER_SECRET`, so the legacy chain silently early-returned, left `_ws_consumer = None`, and starved every G4 evaluation of live volumes (see `diagnostics/shadow_data_pipeline.md` Issue 2).

v5.5.3 cred chain. `_start_volume_profile()` now resolves keys in this strict order:

1. `VAL_ALPACA_PAPER_KEY` / `VAL_ALPACA_PAPER_SECRET` (prod default)
2. `ALPACA_PAPER_KEY` / `ALPACA_PAPER_SECRET` (legacy)
3. `ALPACA_KEY` / `ALPACA_SECRET` (legacy)
4. fail → `[SHADOW DISABLED]` log line + `SHADOW_DATA_AVAILABLE = False`

Market-data-only constraint. The shadow path may use Val's Alpaca paper key **only** for market data: `/v2/stocks/*` (REST bars) and `wss://stream.data.alpaca.markets/v2/iex` (live IEX bars via `alpaca.data.live.StockDataStream` with `feed=DataFeed.IEX`). Trading endpoints — `/v2/positions`, `/v2/account`, `/v2/orders`, `/v2/portfolio/history`, any submit-order surface — are forbidden in this code path. Shadow positions stay in our own SQLite ledger (`shadow_positions`); they never touch Val's Alpaca portfolio. A smoke test pins this by asserting that `volume_profile.py` does not import `TradingClient` / `TradingStream` and contains none of the forbidden URL paths. An inline comment at the cred lookup repeats the constraint for any future reader who adds a new fallback.

Module-level state + dashboard surface. `trade_genius.SHADOW_DATA_AVAILABLE` is a `bool` flipped to `True` only after `_ws_consumer.start()` returns successfully. `dashboard_server` exposes it as `shadow_data_status: "live" | "disabled_no_creds"` on `/api/state`. The Shadow strategies card-head renders a `chip-warn` pill `SHADOW DISABLED — no market-data creds` when the status is

`disabled_no_creds`, hidden otherwise. A silent "no shadow rows today" session is therefore no longer ambiguous — the dashboard will say so.

v5.5.4 hotfix. `WebsocketBarConsumer._on_bar` is `async def` — alpaca-py's `StockDataStream.subscribe_bars()` rejects sync handlers with `handler must be a coroutine function` and crash-loops the consumer. Smoke test `v5.5.4: shadow WS bar handler is a coroutine function` pins this via `inspect.iscoroutinefunction`.

v5.5.9 Shadow-tab unrealized fallback chart + visual polish. End of the v5.5.8 trading day looked like this for the Shadow tab: OOMPH_ALERT held 207 open shadow positions for ~−$7.8k unrealized, and zero closed trades existed for any of the 7 configs because the weekly batch that materializes closed-trade rows from intraday WS state didn't run until Sat May 2. Result: `/api/shadow_charts` returned empty `equity_curve` / `daily_pnl` / `win_rate_rolling` for every config, and the EQUITY CURVES, DAY P&L HEATMAP, and ROLLING WIN RATE groups each painted seven "no closed trades" placeholder rows on top of a panel that did, in fact, have rich live data. v5.5.9 is a dashboard-only release that reuses the existing `/api/state` `shadow_pnl.configs[i].open_positions` payload (already present since v5.3.0) to make the panel useful TODAY without any server change. `dashboard_static/app.js` gains a new `_scBuildBarChart` helper that, when a config's `equity_curve` is empty but `open_positions` is non-empty, renders a per-ticker unrealized P&L bar chart in place of the placeholder: bars sorted descending by `unrealized` (gains left → losses right), green/red against the existing `--up` / `--down` CSS tokens (no new hex literals), capped at 30 with a top-15-winners + top-15-losers split when the open count exceeds 30, plus a title overlay reading `<config> · <N> open · <±$total> unrealized`. The fallback is bypassed the moment `equity_curve` becomes non-empty, so once Sat May 2's batch lands the existing line-chart code wins automatically. `_scBuildEquityRows`, `_scBuildHeatmap`, and `_scRender` also stop rendering rows for configs that have neither closed nor open trades (the CHARTS header count `· X / 7` now reflects rendered configs), and a small `#shadow-summary-band` strip at the top of `#tg-panel-shadow` aggregates total open / total unrealized $ / most-active config / state `as_of` timestamp using the same compact-strip vocabulary as the index ticker. The SHADOW STRATEGIES table picks up two visual upgrades: rows tint subtly green/red by `today.total` sign via `color-mix` on `--up` / `--down` (so the sentiment color path stays token-driven), and the `CONFIG · TODAY · CUMULATIVE` header is now `position: sticky` so it remains visible while the user scrolls the open-positions detail inside an expanded config row. `dashboard_server.py` is unchanged for this release — every byte of new functionality is in `dashboard_static/{app.js,app.css,index.html}`.

v5.5.8 Main-tab SHORT entry-row synthesis. With v5.5.7's classification fix in place, the Main tab's Today's Trades for a closed short showed only the COVER row — header read `0 opens · 1 close` because the dashboard had no entry row to count. Root cause was upstream of the renderer: short *entries* are intentionally never persisted to `paper_trades` or `short_trade_history` (the single-source-of-truth invariant that prevents double-counting on `/trades`), so `_today_trades()` had nothing to emit for the entry leg. v5.5.8 fixes this server-side as a read-only synthesis. For each row in `short_trade_history` we now emit *two* rows: a synthesized SHORT entry row built from the cover's embedded `entry_price` / `entry_time` / `entry_time_iso` / `entry_num` / `shares` / `date` (with `action="SHORT"`, `side="SHORT"`, `cost = shares * entry_price`, and `time = cover.entry_time` so the existing sort places entry before cover), plus the existing COVER row unchanged. We also sweep the live `short_positions` dict for entries dated today and synthesize a SHORT entry row for any open short whose `(ticker, entry_time)` was not already covered. The sort key for closes (`SELL` / `COVER`) now prefers `exit_time` when no unified `time` field is set, so a long BUY between a SHORT entry and its COVER still lands chronologically. Storage is unchanged — `paper_trades` and `short_trade_history` write paths are untouched, the "short opens are intentionally NOT appended" invariant is preserved, and the synthesis exists only on the read side of `_today_trades()`.

v5.5.7 Main-tab Today's-Trades + LAST SIGNAL fix. The /api/state payload exposed correct SHORT/COVER fills (with `pnl`, `pnl_pct`, `entry_price`, `exit_price`) but the Main tab's client-side renderers in `dashboard_static/app.js` only counted literal `BUY` and `SELL`. A live SHORT entry + COVER exit therefore rendered `0 opens 0 closes realized —`, and the COVER row's tail column stayed on the em-dash placeholder even though the row itself was visible. v5.5.7 rewrites `computeTradesSummary` and `renderTrades` to classify by open vs close: opens are `BUY` or `SHORT`; closes are `SELL` or `COVER`. Realized P&L on a close applies whenever a numeric `pnl` is present, so SHORT+COVER pairs now contribute to the daily realized total and win-rate denominator. Separately, the Main panel had no LAST SIGNAL card — that surface only existed inside the per-executor (Val/Gene) panels, which read `last_signal` from their `TradeGeniusBase` instance. v5.5.7 mirrors the same field for the paper book at module scope: `_emit_signal` now writes the most recent event into `trade_genius.last_signal` before dispatching to listeners, `dashboard_server.snapshot()` reads it via `getattr(m, "last_signal", None)`, and `/api/state` exposes it at the top level. A new LAST SIGNAL card on `#tg-panel-main` (`#last-sig-chip` / `#last-sig-body`) and `renderLastSignal(s)` in `app.js` complete the surface, using the same kind / ticker / price / reason / timestamp formatting the per-executor panels use. No change to `_today_trades()`, `paper_trades` / `short_trade_history`, `evaluate_g4`, or the WS consumer.

v5.5.6 shadow gate same-minute race fix. Once v5.5.5's `/api/ws_state` proved the WS feed was healthy (`volumes_size_per_symbol = 5` per ticker), it became obvious that every shadow log line was still emitting `cur_v=0` / `t_pct=0` / `qqq_pct=0` / `verdict=BLOCK`. Root cause: every shadow caller computed `volume_profile.session_bucket(datetime.now(ET))`, which returns the still-forming current minute. The Alpaca IEX websocket only delivers a 1-minute bar at the END of that minute, so reading `_ws_consumer.current_volume(ticker, current_bucket)` always raced the WS bar close-out and returned `None` (silently coerced to 0 by the `or 0` guard). v5.5.6 introduces `volume_profile.previous_session_bucket(ts_et)` (floor to the minute, subtract 1 minute, then `session_bucket(prev)`) and switches all four shadow-path call sites in `trade_genius.py` (`_shadow_log_g4`, `_v512_emit_candidate_log`, the REHUNT_VOL_CONFIRM and OOMPH_ALERT checks) to use it. The shadow gate now evaluates the just-closed minute, not the still-forming one — current-bucket reads always race the WS bar close-out, while the just-closed bucket is in `_ws_consumer._volumes` within ~100 ms of close. The bar-archive code path (which writes the live bar's `et_bucket` label) intentionally still uses `session_bucket(now_et)` because its job is to label the bar being archived right now, not read against future state. The pure functions `evaluate_g4` / `evaluate_g4_config` are unchanged — only what the shadow callers pass them as `minute_bucket` was affected. No trading-decision change.

v5.5.5 observability + watchdog. v5.5.4 fixed the crash-loop, but in 11.5 hours of prod runtime no `[VOLPROFILE]` line ever fired — the connection was up, `subscribe_bars` had succeeded, and yet no bar was being processed. To make that state diagnosable in seconds rather than half a day, `WebsocketBarConsumer` now (a) counts every successful `_on_bar` call into `self._bars_received`, stamps `self._last_bar_ts`, and records exceptions in `self._last_handler_error` before the warning log; (b) emits `[VOLPROFILE] sample bar #N sym=… vol=… bucket=…` for the first 5 bars at INFO so live data flow shows up immediately on connect, plus a `[VOLPROFILE] heartbeat: total=N` line every 100th bar; (c) runs a `VolProfileWatchdog` daemon thread that polls every 30 s and, while the regular session is open (`session_bucket(now_et)` not None), calls `self._stream.stop()` whenever no bar has arrived for ≥ `VOLPROFILE_WATCHDOG_SEC` seconds (default 120, env-tunable, clamped ≥ 30) — `_run_forever`'s outer loop then reconnects with backoff and the watchdog bumps `_watchdog_reconnects`. The watchdog's own loop body is wrapped in `try/except` so it can never silently die. The same numbers are surfaced over `GET /api/ws_state` (same `spike_session` cookie auth as `/api/state`) as `{available, bars_received, last_bar_ts, last_handler_error,`

`volumes_size_per_symbol, tickers, watchdog_reconnects, silence_threshold_sec}` so an operator can discriminate "WS idle" from "handler error" from "everything fine" without grepping logs. Separately, the bar archive's `iex_volume` field is now sourced from `_ws_consumer.current_volume(ticker, bucket)` whenever the WS path is healthy and `session_bucket(now_et)` resolves — the existing Yahoo `vols[idx]` path stays as fallback. `et_bucket` is now populated from the same `session_bucket()` call (was hardcoded `None` since v5.5.2).

---

10. Environment variables

Dumped from `os.getenv` / `os.environ` in `trade_genius.py` and `dashboard_server.py`.

10.1 Required

Variable	Used by	Notes
`TELEGRAM_TOKEN`	main bot	`@BotFather` token for the main TradeGenius bot
`CHAT_ID`	main bot	Group/channel id (negative for groups)

10.2 Owner whitelist

Variable	Default	Notes
`TRADEGENIUS_OWNER_IDS`	`5165570192`	Comma-separated Telegram user ids; group=-1 TypeHandler
		drops non-owners silently. Renamed from
		`RH_OWNER_USER_IDS` in v3.6.0 — old name no longer read

Production typically sets this to `"5165570192,167005578"` (Val + Gene).

10.3 Market data

Variable	Default	Notes
`FMP_API_KEY`	hard-coded fallback (free key)	PDC + sector / profile lookups

Yahoo Finance is the **sole** source of 1-minute bars from v5.1.3 onward (the prior Finnhub data path was removed in v5.1.3). Alpaca is the order-routing surface and the live websocket source for the v5.1.0 Forensic Volume Filter; FMP is used only for prior-day-close + sector lookups.

10.3b Data root (v6.9.1)

`TG_DATA_ROOT` (default `/data`) is the single root from which all per-subsystem on-disk paths are derived when no finer-grained env var overrides them. Setting `TG_DATA_ROOT=/tmp/tg_data` (or any writable directory) lets sweep runners, CI jobs, and sandbox environments operate without access to `/data`. Each subsystem also keeps its own override so deploys that pin a specific volume mount on one path while keeping others at the root default continue to work.

Path derivation table:

Subsystem path	Per-subsystem override	Fallback formula
`/data/or`	`OR_DIR`	`TG_DATA_ROOT + /or`
`/data/bars`	`BARS_DIR`	`TG_DATA_ROOT + /bars`
`/data/forensics`	`FORENSICS_DIR`	`TG_DATA_ROOT + /forensics`
`/data/bars/daily`	`DAILY_BAR_DIR`	`TG_DATA_ROOT + /bars/daily`
`/data/lifecycle`	`LIFECYCLE_DIR`	`TG_DATA_ROOT + /lifecycle`
`/data/volume_profiles`	`VOLUME_PROFILE_DIR`	`TG_DATA_ROOT + /volume_profiles`
`/data/state.db`	`STATE_DB_PATH`	`TG_DATA_ROOT + /state.db`
`/data/ingest_audit.db`	`INGEST_AUDIT_DB_PATH`	`TG_DATA_ROOT + /ingest_audit.db`
`/data/tickers.json`	`UNIVERSE_GUARD_PATH`	`TG_DATA_ROOT + /tickers.json`
`/data/executor_chats_{name}.json`	`{P}EXECUTOR_CHATS_PATH`	`TG_DATA_ROOT + /executor_chats_{name}.json`

`_check_disk_space` in `trade_genius.py` calls `shutil.disk_usage(TG_DATA_ROOT)` and returns `ok / sandbox mode` (not `CRITICAL`) if the path does not exist, so ephemeral CI environments do not trip the health-check alarm.

10.3c Sweep env helper (v6.9.4)

`backtest/sweep_env.py` is the canonical source of hermetic subprocess environments for all sweep runners. It exists to prevent the three silent failure modes that invalidated consecutive Wave 2 sweeps (v6.9.0-6.9.2).

`REQUIRED_ENV` -- module-level dict of three guard keys that every sweep subprocess must have. Injected unconditionally by `build_sweep_env`:

Key	Value	Guards against
`SSM_SMOKE_TEST`	`"1"`	scheduler / Telegram import side-effects
`TELEGRAM_BOT_TOKEN`	`"000:fake"`	token-present check without hitting API
`FMP_API_KEY`	`"sweep_dummy_key"`	hard-fail at import when key absent (v6.9.2)

`build_sweep_env(*, isolate_dir, tg_data_root, extra)` -- builds a full env dict from `os.environ`, overlays `REQUIRED_ENV`, sets `PAPER_STATE_PATH` / `PAPER_LOG_PATH` under `isolate_dir` (per-worker isolation, v6.9.1 fix), sets `TG_DATA_ROOT`, derives and sets all subsystem path env vars (v6.9.4), then layers `extra` last. Raises `ValueError` if either directory is absent so callers get a loud error at setup time rather than a silent failure inside the worker.

v6.9.4 derived path env vars -- `build_sweep_env` now derives and sets the following env vars under `tg_data_root`, creating directories as needed:

Env var	Path under tg_data_root	Consumer
`STATE_DB_PATH`	`state.db`	`persistence.py` SQLite store
`BAR_ARCHIVE_BASE`	`bars/`	`bar_archive.py` , `ingest/algo_plus.py`
`UNIVERSE_GUARD_PATH`	`tickers.json`	`trade_genius.py` universe guard
`INGEST_AUDIT_DB_PATH`	`ingest_audit.db`	`ingest/audit.py`
`VOLUME_PROFILE_DIR`	`volume_profiles/`	`volume_profile.py`
`OR_DIR`	`or/`	`trade_genius.py` OR-break leg
`FORENSICS_DIR`	`forensics/`	`forensic_capture.py` , dashboard
`TRADE_LOG_PATH`	`trade_log.jsonl`	`trade_genius.py` trade log
`SSM_BAR_CACHE_DIR`	`bar_cache/`	`backtest/bar_cache.py`

SSM_SMOKE_TEST permission warning suppression (v6.9.4) -- when `SSM_SMOKE_TEST=1` , `PermissionError` from write operations in `persistence.py` , `bar_archive.py` , and `trade_genius.py` (UNIVERSE_GUARD) is logged at `DEBUG` level instead of `WARNING` / `ERROR` . This eliminates the 54k warning flood per sweep day that was masking real errors.

`preflight_smoke(*, workdir, bars_dir, sample_date, env, timeout_sec)` — runs a single-day `replay_v511_full` replay before fanning out the full day grid. Raises `RuntimeError` (aborting the sweep) on: nonzero exit code, missing or empty output JSON, missing `summary.entries` / `summary.exits` keys, or stderr containing `'Traceback'` / `'Permission denied'` . Logs first 50 lines of stderr on failure for immediate forensic context.

`backtest/sweep_runner_template.py` — canonical copy-paste skeleton for new sweep runners: `build_sweep_env` + `preflight_smoke` + `ProcessPoolExecutor` + empty-streak kill switch (`MAX_EMPTY_STREAK=3`). Do not modify the frozen v651 runner; use this template for every new wave.

10.4 Persistence (Railway Volume)

Variable	Default	Notes
`STATE_DB_PATH`	`/data/state.db`	v5.1.8 SQLite-backed state store (replaces JSON)
`PAPER_STATE_PATH`	`paper_state.json`	Legacy JSON path; kept for one-shot import migration
`PAPER_LOG_PATH`	`investment.log`	Trade log file path (`paper_log()`)
`TRADE_LOG_FILE`	(derived)	Append-only `.jsonl` for full trade log
`TICKERS_FILE`	`tickers.json`	Editable ticker universe

10.5 Risk / strategy

Variable	Default	Notes
`DAILY_LOSS_LIMIT`	`-500`	Halt new entries when realized P&L ≤ floor
`OR_WINDOW_MINUTES`	`5`	Width of the opening range (used for premarket seed)
`OR_STALE_THRESHOLD`	`0.05`	OR vs live drift before entry skip (5%)
`ENTRY_EXTENSION_MAX_PCT`	`1.5`	Max % entry can extend past OR trigger
`ENTRY_STOP_CAP_REJECT`	`1`	Reject entries that would need the 0.75% cap to fire
`TIGER_V2_DI_THRESHOLD`	`25`	DI+/− floor on resampled 5-min bars
`TIGER_V2_REQUIRE_VOL`	`false`	Optional volume confirmation for Tiger v2
`PAPER_DOLLARS_PER_ENTRY`	`10000`	Dollar size per paper entry
`DI_PREMARKET_SEED`	`1`	Include premarket bars in startup DI seed

10.6 Dashboard

Variable	Default	Notes
`DASHBOARD_PASSWORD`	unset	If unset, dashboard does **not** start. Min 8 chars.
`DASHBOARD_PORT`	`8080`	aiohttp listen port
`DASHBOARD_SESSION_SECRET`	unset	Override the persisted `dashboard_secret.key` HMAC key
`DASHBOARD_TRUST_PROXY`	unset	Set to `1` behind Railway / Cloudflare for client-IP

10.7 Executors (Val / Gene)

Each executor uses the prefix below; replace `{P}` with `VAL_` or `GENE_`. Paper and live keys are **strictly independent** — paper mode reads only `<PREFIX>ALPACA_PAPER_KEY` / `<PREFIX>ALPACA_PAPER_SECRET`, live mode reads only `<PREFIX>ALPACA_LIVE_KEY` / `<PREFIX>ALPACA_LIVE_SECRET`. No fallback. (v5.0.4 reverted the brief v5.0.3 paper→un-prefixed fallback; see §11.7.)

Variable	Default	Notes
`{P}ENABLED`	`1`	Set to `0` (or `false`) to skip startup
`{P}ALPACA_PAPER_KEY`	empty	Paper-only Alpaca key. Required to start in paper mode.
`{P}ALPACA_PAPER_SECRET`	empty	Paper-only secret.
`{P}ALPACA_LIVE_KEY`	empty	Live-only key. Required only to flip live.
`{P}ALPACA_LIVE_SECRET`	empty	Live-only secret.
`{P}TELEGRAM_TG`	empty	Per-executor Telegram bot token
`{P}TELEGRAM_CHAT_ID`	empty	v5.0.3: optional seed only (auto-learn fills the map)
`{P}EXECUTOR_CHATS_PATH`	`/data/executor_chats_{name}.json`	v5.0.3 auto-learned chat-map
`{P}DOLLARS_PER_ENTRY`	`10000`	Dollar size per executor entry
`{P}MAX_PCT_PER_ENTRY`	`10.0`	v5.1.4 equity-aware cap: max % of equity per entry
`{P}MIN_RESERVE_CASH`	`500`	v5.1.4 equity-aware cap: USD cash floor after entry
`ALPACA_ENDPOINT_PAPER`	unset	URL host override (no `/v2`); shared by both executors
`ALPACA_ENDPOINT_TRADE`	unset	URL host override (no `/v2`); shared by both executors

Production Railway naming (as of v5.3.0).

- Val executor — paper. `VAL_ALPACA_PAPER_KEY` (26 chars) and `VAL_ALPACA_PAPER_SECRET` (44 chars) are set on Railway from Val's paper Alpaca account. Val executor starts in paper mode at boot.
- Gene executor — currently NOT booting paper. `GENE_ALPACA_KEY` / `GENE_ALPACA_SECRET` exist on Railway but are **LIVE** keys, and the paper-prefixed names (`GENE_ALPACA_PAPER_KEY` / `GENE_ALPACA_PAPER_SECRET`) are unset — so the executor startup gate correctly logs `[Gene] skipped (GENE_ALPACA_PAPER_KEY set=False)` and skips. To bring Gene up in paper mode, mint a fresh paper key from Gene's paper Alpaca account and set the paper-prefixed names; do NOT rename the existing live keys.

10.8 Forensic Volume Filter (v5.1.0+)

Variable	Default	Notes
`VOL_GATE_ENFORCE`	`0`	Master enforcement flag. Stays `0` through v5.3.0 (shadow only).
`VOL_GATE_TICKER_ENABLED`	`1`	Active config: anchor on per-ticker volume gate.
`VOL_GATE_INDEX_ENABLED`	`1`	Active config: anchor on QQQ "Market Tide" gate.
`VOL_GATE_TICKER_PCT`	`70`	Active-config ticker threshold (percent of ToD avg).
`VOL_GATE_QQQ_PCT`	`100`	Active-config QQQ threshold (percent of ToD avg).
`VOL_GATE_INDEX_SYMBOL`	`QQQ`	Hard-locked to QQQ per Val.
`VOLUME_PROFILE_DIR`	`/data/volume_profiles`	Per-ticker baseline JSONs.

Defaults preserve the v5.1.1 Apr 20-24 backtest-recommended thresholds. The five `SHADOW_CONFIGS` (TICKER+QQQ, TICKER_ONLY, QQQ_ONLY, GEMINI_A, BUCKET_FILL_100) plus the two event-driven extras (REHUNT_VOL_CONFIRM, OOMPH_ALERT) are **not** env-driven — they are hard-coded module constants in `volume_profile.py` and `trade_genius.py`; env vars only control which one would gate trades if `VOL_GATE_ENFORCE=1`. See §17 for details.

10.9 Testing

Variable	Default	Notes
`SSM_SMOKE_TEST`	unset	When `=1`, skips the network-touching startup at import time

10.10 Do not set

`TZ` — the bot uses `zoneinfo` and `pytz` internally; setting this breaks scheduling. Documented in `.env.example`.

---

11. Known gotchas

11.1 Dockerfile per-file COPY whitelist

`Dockerfile` does **not** `COPY . .` — it explicitly copies each top-level Python module:

```
COPY trade_genius.py .
COPY telegram_commands.py .
COPY paper_state.py .
COPY side.py .
COPY error_state.py .
COPY tiger_buffalo_v5.py .   # v5.0.0
COPY volume_profile.py .     # v5.1.0
COPY indicators.py .         # v5.1.2
COPY bar_archive.py .        # v5.1.2
COPY persistence.py .        # v5.1.8
COPY shadow_pnl.py .         # v5.2.0
COPY dashboard_server.py .
COPY dashboard_static/ ./dashboard_static/
```

Footgun. Any new top-level Python module must be added to this whitelist in the same PR or the container crashes on boot with `ModuleNotFoundError`. v4.11.0 shipped `error_state.py` without the matching `COPY` line and prod stayed 502 for ~3 hours (`v4.11.1` was the one-line fix). v5.0.0 shipped `tiger_buffalo_v5.py` with the same omission and prod was again 502 until `v5.0.2`. v5.0.2 added a CI infra-guard test (`smoke_test.py: infra: Dockerfile COPY whitelist includes every top-level imported module`) that parses every `import` / `from` line in `trade_genius.py`, intersects against local `.py` modules, and grep-extracts the `COPY <module>.py` directives in the Dockerfile — any missing entry fails CI before merge. v5.1.0 added `COPY volume_profile.py`; v5.1.2 added `COPY indicators.py` and `COPY bar_archive.py`; v5.1.8 added `COPY persistence.py`; v5.2.0 added `COPY shadow_pnl.py` per this guard.

`COPY tickers.json` is intentionally absent — the file lives on the Railway Volume (`TICKERS_FILE=/data/tickers.json`) and is created at runtime by `_save_tickers_file()`.

11.2 Yahoo "real-time" is not real-time

Yahoo's `v8/finance/chart` endpoint returns delayed quotes (typically ~15 minutes for cash indices). The dashboard treats the cash + futures rows as *informational* — the bot never trades off Yahoo data; entries and stop management run off the Alpaca / FMP feed. The frontend's `data delayed` chip is the operator-facing signal that the Yahoo fallback engaged for the *whole batch*.

11.3 `marketState` is `None`

Yahoo's chart-endpoint metadata `marketState` field comes back `null` on most responses regardless of session. The dashboard intentionally does **not** trust it; `_classify_session_et()` derives the session from the local `America/New_York` clock instead.

11.4 Dashboard is read-only

By design, no dashboard endpoint mutates bot state, places orders, or toggles anything. Every shared-state read is best-effort and silent on failure (`_safe(fn, default)`); a dashboard exception cannot tank the bot.

11.5 Don't import `trade_genius` from helper threads

`trade_genius.py` is the program's `__main__`; doing a fresh `import trade_genius` from another thread re-executes the module top level (which calls `loop.add_signal_handler(...)` from a non-main thread and

crashes). `dashboard_server._ssm()` resolves the live module via `sys.modules['__main__']` first; helper modules (`paper_state.py` , `telegram_commands.py`) follow the same pattern.

11.6 Executor trade-confirmation DM (v5.0.3 auto-learn)

Each executor's Telegram bot ships its trade confirmations via `_send_own_telegram` , which fans out to a learned `owner_id -> chat_id` map persisted at `/data/executor_chats_{name}.json` (path overridable via `<PREFIX>EXECUTOR_CHATS_PATH`). The map is populated transparently on the existing `_auth_guard` choke point — any inbound DM from an owner registers that owner's chat_id, atomically writes to disk, and survives Railway redeploys. **No `<PREFIX>TELEGRAM_CHAT_ID` env var is required** (it's accepted as a back-compat seed only). If the map is empty at signal time, the bot logs once at startup (`[Val] notifications EMPTY — DM this executor's bot /start ...`) and silently skips that signal — the trade still hits Alpaca, but the operator must DM the bot once to enable confirmations. This is the bug v5.0.3 fixed; pre-5.0.3 the executor silently dropped every confirmation when `TELEGRAM_CHAT_ID` was unset on Railway.

11.7 Paper and live Alpaca keys are independent (v5.0.4 revert)

Alpaca paper keys and live (real-money) keys are independent credentials issued against different endpoints. They are **not** interchangeable and must **never** be silently substituted for one another. v5.0.3 briefly added a fallback in `TradeGeniusBase.__init__` that read `<PREFIX>ALPACA_PAPER_KEY` and fell back to `<PREFIX>ALPACA_KEY` if the paper key was unset; v5.0.4 reverted that fallback after Val confirmed `GENE_ALPACA_KEY` on Railway is a LIVE key. Had Gene's executor instantiated under v5.0.3, paper-mode orders would have been routed through live credentials. The correct contract — enforced both by `__init__` (v5.0.4 strict reads) and the executor startup gate at module scope — is that paper mode reads **only** `<PREFIX>ALPACA_PAPER_KEY` / `<PREFIX>ALPACA_PAPER_SECRET` , and live mode reads **only** `<PREFIX>ALPACA_LIVE_KEY` / `<PREFIX>ALPACA_LIVE_SECRET` . Operators provisioning a new executor must set the paper-prefixed env vars from a paper Alpaca account; do not repurpose un-prefixed keys, which by repo convention are the live-account credentials.

---

12. Persistence

12.1 SQLite store (v5.1.8+)

`persistence.py` owns the canonical state store at `STATE_DB_PATH` (default `/data/state.db` on the Railway Volume). The legacy `paper_state.json` is read once on first boot if the SQLite file does not yet exist and its rows are imported into the new schema; subsequent boots ignore the JSON. The store holds the same logical fields the JSON held — `paper_cash` , `positions` , `short_positions` , `paper_trades` , `paper_all_trades` , `daily_entry_count` , `daily_short_entry_count` , `or_high` , `or_low` , `pdc` , `_trading_halted` , `_trading_halted_reason` , `bot_version` — plus the v5.2.0 `shadow_positions` table backing `shadow_pnl.ShadowPnL` , plus the v5.5.10 `executor_positions` table backing the executor mirrors (see § 12.1.1 below). Writes are routed through `persistence.save_*` helpers; failures never raise into the trading loop. The store survives Railway redeploys via the persistent `/data` volume and is the source of truth for `_load_state()` after v5.1.8.

12.1.1 `executor_positions` table (v5.5.10+)

`TradeGeniusBase.self.positions` is the executor-side mirror of the broker's open-position set, populated by `_record_position` after a successful submit and by `_reconcile_broker_positions` at boot. Through v5.5.9 this dict was process-local: every reboot started with `{}`, and `_reconcile_broker_positions` then "grafted" every broker-side position as an orphan with a Telegram alert — even when the bot had itself opened those positions normally moments before the restart.

v5.5.10 mirrors the dict to a new `executor_positions` table in `state.db`:

```
CREATE TABLE executor_positions (
  executor_name TEXT NOT NULL,    -- 'Val' or 'Gene'
  mode TEXT NOT NULL,             -- 'paper' or 'live'
  ticker TEXT NOT NULL,
  side TEXT NOT NULL,             -- 'LONG' or 'SHORT'
  qty INTEGER NOT NULL,
  entry_price REAL NOT NULL,
  entry_ts_utc TEXT NOT NULL,
  source TEXT NOT NULL,           -- 'SIGNAL' | 'RECONCILE' | 'MANUAL'
  stop REAL,
  trail REAL,
  last_updated_utc TEXT NOT NULL,
  PRIMARY KEY (executor_name, mode, ticker)
);
```

The PK includes both `executor_name` AND `mode` so Val/paper, Val/live, Gene/paper, and Gene/live each have an independent row set. A `/mode` flip wipes the in-memory dict and reloads the bucket for the new mode so paper rows never bleed into live or vice versa.

Three lifecycle hooks keep the dict and the table in sync:

- `_load_persisted_positions()` is called from `__init__` BEFORE `_reconcile_broker_positions()` runs in `start()`, so a plain reboot during a live session sees the persisted dict already populated.
- `_record_position(...)` calls `_persist_position(ticker)` immediately after stamping `self.positions[ticker]`.
- `_remove_position(ticker)` is the single hook every position-close path calls (`EXIT_LONG` / `EXIT_SHORT` / `EOD_CLOSE_ALL` / `cmd_halt`); it pops the dict entry and deletes the DB row in one call.

`_reconcile_broker_positions` reframes as a true safety net with three explicit outcomes, distinguished by set comparison of persisted-tickers vs broker-tickers:

1. Persisted == broker → INFO log `[RECONCILE] clean: N position(s) match broker`, no Telegram (the common reboot case).
2. Broker has tickers persisted does not → graft as today (source= `RECONCILE`, persist the new row), WARN log per orphan, single Telegram suffixed `(true divergence)`.
3. Persisted has tickers broker does not → quiet self-heal: WARN log + `_remove_position(ticker)`, no Telegram, no close/exit-path call.

12.2 Legacy JSON shape (still relevant for the import path)

The pre-v5.1.8 `paper_state.json` shape is preserved here because the JSON-to-SQLite import path reads it verbatim and the synthetic harness golden files retain the same shape:

```
{
  "paper_cash": 97543.21,
  "positions": {
    "NVDA": {
      "entry_price": 142.50, "shares": 70, "entry_count": 1,
      "stop": 141.44, "initial_stop": 141.23,
      "trail_high": 143.80, "trail_active": true,
      "entry_time": "2026-04-24T13:42:11Z"
    }
  },
  "short_positions": { "...": "..." },
  "paper_trades":     [ "...today's trades..." ],
  "paper_all_trades": [ "...all trades, capped at 500..." ],
  "daily_entry_count":       { "NVDA": 1 },
  "daily_short_entry_count": { "NVDA": 0 },
  "or_high": { "NVDA": 142.10, "...": "..." },
  "or_low":  { "...": "..." },
  "pdc":     { "SPY": 565.30, "QQQ": 480.18, "...": "..." },
  "_trading_halted": false,
  "_trading_halted_reason": "",
  "bot_version": "4.13.0"
}
```

A separate append-only `trade_log.jsonl` (path resolved by `TRADE_LOG_FILE`) logs every entry/exit with full context — the dashboard's `/api/trade_log` reads its tail.

Unknown fields on disk (e.g. legacy `avwap_data`, `avwap_last_ts`) are silently ignored on load so old state files survive code purges.

---

13. Testing

13.1 Local smoke (`smoke_test.py`)

262/262 tests at v5.3.0 (was 161 at v5.0.4, 220 at v5.1.2, 262 at v5.3.0). Tests are grouped via the `@t(name)` decorator. Runs on every PR via `.github/workflows/post-deploy-smoke.yml` (after the prod rollout poll succeeds). Coverage spans the trade-decision arithmetic (stop-chain layers, breakeven ratchet, ladder tiers, entry-extension guard), data helpers (`_classify_session_et`, `_or_price_sane`), state I/O (now `persistence.py` / SQLite via v5.1.8), the index payload schema, the Yahoo helper API surface, the v5 Tiger/Buffalo rule grid (every `L-P*-R*` / `S-P*-R*` / `C-R*` rule has at least one test docstring citing the rule ID), the v5.0.2 Dockerfile COPY infra-guard, the v5.0.3 chat-map auto-learn / fan-out, the v5.0.4 paper-key strict read, the v5.1.0 `[VOLPROFILE]` section (calendar helpers, `session_bucket` boundaries, `evaluate_g4` PASS/BLOCK paths, disable-on-30-symbol cap, JSON round-trip), the v5.1.1 env-driven A/B toggles + parallel `SHADOW_CONFIGS` emit, the v5.1.2 forensic capture additions (`bar_archive.write_bar` + `cleanup_old_dirs`, `indicators.rsi14` / `ema9` / `ema21` / `atr14` / `vwap_dist_pct` / `spread_bps` happy-

path + insufficient-bars-returns-None, emitter formats for `[V510-MINUTE]` / `[V510-CAND]` / `[V510-FSM]` / `[V510-ENTRY]`), the v5.1.4 equity-aware sizing caps (`MAX_PCT_PER_ENTRY`, `MIN_RESERVE_CASH`), the v5.1.6 bucket-fill velocity streams (`[V510-VEL]` / `[V510-IDX]` / `[V510-DI]`), the v5.1.8 SQLite migration (JSON-import idempotency, schema, write barriers), the v5.1.9 `REHUNT_VOL_CONFIRM` arm/check + `OOMPH_ALERT` minute-1/2 state machine, the v5.2.0 `shadow_pnl` tracker (open/close/MTM, `open_positions_for`, `recent_closed_for`), the v5.2.1 idempotency layer (`_submit_order_idempotent` deterministic `client_order_id`, `_reconcile_broker_positions` startup graft, EOD orphan force-close at entry_price with `EOD_NO_MARK`, shadow MTM ungated from `paper_holds`, `_v521_all_shadow_config_names` registry, `(ticker, side)`-keyed REHUNT watch), and the v5.3.0 dashboard wiring (per-config open + recent-closed list, Shadow tab visibility), plus dashboard-route contract tests against a stub aiohttp app.

13.2 Synthetic harness (`synthetic_harness/`)

50 named, byte-equal-replay scenarios under `synthetic_harness/scenarios/{long_entries, short_entries, long_closes, short_closes, scan_loops, edge_cases}.py` with goldens under `synthetic_harness/goldens/*.json`. CLI:

```
python -m synthetic_harness list
python -m synthetic_harness record <scenario>
python -m synthetic_harness replay <scenario>
python -m synthetic_harness diff   <scenario>
```

Each scenario seeds module state through `_reset_module_state`, drives the bot through a `FrozenClock` and `SyntheticMarket`, and records `OutputRecorder` events. `replay_scenario()` strips `trade_genius_version` from both the observed and golden dicts before compare (v4.11.5), so a `BOT_VERSION` bump alone never invalidates all 50 goldens. `record_scenario()` still stamps the current version into freshly-recorded goldens.

`smoke_test.py --synthetic` lifts the local smoke total by replaying all 50 goldens. **v5.1.0 / v5.1.1 / v5.1.2 are observation-only**, so the 50/50 byte-equal still passes after each release without re-record.

13.3 CI guards

- **`version-bump-check.yml`** — blocks PRs to `main` unless **both** `BOT_VERSION` in `trade_genius.py` and the top `##` heading in `CHANGELOG.md` change in the diff. **Escape hatch:** include the literal token `[skip-version]` anywhere in the PR title or body. Use it only for docs-only / CI-only changes.
- **`post-deploy-smoke.yml`** — fires on every push to `main`. Polls `/api/version` against `https://tradegenius.up.railway.app` until the new `BOT_VERSION` is live (or times out at 5 min), then runs the full smoke suite. v4.11.4 repointed `DASHBOARD_URL` here from the pre-rename Railway domain that had been 404'ing since v3.5.1.

---

14. Deployment

14.1 Railway

1. Connect the GitHub repo. Railway auto-builds and deploys on every push to `main` (`railway.json` + `nixpacks.toml` ; Docker builder is the canonical path via `Dockerfile`).
2. Attach a Railway Volume; mount it at `/data` .
3. Set the env vars listed in §10. At minimum: `TELEGRAM_TOKEN` , `CHAT_ID` , `TRADEGENIUS_OWNER_IDS` , `DASHBOARD_PASSWORD` , plus the executor key sets you want active.
4. Verify rollout via `https://tradegenius.up.railway.app/api/version` .

14.2 Local development

```
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env  # then edit
DASHBOARD_PASSWORD=devpassword python trade_genius.py
```

Smoke tests against the running local instance:

```
DASHBOARD_URL=http://localhost:8080 \
DASHBOARD_PASSWORD=devpassword \
python smoke_test.py --local --prod
```

14.3 Logging

Dual handler: file (`trade_genius.log`) + stdout. All stdout/stderr is visible in the Railway dashboard. Errors that go through `report_error()` also fan out to the matching executor's Telegram channel (with a 5-minute per-`(executor, code)` dedup) and increment the dashboard health pill's per-executor count.

---

15. Maintenance

15.1 Regenerating `trade_genius_algo.pdf`

```
pip install weasyprint
python scripts/build_algo_pdf.py
```

The script renders this `ARCHITECTURE.md` plus a cover page and table-of-contents into `trade_genius_algo.pdf` at the repo root. Re-run it after any architecture change you want reflected in the PDF; commit the regenerated binary in the same PR. The script is intentionally a single file with no project-specific dependencies beyond `weasyprint` and the markdown source so it survives module renames.

15.2 Bumping `BOT_VERSION`

Bump the constant on line 61 of `trade_genius.py` AND prepend a `## v{N} — YYYY-MM-DD — <one-line summary>` heading at the top of `CHANGELOG.md`. The CI `version-bump-required` job blocks the PR otherwise. For **docs-only** or **CI-only** PRs (this one included), leave `BOT_VERSION` alone and put `[skip-version]` in the PR title.

15.3 Adding a new top-level Python module

Always add a matching `COPY <name>.py .` line to the `Dockerfile`. The container will not pick the file up otherwise and will crash on boot. (See §11.1.)

15.4 Reading the live state without restarting

Every piece of bot state is reachable through the dashboard's `/api/state`, `/api/executor/{name}`, `/api/errors/{executor}`, and `/api/trade_log` endpoints — none of them mutate, all of them are cookie-authenticated, and the state snapshot is cached for ≤ 10 s so hammering the dashboard does not stall the scanner.

---

16. v5 strategy quick reference

> Mirror summary of `STRATEGY.md` for at-a-glance reading. The spec is still authoritative — this section just collects the rule-ID grid in one place so a reader paging through the architecture doc can find the entry/exit thresholds without flipping files.

16.1 State machine

```
IDLE —[L-P1/S-P1 gates]→ ARMED —[L-P2-R2 / S-P2-R2 2× DI≥25]→ STAGE_1
   STAGE_1 —[L-P3-R2 / S-P3-R2 2× DI≥30 + winning rule]→ STAGE_2
   STAGE_2 —[next 5m close]→ TRAILING
   TRAILING —[stop / DI<25]→ EXITED —[reclaim once]→ ARMED
   EXITED —[2nd exit]→ LOCKED_FOR_DAY
```

16.2 Rule grid

ID	Where it fires
L-P1-G1..G4	Long permission gates (4 lights on dashboard).
L-P2-R1..R5	Stage-1 long entry (50%, prior-5m-low stop, record entry).
L-P3-R1..R5	Stage-2 long add (full size, safety lock to entry price).
L-P4-R1..R4	TRAILING long: 5m HL ratchet up; stop OR DI<25 ⇒ flatten.
L-P5-R1..R3	Long re-hunt: dormant until reclaim; one shot; 2nd exit ⇒ LOCK.
S-P1-G1..G4	Short permission gates; indices-green vetoes shorts entirely.
S-P2-R1..R5	Stage-1 short entry (50%, prior-5m-high stop).
S-P3-R1..R5	Stage-2 short add; safety lock to entry.
S-P4-R1..R5	TRAILING short: 5m LH ratchet down; **DI<25 priority-1.**
S-P5-R1..R3	Short re-hunt; 2nd exit ⇒ LOCK.
C-R1	Long ⊥ short on same ticker per session.
C-R2	DMI period = 14 (down from v4's 15).
C-R3	Closed-candle entries; tick-rate exits.
C-R4	Daily-loss-limit ⇒ LOCKED_FOR_DAY for all tracks.
C-R5	EOD 15:55 ET force-close.
C-R6	Sovereign Regime Shield global kill.
C-R7	9-name spike universe; SPY/QQQ pinned, never traded.

16.3 Where the v5 logic lives

Concern	File / function
Rule logic (pure helpers)	`tiger_buffalo_v5.py` (no imports from `trade_genius`)
Per-track state	`trade_genius.v5_long_tracks` / `v5_short_tracks`
Mutex + active direction	`trade_genius.v5_active_direction`
DMI on 1m + 5m timeframes	`trade_genius.v5_di_1m_5m()`
First-hour high (L-P1-G4)	`trade_genius.v5_first_hour_high()`
Opening-range-low (S-P1-G4)	`trade_genius.v5_opening_range_low_5m()`
Lock all tracks	`trade_genius.v5_lock_all_tracks(reason)`
Persistence	`paper_state.py` — extends save/load with v5 keys
Tests	`smoke_test.py` (search for `L-P` or `S-P`)

16.4 Migration from v4

A v4 paper-state file (no `v5_*` keys) loads cleanly: the loader treats each missing track as IDLE via `tiger_buffalo_v5.load_track()`. The first session under v5.0.0 starts with empty track dicts; the state machine builds up state lazily as gates pass and stages fire. v4 `paper_trades`, `trade_history`, `positions`, `short_positions`, and the daily entry counters all roll forward unchanged.

16.5 v5.0.x → v5.4.x — what changed (and what didn't)

The Tiger/Buffalo state machine in §6 is the same code in v5.4.1 as it was in v5.0.0. Every v5.1.x / v5.2.x / v5.3.x / v5.4.x release is observation-, infrastructure-, or display-only:

Release	Subject	Behavior change?
v5.0.2	Dockerfile `COPY tiger_buffalo_v5.py` + infra-guard test	No (hotfix)
v5.0.3	Per-executor trade-confirmation DM auto-learn (chat-map)	No (plumbing)
v5.0.4	Revert v5.0.3 paper-key fallback (paper/live key strict)	No (safety)
v5.1.0	Forensic Volume Filter (`volume_profile.py`) — SHADOW	No (logging)
v5.1.1	Env-driven A/B toggles + 3-config parallel shadow logging	No (logging)
v5.1.2	Forensic capture: bar archive + indicator snapshots	No (logging)
v5.1.3	Finnhub data source removed (Yahoo + Alpaca + FMP only)	No (cleanup)
v5.1.4	Equity-aware position sizing (`MAX_PCT_PER_ENTRY` / `MIN_RESERVE_CASH`)	No (sizing safety)
v5.1.5	`/test` command timeout fix	No (UX)
v5.1.6	`BUCKET_FILL_100` 5th shadow config + V510-VEL/IDX/DI streams	No (logging)
v5.1.8	SQLite-backed persistence on `/data/state.db` (replaces JSON)	No (storage)
v5.1.9	`REHUNT_VOL_CONFIRM` + `OOMPH_ALERT` shadow configs (7 active)	No (logging)
v5.2.0	Real-time shadow-strategy P&L tracker on dashboard	No (display)
v5.2.1	Alpaca order idempotency + startup broker reconcile + shadow accounting fixes (H1/H2/H3/M3/M4)	No (safety)
v5.3.0	Shadow strategies moved to dedicated tab + per-config detail	No (display)
v5.3.1	Doc refresh: ARCHITECTURE.md + `trade_genius_algo.pdf` to v5.3.0 state	No (docs)
v5.4.0	Offline backtest CLI (`backtest/` package + `python -m backtest.replay` with `--validate`)	No (offline)
v5.4.1	Shadow tab charts (equity curves + day heatmap + win-rate sparklines) + `/api/shadow_charts` endpoint	No (display)
v5.4.2	Doc refresh: ARCHITECTURE.md + `trade_genius_algo.pdf` to v5.4.1 state	No (docs)

Result: a v5.0.0 paper-state file still loads cleanly on v5.4.1 (via the v5.1.8 one-shot JSON-to-SQLite import path); the synthetic harness 50/50 byte-equal goldens still match; every entry, exit, sizing, and stop-placement decision is identical to v5.0.0.

---

17. Forensic Volume Filter (Anaplan logic) — v5.1.0 → v5.4.1

The §17 layer asks a sharper volume question than the legacy "is volume high" tests: *is this minute's volume higher than the 55-trading-day seasonal average for THIS exact ET timestamp?* Shipped in v5.1.0 in **shadow mode** (computed and logged, but no entry decision changes), extended in v5.1.1 with env-driven A/B toggles and 3-config parallel shadow logging, in v5.1.2 with `GEMINI_A` as a 4th shadow config, in v5.1.6 with `BUCKET_FILL_100` as a 5th plus the bucket-fill velocity streams, and in v5.1.9 with two event-driven extras (`REHUNT_VOL_CONFIRM` and `OOMPH_ALERT`) that own their own virtual positions. **Seven** shadow configs are active through v5.3.0; the gate remains observation only — `VOL_GATE_ENFORCE` defaults to `0` and is unchanged.

17.1 Volume Library — parameters

Parameter	Value
Window	55 NYSE trading days, weekends/holidays excluded.
Bucket	Per-minute, ET, `"0931".."1559"`. 09:30 auction excluded.
Baseline feed	Alpaca historical 1m bars, `feed=sip` , `end < now-16min` .
Live feed	Alpaca websocket `/iex` (free-plan, 30-symbol cap).
Cross-feed scale	Per-ticker IEX/SIP volume ratio over the window. Median is published on the IEX scale.
Per-bucket store	`{"median": int, "p75": int, "p90": int, "n": int}` .
Holiday calendar	Hard-coded NYSE list for 2026-2027 in `volume_profile.py` . No new dep.
Stale threshold	36 hours since `build_ts_utc` .
Disk format	`/data/volume_profiles/<TICKER>.json` (atomic tmp+rename).
Rebuild cadence	Nightly at 21:00 ET (daemon thread); synchronous on startup if any profile is missing or stale.
30-symbol cap	If `len(TICKERS) > 30` , the module hard-disables itself (`VOLUME_PROFILE_ENABLED = False`); the bot trades normally with G4 short-circuited to `DISABLED` .
Websocket reconnect	Jittered backoff; 5-min REST replay on resume to repopulate the in-memory volume table.

17.2 V-P1 grid (legacy fixed-threshold evaluator)

`evaluate_g4(ticker, minute_bucket, current_volume, profile, qqq_current_volume, qqq_profile, stage)` returns `{green, reason, ticker_pct, qqq_pct, rule}` with the original v5.1.0 fixed thresholds (Stage 1: ticker ≥ 120% AND QQQ ≥ 100%; Stage 2: ticker ≥ 100%). Preserved unchanged so v5.1.0 grep tooling and the synthetic harness 50/50 byte-equal still pass.

17.3 Env-driven active config + helper (v5.1.1+)

`load_active_config()` reads six env vars at module-import time:

Env var	Default	Role
`VOL_GATE_ENFORCE`	`0`	Master enforcement flag. Stays `0` through v5.1.2.
`VOL_GATE_TICKER_ENABLED`	`1`	Active config: anchor on per-ticker.
`VOL_GATE_INDEX_ENABLED`	`1`	Active config: anchor on QQQ.
`VOL_GATE_TICKER_PCT`	`70`	Active-config ticker threshold (% of ToD avg).
`VOL_GATE_QQQ_PCT`	`100`	Active-config QQQ threshold (% of ToD avg).
`VOL_GATE_INDEX_SYMBOL`	`QQQ`	Hard-locked to QQQ.

Defaults reflect the **Apr 20-24 backtest recommendation**: 70/100 was the best risk-adjusted config (11 trades, +$482.90, 82% win rate; retains 96% of upside vs unfiltered). Garbage-int env values fall back to defaults rather than crash. Env reads are dict-lookup-cheap; the design intent is "set once at deploy, don't flip mid-week."

`evaluate_g4_config(ticker, minute_bucket, current_volume, profile, index_current_volume, index_profile, *, ticker_enabled, index_enabled, ticker_pct, index_pct)` is the per-anchor configurable evaluator used for the parallel shadow lines. It returns `{verdict, reason, ticker_pct, qqq_pct}` with `verdict ∈ {PASS, BLOCK}` and `reason ∈ {OK, LOW_TICKER, LOW_QQQ, STALE_PROFILE, NO_BARS, NO_PROFILE, DISABLED}`.

17.4 `SHADOW_CONFIGS` — fixed analysis configs (v5.1.1 → v5.1.9)

`volume_profile.SHADOW_CONFIGS` is a hard-coded **5-tuple** (was 3-tuple in v5.1.1; `GEMINI_A` was added in v5.1.2; `BUCKET_FILL_100` was added in v5.1.6 to support the Bucket-Fill Protocol intraminute velocity capture — see §17.4a). Two further configs ship in v5.1.9 as event-driven extras outside the tuple:

Name	ticker_enabled	index_enabled	ticker_pct	index_pct	Source
`TICKER+QQQ`	True	True	70	100	`volume_profile`
`TICKER_ONLY`	True	False	70	(unused)	`volume_profile`
`QQQ_ONLY`	False	True	(unused)	100	`volume_profile`
`GEMINI_A`	True	True	**110**	**85**	`volume_profile`
`BUCKET_FILL_100`	True	True	**100**	**100**	`volume_profile`
`REHUNT_VOL_CONFIRM`	n/a	n/a	event-driven (≥100% bucket median + DI≥25 within 10 min after `HARD_EJECT_TIGER`)	—	`_V521_EXTRA_SH` (v5.1.9)
`OOMPH_ALERT`	n/a	n/a	event-driven (DI≥25 + BUCKET_FILL≥100% on minute 1; DI≥25 confirm on minute 2)	—	`_V521_EXTRA_SH` (v5.1.9)

These are NOT env-driven — env vars only choose which one would gate trades if `VOL_GATE_ENFORCE=1`. The point is that every line of shadow data is comparable across all seven configs post-hoc. The **v5.2.1 M3** helper `_v521_all_shadow_config_names()` is the canonical union; close-fanout (`_v520_close_shadow_all`) and EOD code paths iterate it so any new tuple entry plus the event-driven extras is picked up automatically.

17.4a Bucket-Fill Velocity (v5.1.6)

The `BUCKET_FILL_100` row above is paired with three new log streams that capture the data needed to evaluate the Bucket-Fill Protocol post-hoc (see Gene's design note). All three are pure observation; none of them touch the trading decision.

- `[V510-VEL] ticker=X minute=HH:MM second=N running_vol=V bucket=B pct=P qqq_pct=Q` fires **once per (ticker, minute)** on the FIRST tick where running IEX volume crosses 100% of the bucket median for the active candle. This is the data that lets us evaluate the "fires at second 40" velocity hypothesis — the closed-minute backtest cannot see this.
- `[V510-IDX] spy_close=X spy_pdc=Y spy_above=Y/N qqq_close=A qqq_pdc=B qqq_above=Y/N` fires once per candidate consideration. Required for the L-P1 / S-P1 index-direction leg of the protocol (QQQ > PDC AND SPY > PDC, long; mirror for short).
- `[V510-DI] ticker=X di_plus_t-1=A di_plus_t=B di_minus_t-1=C di_minus_t=D double_tap_long=Y/N double_tap_short=Y/N` fires once per candidate consideration. Required for the L-P2 / S-P2 "double-tap" leg (DI+ > 25 on both the prior and current 1-minute bar; mirror DI- for short). DI+ / DI- are computed via Wilder's smoothing in `indicators.py` (`di_plus`, `di_minus`).

17.5 Shadow log lines

Per candidate, `_shadow_log_g4` emits **6** log lines (one back-compat plus five `[CFG=...]` lines for the tuple). The v5.1.9 event-driven extras emit one additional `[V510-SHADOW][CFG=...]` line each on their own trigger conditions:

```
[V510-SHADOW] ticker=AMD bucket=1448 stage=1 g4=GREEN ticker_pct=84 qqq_pct=112 reason=OK entry_decision=ENT
[V510-SHADOW][CFG=TICKER+QQQ][PCT=70/100] ticker=AMD bucket=1448 stage=1 t_pct=84 qqq_pct=112 verdict=PASS
[V510-SHADOW][CFG=TICKER_ONLY][PCT=70]    ticker=AMD bucket=1448 stage=1 t_pct=84              verdict=PASS
[V510-SHADOW][CFG=QQQ_ONLY][PCT=100]      ticker=AMD bucket=1448 stage=1            qqq_pct=112 verdict=PASS
[V510-SHADOW][CFG=GEMINI_A][PCT=110/85]   ticker=AMD bucket=1448 stage=1 t_pct=84 qqq_pct=112 verdict=BLOCK
[V510-SHADOW][CFG=BUCKET_FILL_100][PCT=100/100] ticker=AMD bucket=1448 stage=1 t_pct=84 qqq_pct=112 verdict=
[V510-SHADOW][CFG=REHUNT_VOL_CONFIRM] ticker=AMD side=long bucket=1453 t_pct=104 di_plus=27 verdict=PASS
[V510-SHADOW][CFG=OOMPH_ALERT] ticker=AMD side=long bucket=1448 minute1_di=27 minute1_bucket=104 minute2_di=
```

`entry_decision` always reflects what the bot actually did — these lines never gate the decision. The v5.1.9 extras drop the `entry_decision` field because they fire post-hoc relative to the underlying live trade.

17.6 Failure modes

Condition	Reason
Profile file missing for ticker	`NO_PROFILE`
Profile older than `STALE_HOURS` (36h)	`STALE_PROFILE`
Bucket not in profile (e.g. 09:30 or 16:00)	`NO_BUCKET_<T>_<bucket>`
`len(TICKERS) > 30` (free IEX cap exceeded)	`DISABLED`
Ticker pct < threshold and ticker_enabled	`LOW_TICKER`
QQQ pct < threshold and index_enabled	`LOW_QQQ`

17.7 Rollout plan (current)

1. **v5.1.0** — G4 computed and logged. Existing entry decisions unchanged.
2. **v5.1.1** — Env-driven active config + 3 parallel shadow configs per candidate. Defaults preserve v5.1.0 behavior.
3. **v5.1.2** — `GEMINI_A` added as 4th shadow config; forensic capture (§18) closes the asymmetric blind spot where we only logged what fired. Still SHADOW.
4. **v5.1.6** — `BUCKET_FILL_100` added as 5th shadow config plus the `[V510-VEL]` / `[V510-IDX]` / `[V510-DI]` velocity streams (§17.4a).
5. **v5.1.9** — `REHUNT_VOL_CONFIRM` and `OOMPH_ALERT` added as event-driven extras (now 7 active configs).
6. **v5.2.0** — Real-time shadow P&L tracker (`shadow_pnl.py`) lifts the data from append-only logs into a queryable SQLite table so the dashboard can render running P&L per config.
7. **v5.3.0** — Shadow strategies move to a dedicated dashboard tab with click-to-expand per-config detail (open positions + last 10 closed trades). See §9.5 for the wiring.
8. **v5.4.0** — Offline `backtest/` replay package + `python -m backtest.replay` CLI with `--validate` mode pairing replay vs prod `shadow_positions` (§20).
9. **v5.4.1** — Shadow tab charts (equity curves, day-P&L heatmap, rolling 20-trade win-rate sparklines) backed by `/api/shadow_charts` (§21).
10. **Observation window** — Val collects multiple weeks of 7-config shadow data alongside `[V510-CAND]` / `[V510-MINUTE]` / `[V510-VEL]` / `[V510-IDX]` / `[V510-DI]` streams.

11. **Future PR** — Flip `VOL_GATE_ENFORCE=1` once Val has chosen the best config off live data; G4 joins G1/G2/G3 as a hard ARM precondition (see §19.1).

18. Forensic Capture (v5.1.2)

v5.1.2 closes four data gaps so any future backtest is fully replayable from disk:

1. We only logged trades that **fired** — never candidates that didn't.
2. We only logged at the **active threshold** — not the baseline.
3. We did **not persist 1m bars** to disk.
4. We did **not record indicator state** at decision time.

All four streams are observation-only. None of them touch entry, exit, sizing, or stop placement.

18.1 Bar archive (`bar_archive.py` — Tier-1.1)

For every minute close per ticker (+ QQQ + SPY) in the active TICKERS list, append one JSONL line to `/data/bars/YYYY-MM-DD/{TICKER}.jsonl`. Schema:

```
{
  "ts": "...", "et_bucket": "1448",
  "open": ..., "high": ..., "low": ..., "close": ...,
  "iex_volume": ..., "iex_sip_ratio_used": ...,
  "bid": ..., "ask": ..., "last_trade_price": ...
}
```

Append in `a` mode; on Linux ext4 a write of < `PIPE_BUF` (4096) is atomic, and the lines are ~150 bytes — no tmp+rename needed. Lazy directory creation. **Failure-tolerant — never raises into the trading loop.** Disk projection: ~18 tickers × 390 minutes × ~150 bytes ≈ 1 MB/day. The 30-symbol IEX cap from v5.1.0 still bounds the universe. Stale or empty minutes write nothing.

Wiring (v5.5.2). `bar_archive.py` was authored in v5.1.2, but the wrapper `_v512_archive_minute_bar(ticker, bar)` in `trade_genius.py` had **zero callers** until v5.5.2 — `/data/bars/` therefore never existed on prod, and the v5.4.0 backtest CLI had nothing to replay. v5.5.2 wires the wrapper into the per-ticker scan-loop branch alongside the existing v5.2.1 H3 mark-to-market hook. The most-recently-completed bar from the cached `fetch_1min_bars` result is projected onto the canonical `BAR_SCHEMA_FIELDS` (11 fields; `backtest/loader.py` expects this exact shape) and passed to `bar_archive.write_bar`. The call is wrapped in its own `try/except` and logs `[V510-BAR] archive hook` on failure so archival can never disrupt the trading scan. Two smoke tests (`v5.5.2: _v512_archive_minute_bar has a caller outside its own def` and `v5.5.2: bar_archive.cleanup_old_dirs is invoked from eod_close`) guard the call site against future refactor regressions.

Retention. `eod_close()` invokes `bar_archive.cleanup_old_dirs(retain_days=90)` once per session close, deleting any dated directories older than 90 days. This runs on the same path that already flattens positions, locks v5 tracks, and saves paper state, so it requires no extra scheduler. The call is failure-tolerant — a cleanup error logs at warning level and never raises.

18.2 Per-minute volume log (`[V510-MINUTE]` — Tier-1.2)

One line per ticker per minute close, regardless of candidate state:

```
[V510-MINUTE] ticker=AMD bucket=1448 t_pct=84 qqq_pct=112 close=346.19 vol=12345
```

This lets us replay "what if the candidate threshold itself were different" without re-pulling 1m bars from Alpaca.

18.3 Skipped-candidate log (`[V510-CAND]` — Tier-1.3)

Pre-v5.1.2 we only logged candidates that fired. v5.1.2 emits one line on **every entry consideration** — fired AND not-fired:

```
[V510-CAND] ticker=AMD bucket=1448 stage=1 fsm_state=ARMED entered=NO reason=NO_BREAKOUT
            t_pct=84 qqq_pct=112 close=346.19 stop=null
            rsi14=null ema9=null ema21=null atr14=null vwap_dist_pct=null spread_bps=null
```

`reason ∈ {NO_BREAKOUT, STAGE_NOT_READY, ALREADY_OPEN, COOL_DOWN, MAX_POSITIONS, BREAKOUT_CONFIRMED}`. Wired into the entry-consideration loop next to the existing `_shadow_log_g4` call. All `indicators.*` values render as `null` (not `0.0`) on insufficient bars.

18.4 Entry log enrichment (`[V510-ENTRY]` — Tier-1.4)

When a trade fires, a new line is emitted alongside the existing entry surface (Telegram + paper_log) carrying bid/ask + account state:

```
[V510-ENTRY] ticker=AMD bid=346.18 ask=346.20
             cash=85432.10 equity=99214.55
             open_positions=2 total_exposure_pct=18.4 current_drawdown_pct=0.0
```

Strictly additive — the existing entry log line, paper_log entry, and Telegram card are unchanged byte-for-byte (the synthetic harness 50/50 byte-equal goldens still pass).

18.5 FSM transition log (`[V510-FSM]` — Tier-2.1)

Pure observation hook. Refuses to emit on `from == to` no-ops (asserted by smoke test). Format:

```
[V510-FSM] ticker=AMD from=IDLE to=WATCHING reason=VOL_SPIKE_DETECTED bucket=1445
```

v5.1.2 ships the emitter only; the wider FSM-call-site sweep is intentionally minimal so we do not accidentally change v5.0.0 Tiger/Buffalo behavior. A future PR will fan the emitter out to every transition site.

18.6 Pre-trade indicator snapshots (`indicators.py` — Tier-2.2)

Pure functions, no `trade_genius` imports, no exceptions raised on bad input:

Function	Inputs	Output	None when
`rsi14(closes)`	closes (newest last)	float	< 15 closes
`ema9(closes)`	closes	float	< 9 closes
`ema21(closes)`	closes	float	< 21 closes
`atr14(bars)`	bars with high/low/close	float	< 15 bars
`vwap_dist_pct(bars)`	bars with H/L/C/volume	float	no bars or zero volume
`spread_bps(bid, ask)`	bid, ask	float	missing or non-positive

All values are wired into `[V510-CAND]` so every candidate moment carries the indicator state at decision time. Callers MUST emit `null` (not `0.0`) into log lines on `None` returns.

18.7 Out of scope (deferred)

News / halt flags (needs Polygon or Benzinga subscription); L2 / order-book snapshots; tick-level trades; `VOL_GATE_ENFORCE=1`; new env-driven configs beyond v5.1.1; adaptive runtime config switching.

18a. Always-On Algo Plus Ingest (v6.5.0)

This section documents the always-on ingest layer introduced in v6.5.0. Source module: `ingest/algo_plus.py`.

18a.1 Component overview

Five components collaborate to provide continuous bar archiving across the extended session (04:00–20:00 ET) via Alpaca Algo Plus SIP feed:

Component	Responsibility
`AlgoPlusIngest`	Top-level coordinator. Resolves credentials, subscribes the `StockDataStream`, owns the reconnect loop, and wires `on_bar_callback` to `BarAssembler`. Launched as a daemon thread at boot alongside `scheduler_thread`. Source: `ingest/algo_plus.py`.
`BarAssembler`	Validates the raw SIP bar payload, fills `trade_count` and `bar_vwap` from SIP-specific fields, and emits a canonical bar dict ready for `bar_archive.write_bar`.
`ConnectionHealth`	5-state machine (see §18a.2). Tracks WebSocket liveness and gates whether `GapDetector` runs on the normal 5-minute cadence or the accelerated 1-minute REST_ONLY cadence.
`GapDetector`	Reads `/data/bars/TODAY/*.jsonl` bar timestamps for each subscribed ticker, finds spans of ≥ 3 consecutive missing 1-minute bars (`GAP_THRESHOLD_MINUTES = 3`), and enqueues `(ticker, gap_start_utc, gap_end_utc)` tuples to `RestBackfillWorker`. Runs on boot, on every LIVE reconnect, and periodically.
`RestBackfillWorker`	Background thread. Dequeues gap tuples, issues `GET /v2/stocks/{sym}/bars?timeframe=1Min&feed=sip` REST requests at ≤ 200 req/min (Algo Plus budget), and writes bars that pass the `ts`-dedup check via `bar_archive.write_bar`. Sleep between requests: 0.35 s (≈ 171 req/min).

18a.2 ConnectionHealth state machine

```
boot ──►│  CONNECTING  │
        └──────┬───────┘
               │ handshake OK
               ▼
        ┌──────────────┐
┌──────►│     LIVE     │◄──────────────────────┐
│       └──────┬───────┘                       │
│              │ stream error / timeout        │ reconnect OK
│              ▼                               │
│       ┌──────────────┐   backoff(n)    ┌──────┴──────┐
│       │  DEGRADED    │────────────────►│ RECONNECTING│
│       └──────┬───────┘                 └─────────────┘
│              │ 3 consecutive failures
│              ▼
│       ┌──────────────┐
└───────│   REST_ONLY  │  (GapDetector continues; WS suspended)
        └──────────────┘
               │ next scheduled reconnect attempt (every 5 min)
               └──────► RECONNECTING
```

State transitions:

- **LIVE → DEGRADED:** Any stream error, missed heartbeat (> 120 s with no bar on a session day), or exception in `on_bar_callback`.
- **DEGRADED → RECONNECTING:** After exponential backoff (5 s, 10 s, 20 s, 40 s, 80 s, cap 300 s).
- **RECONNECTING → LIVE:** Successful re-subscribe and first bar received.

- **RECONNECTING → DEGRADED:** Reconnect attempt fails; failure counter increments.
- **DEGRADED → REST_ONLY:** 3 consecutive reconnect failures. `GapDetector` triggers an immediate REST backfill for the outage window.
- **REST_ONLY → RECONNECTING:** Every 5 minutes, one reconnect attempt.

On any transition to DEGRADED or REST_ONLY, `GapDetector` fires immediately (does not wait for the 5-minute periodic cadence).

18a.3 Bar flow sequence — normal boot and live path

```
Process boot
    │
    ├── _alpaca_data_client() resolves VAL_ALPACA_PAPER_KEY ✓
    │
    ├── AlgoPlusIngest.start()
    │       │
    │       ├── StockDataStream(key, secret, feed="sip").subscribe_bars(
    │       │       on_bar_callback, *TICKERS
    │       │   )
    │       │
    │       ├── GapDetector.run_on_boot()
    │       │       │
    │       │       └── for each ticker: find last archived bar ts
    │       │               if gap > 3 min → enqueue (ticker, gap_start, now)
    │       │
    │       └── ConnectionHealth → CONNECTING
    │
    ├── (WebSocket handshake completes)
    │       │
    │       └── ConnectionHealth → LIVE
    │
    ├── Per-minute: on_bar_callback(bar_data)
    │       │
    │       ├── BarAssembler.accept(bar_data)
    │       │       (validates schema; fills trade_count, bar_vwap from SIP fields)
    │       │
    │       └── bar_archive.write_bar(ticker, bar)
    │               → /data/bars/2026-04-28/NVDA.jsonl  (append)
    │
    ├── Every 5 min: GapDetector.run_periodic()
    │       └── same gap scan → RestBackfillWorker queue if gaps found
    │
    └── RestBackfillWorker (background thread, rate-limited)
            │
            ├── dequeue (ticker, start_ts, end_ts)
            ├── GET /v2/stocks/{ticker}/bars?start=...&end=...
            │       &timeframe=1Min&feed=sip&limit=1000
            ├── for each bar: dedup check → write_bar() if new
            └── sleep(0.35) between requests  (≤200 req/min budget)
```

Bar path in plain terms: WebSocket bar → `on_bar_callback` → `BarAssembler.accept` → `bar_archive.write_bar` → `/data/bars/YYYY-MM-DD/TICKER.jsonl`.

18a.4 Gap detection contract

- **Threshold:** `GAP_THRESHOLD_MINUTES = 3`. A span of 3 or more consecutive missing 1-minute bars triggers a REST backfill request.
- **Run triggers:**
- On boot — after WebSocket subscription succeeds. Looks back to 04:00 ET for the current session day (skips weekends and holidays).
- On reconnect — immediately after `ConnectionHealth` transitions to LIVE, covering the outage window precisely.
- Periodic — every 5 minutes via the `scheduler_thread` elapsed-time check (analogous to the `state_elapsed >= 5` pattern at `trade_genius.py:5292`). In `REST_ONLY` state the cadence tightens to every 1 minute to bound backfill lag.

18a.5 REST backfill rate-limit budget

Alpaca Algo Plus tier: 200 requests/min on the data REST API.

Parameter	Value	Rationale
Endpoint	`GET /v2/stocks/{sym}/bars`	Alpaca data API v2
Feed param	`feed=sip`	Algo Plus unlocks SIP; required for 04:00–20:00
Timeframe	`timeframe=1Min`	Matches `BAR_SCHEMA_FIELDS` granularity
Limit per request	1000 bars	Alpaca max per call
Sleep between requests	0.35 s	~171 req/min, within 200 req/min budget
Chunk size (time)	1000 min ≈ 16.7 h	One request covers a full extended session
Dedup check	Compare `ts` field vs existing JSONL `ts` values	Skip write if already present

For a 10-ticker watchlist a full-day backfill (04:00–20:00 ET, 960 min/ticker) requires at most 10 REST calls. At 0.35 s sleep that is 3.5 s total — well within any restart recovery budget.

18a.6 BAR_SCHEMA_FIELDS — `feed_source` addition (M-4)

v6.5.0 adds `feed_source` as a new field in `BAR_SCHEMA_FIELDS` (`bar_archive.py:31–45`). Value is `"sip"` for bars collected under Algo Plus or `"iex"` for bars collected on the legacy IEX feed. Legacy bars written before the migration carry `None` for this field.

Backward-compat guarantee: `backtest/loader.py` already reads all schema fields via `.get(k)`, so a missing `feed_source` key returns `None` without any code change. Do not rename or reorder existing `BAR_SCHEMA_FIELDS` entries — only additive extension is permitted at this boundary.

18a.7 `shadow_data_status` contract

`shadow_data_status` is a discrete observability field in `/api/state` (populated by `IngestHealthReporter` every 30 s). Contract:

```
shadow_data_status: "live" | "degraded" | "offline" | "unconfigured"
```

Status	Meaning	Operator action
`live`	WS connected; last bar received < 90 s ago (or market closed)	None
`degraded`	WS disconnected; REST backfill running; gap < 15 min	Monitor
`offline`	REST_ONLY state; gap ≥ 15 min or WS has never connected	Investigate
`unconfigured`	`VAL_ALPACA_PAPER_KEY` not set; no ingest possible	Fix env vars

`shadow_data_status="live"` is trustworthy only when **both** conditions hold: (a) `ConnectionHealth = LIVE` AND (b) last bar timestamp for at least one subscribed ticker is within 90 seconds of wall clock (or the market is outside 04:00–20:00 ET). The dashboard shadow panel renders this as a color pill: green = live, yellow = degraded, red = offline/unconfigured. Prior "silent zeros" failure mode (empty shadow P&L table with no visible warning) is eliminated.

18a.8 `/api/state` additions

Three new fields in the `/api/state` JSON response (additive; no breaking change to existing consumers):

Field	Type	Description
`shadow_disabled`	bool	`True` when `VAL_ALPACA_PAPER_KEY` is absent; emits `[INGEST SHADOW DISABLED]` at boot
`shadow_data_status`	string	See §18a.7 contract
`ingest_status`	dict	Snapshot from `IngestHealthReporter` (see below)

`ingest_status` shape:

```
{
  "status": "live",
  "last_bar_age_s": 47,
  "open_gaps_today": 0,
  "bars_today": 1203,
  "ws_state": "LIVE"
}
```

18a.9 Extended REST window and SIP feed promotion

REST window: `_fetch_1min_bars_alpaca()` window expanded from `08:00–18:00 ET` to `04:00–20:00 ET` (`trade_genius.py:3629–3630`). This covers the full extended-session range that Alpaca SIP supports on US

equity days. The change only affects what bars are available in `_cycle_bar_cache`; no logic dependency exists on the old window boundary.

SIP feed promotion: All REST calls that previously pinned `feed=DataFeed.IEX` or `feed="iex"` are promoted to `feed=DataFeed.SIP` system-wide (patch P-5, `trade_genius.py:3557, 3637, 2579`). IEX is retained as a fallback: if a SIP request returns an empty bar list, the fetcher retries with IEX and logs the downgrade. SIP is the consolidated tape; premarket and after-hours bars are only available via SIP on Algo Plus.

18a.10 Boot wiring

The `ingest_loop` daemon thread is launched in `trade_genius.py` during the process boot sequence, immediately adjacent to `scheduler_thread`:

```
# trade_genius.py — boot sequence (alongside scheduler_thread)
scheduler_thread = threading.Thread(target=run_scheduler, daemon=True)
scheduler_thread.start()

ingest_thread = threading.Thread(target=ingest_loop, daemon=True)
ingest_thread.start()
```

`ingest_loop` (defined in `ingest/algo_plus.py`) runs its own internal watchdog: on any unhandled exception it sets `ConnectionHealth` to `DEGRADED`, applies exponential backoff, and re-enters the WebSocket subscription loop. The ingest thread is fully isolated from the scan loop — it shares no mutable globals and all archive writes are append-only to JSONL files via the existing failure-tolerant `bar_archive.write_bar` surface.

---

19. Permission gates — v5.9.0 QQQ Regime Shield + ticker AVWAP + OR

19.1 Three-gate set (v5.9.0+: G1 swapped to EMA cross)

v5.6.0 shipped a 3-gate unified AVWAP set (G2 retired). v5.9.0 keeps the 3-gate shape but **swaps G1 from a `QQQ.last vs QQQ.opening_avwap` penny-switch to a structural `QQQ 5m EMA(3) vs EMA(9)` cross.** G3 (ticker AVWAP) and G4 (OR breakout) are unchanged.

Gate	Light name	Long (L-P1)	Short (S-P1)
G1	Regime	`QQQ.5m_3EMA > QQQ.5m_9EMA`	`QQQ.5m_3EMA < QQQ.5m_9EMA`
~~G2~~	retired	(deleted in v5.6.0)	(deleted in v5.6.0)
G3	Ticker AVWAP	`Ticker.Last > Ticker.Opening_AVWAP`	`Ticker.Last < Ticker.Opening_AVWAP`
G4	Structure	`Ticker.Last > Ticker.OR_High`	`Ticker.Last < Ticker.OR_Low`

Conventions (locked):

- **Index = QQQ only.** SPY no longer participates in the permission scan.

- **G1 EMA source.** `qqq_regime.QQQRegime` maintains EMA(3) and EMA(9) on closed 5m QQQ bars. EMAs are **seeded pre-market** (04:00–09:30 ET) with priority `bar archive → Alpaca historical IEX → prior session` so the compass is hot at 09:35 ET instead of warming up live.
- **Compass.** `current_compass()` returns `UP` (EMA3 > EMA9), `DOWN` (EMA3 < EMA9), `FLAT` (equal), or `None` (warmup). G1 fails closed for `FLAT` and `None`.
- **AVWAP (ticker)** = session-open anchored VWAP for G3 only. Anchor 09:30 ET, reset daily, recomputed on every 1m bar close. Implementation: `trade_genius._opening_avwap(ticker)`.
- **OR window** = 5-minute opening range, 09:30–09:35 ET (existing).
- **Comparators**: strict `>` and `<`. Equality returns **FAIL** on every gate.
- **Pre-9:35 ET** (OR not yet defined): G4 returns `False` deterministically.
- **None** inputs: G1/G3/G4 return `False` deterministically.
- **Mid-strike compass flip.** Active trades let their existing exit FSM run. Re-entry is blocked when G1 evaluates to FAIL because the strike-1 compass differs from the live compass; `[V572-ABORT]` records the flip.

Per-direction predicates (strict, fail-closed) live in `tiger_buffalo_v5.py`: `gate_g1_long`, `gate_g1_short`, `gate_g3_long`, `gate_g3_short`, `gate_g4_long`, `gate_g4_short`. The aggregate helpers `gates_pass_long(qqq_last, qqq_opening_avwap, ticker_last, ticker_opening_avwap, ticker_or_high)` and the symmetric `gates_pass_short(..., ticker_or_low)` AND the three predicates together; both have 5-arg signatures (smoke-guarded).

Forensic logging. Every G1/G3/G4 evaluation emits a `[V560-GATE] ticker=X side=LONG|SHORT gate=G1|G3|G4 value=… threshold=… result=True|False` line. Blocked entries also log `[V560-GATE][BLOCK] ticker=X side=… failed=G1,G3,…` and passing entries log `[V560-GATE][PASS] ticker=X side=… qqq_last=… qqq_avwap=… ticker_last=… ticker_avwap=… or_threshold=…`. Saturday's report parses these lines to validate the cut. Existing `[V510-CAND]` and `[V510-SHADOW]` lines are unchanged (the volume_profile shadow grid is orthogonal to the permission scan and continues to evaluate the same TICKER+QQQ_70_100 / TICKER_ONLY_70 / QQQ_ONLY_100 / GEMINI_A_110_85 configs on top of the new permission gates).

Startup confirmation. Every boot logs `[V560] Unified AVWAP gates: L-P1 (G1/G3/G4), S-P1 (G1/G3/G4)` so the operator can confirm at a glance that the new gate set is wired.

19.1.1 Data-collection log schema (v5.6.1)

v5.6.1 ships a **pure observability** patch on top of v5.6.0; the gate predicates in `tiger_buffalo_v5.py` are unchanged. The forensic surface now carries a richer set of structured lines:

- **`[V560-GATE]` (richened, single line)** — every gate evaluation now emits one line carrying all 14 fields: `ticker, side, ts, ticker_price, ticker_avwap, index_price, index_avwap, or_high, or_low, g1, g3, g4, pass, reason`. The legacy per-G1/G3/G4 lines are retained for backwards compat with older parsers.
- **`[ENTRY]` carries `entry_id`** — every entry log line now includes `entry_id=<TICKER>-<YYYYMMDDHHMMSS>` (deterministic from the entry UTC timestamp), so each entry is paired with its eventual exit.
- **`[TRADE_CLOSED]` (new)** — every exit emits a paired `[TRADE_CLOSED] entry_id=… side=… exit_reason=… hold_s=… pnl_usd=…` line. `exit_reason` is normalised to one of `stop|target|eod|time|manual`.

- **`[SKIP]` with `gate_state` (new)** — skip lines now embed the full L-P1/S-P1 gate snapshot as canonical JSON. Pre-gate skips (cooldown, loss-cap, DI warmup, etc.) emit `gate_state=null`.
- **`[UNIVERSE]` (new)** — boot logs `[UNIVERSE] tickers=…` once with the alpha-sorted, dedupe'd, uppercased universe (QQQ included alongside the 8 trade tickers).
- **`[WATCHLIST_ADD]` / `[WATCHLIST_REMOVE]` (new)** — runtime watchlist mutations emit one structured line each.

Bar archive surface. `_v561_archive_qqq_bar` writes the per-cycle 1m QQQ snapshot to `/data/bars/<UTC-date>/QQQ.jsonl` as the 9th file alongside the 8 trade tickers, using the same canonical `bar_archive` schema. The pre-open archive path (09:29:30–09:35 ET) backfills the OR window so the 5 OR-window 1m bars land on disk before the gates turn live at 09:35.

OR persistence. At/after 09:35 ET, `_v561_persist_or_snapshot` writes `{ticker, or_high, or_low, computed_at_utc}` JSON to `/data/or/<UTC-date>/<TICKER>.json`. Idempotent — at most one snapshot per ticker per day; the in-memory `_v561_or_snap_taken` set is reset by `reset_daily_state` on the date rollover.

19.2 Volume-profile G4 shadow grid (legacy v5.1.0+ surface, unchanged)

The volume-profile `evaluate_g4(_config)` shadow grid is independent of the permission scan. It continues to log the four hard-coded shadow configs (`TICKER+QQQ_70_100` , `TICKER_ONLY_70` , `QQQ_ONLY_100` , `GEMINI_A_110_85`) plus `BUCKET_FILL_100` to `[V510-SHADOW][CFG=…]` on every entry consideration. No volume gate is enforced (`VOL_GATE_ENFORCE` remains the default 0); v5.6.0 does not change the volume-profile path.

---

20. Offline backtest CLI (v5.4.0)

The `backtest/` package is a self-contained offline replayer. It reads the JSONL bar archive written by `bar_archive.write_bar` and replays one (or all) of the `volume_profile.SHADOW_CONFIGS` gate rules over a trading-day range, producing a CSV ledger of entry/exit pairs and an optional replay-vs-prod validation report.

20.1 How to run

```
python -m backtest.replay \
    --start 2026-04-20 --end 2026-04-24 \
    --config GEMINI_A \
    [--validate] \
    [--out ./backtest_out/] \
    [--bars-dir /data/bars/] \
    [--state-db /data/state.db]
```

`--config` accepts any `SHADOW_CONFIGS` name (`TICKER+QQQ` , `TICKER_ONLY` , `QQQ_ONLY` , `GEMINI_A` , `BUCKET_FILL_100`) or the literal `ALL` to fan out across every config in one run.

20.2 What `--validate` does

In validate mode, after the replay completes, the CLI queries `shadow_positions` in `state.db` for the same `config_name` over the same date range and pairs each predicted replay entry to a prod row by ticker + side + entry timestamp within ±60s. The output report `<out>/<config>_<start>_<end>_validation.md` lists:

- **match rate** = matches / total prod entries
- **REPLAY_ONLY** entries (potential false-positive — replay would have entered, prod did not)
- **PROD_ONLY** entries (potential false-negative — prod entered, replay did not)
- average entry-price and exit-price drift across matched pairs

Exit code is 0 when match rate ≥ 0.95, else 1, so CI can gate on it.

20.3 CSV ledger columns

The ledger CSV at `<out>/<config>_<start>_<end>.csv` carries one row per closed entry/exit pair plus a leading `# summary:` comment line.

column	meaning
`ticker`	uppercase symbol
`side`	`BUY` (long) or `SHORT`
`entry_ts`	UTC ISO timestamp the position opened on a bar close
`entry_price`	bar-close price at entry
`exit_ts`	UTC ISO timestamp the position closed
`exit_price`	bar-close price at exit
`qty`	shares (sized off a $1000-per-position budget)
`pnl_dollars`	realized P&L in $; direction-aware
`pnl_pct`	realized P&L as % of entry price
`exit_reason`	`trail_stop` · `hard_eject` · `eod`

The `# summary:` line gives quick totals: trade count, wins, losses, total P&L, win rate.

20.4 Reused logic

Gate-pass evaluation (`_gate_pass`) and the entry → exit pairing follow the same model proven out in `backtest_v510/replay_gate.py` and `backtest_v510/replay_gene_configs.py` — the `pnl_per_pair` math is ported directly. The MVP intentionally uses bar-close prices and a simple trail/eject/eod policy rather than re-implementing every nuance of `trade_genius.py`'s live decision tree; the engine's job is *deterministic, diff-able replay*, and `--validate` is the canary that flags drift between replay and prod.

---

21. Shadow tab charts (v5.4.1)

v5.4.1 layers three Chart.js visualizations on the Shadow tab without introducing any new SQLite tables — every chart is derived from the existing v5.2.0 `shadow_positions` rows (closed trades only, `exit_ts_utc IS NOT NULL`) by a new HTTP endpoint.

21.1 Endpoint — `GET /api/shadow_charts`

Defined in `dashboard_server.py`. Same session-cookie auth as the rest of `/api/*`. Server-side cache: a single lock-protected `(ts, payload)` tuple with a 30 s TTL — the same pattern used by `/api/indices` — so multiple browsers polling the Shadow tab in parallel collapse to one SQLite read per window. The response always emits all 7 configs in a fixed order; configs with no closed trades render as empty arrays rather than missing keys.

Response shape:

```
{
  "configs": {
    "GEMINI_A": {
      "equity_curve":     [{"ts": "...", "cum_pnl": 0.0}, ...],
      "daily_pnl":        [{"day": "YYYY-MM-DD", "pnl": 0.0, "trades": 0}, ...],
      "win_rate_rolling": [{"ts": "...", "win_rate": 0.0}, ...]
    },
    "...": { ... }
  },
  "as_of": "<UTC ISO ts>"
}
```

- `equity_curve` — cumulative realized P&L sampled per closed trade.
- `daily_pnl` — per-trading-day total P&L plus closed-trade count.
- `win_rate_rolling` — rolling 20-trade window win rate; the array is empty (and the chart hidden client-side) for any config with fewer than 20 closed trades.

21.2 Frontend — three chart groups on the Shadow tab

Chart.js 4.4.0 is loaded from jsDelivr CDN with a `defer` attribute. The chart code falls back gracefully if `window.Chart` is undefined — empty wrappers render and the rest of the dashboard keeps working.

Three vertically-stacked groups sit above the existing per-config rows:

1. **Equity curves** — one Chart.js line chart per config (~100 px desktop, ~80 px mobile). Y-axis cumulative $, X-axis time.
2. **Day-P&L heatmap** — single ~300 px scatter chart, rows = configs, columns = trading days, cell color = green/red intensity scaled to abs-max P&L across all cells.
3. **Rolling win-rate sparklines** — one per config (~60 px), Y-axis 0–1, hidden when the config has < 20 closed trades.

Each config gets a stable hue across all three groups (`SHADOW_CFG_COLORS`) so `GEMINI_A` 's equity curve, heatmap row, and win-rate sparkline are always the same color. Axis colors and gridlines read from existing CSS variables (`--text-dim` , `--border`); no new color literals.

The "Charts" header is collapsible (click / Enter / Space toggles). Default is **expanded on desktop and collapsed on ≤ 720 px viewports** so the Shadow tab is not dominated by chart real estate on a phone.

21.3 Polling

Tab-aware: `/api/shadow_charts` is fetched once on Shadow-tab activation and then every 60 s **only while the Shadow tab is active** — Main / Val / Gene ticks skip the call entirely. Matches the existing `pollExecutor` pattern.

21.4 Interactivity (v5.5.1)

v5.5.1 adds two interactivity layers on top of the v5.4.1 chart constructors without touching the endpoint or polling cadence. (1) Rich Chart.js tooltips on hover/tap for all three groups, wired via the built-in `plugins.tooltip.callbacks` option so mobile-tap tooltips work without a custom overlay: equity curves show `MM/DD HH:MM ET · ±$cum_pnl · config_name`, the day-P&L heatmap adds the per-day trade count (`config_name · YYYY-MM-DD · ±$pnl · N trades`), and rolling win-rate sparklines show `config_name · trade #N · win_rate%`. (2) Click-to-isolate config: a single `__scIsolated` state variable in the Shadow-tab module holds the active config name; clicking any equity row, sparkline, or heatmap cell sets it (or clears it on a second click of the same config), and the three chart groups re-render with non-isolated configs faded to ~20% opacity. A small "Showing only: *config* · click to clear" hint with an X button appears above the charts whenever isolation is active, and clicking empty space on the heatmap also clears.

---

22. v5.7.0 — Unlimited Titan Strikes (Apr 2026)

Universe — Ten Titans. v5.7.0 extends the default trade universe to ten tickers (`AAPL, AMZN, AVGO, GOOG, META, MSFT, NFLX, NVDA, ORCL, TSLA`). NFLX and ORCL are new in this release; the existing eight roll forward unchanged. QQQ remains the index ticker for the v5.6.0 G1 / G3 gates and is also archived under `/data/bars/<UTC>/QQQ.jsonl` from v5.6.1. The `TITAN_TICKERS` module-level constant in `trade_genius.py` is the source of truth for which tickers see the v5.7.0 unlimited-strike path; non-Titan tickers (anything added later via `[WATCHLIST_ADD]`) continue to use the v5.0.0 R3 re-hunt budget on `tiger_buffalo_v5`.

Strike 1 path — unchanged. First entry on a given (ticker, side, day) flows through the v5.6.0 unified AVWAP permission set. Both the `[V560-GATE]` rich line and the new `[V570-STRIKE]` line are emitted; gate logic is byte-identical to v5.6.1 (`tiger_buffalo_v5.py` is untouched).

Strike 2+ path — Expansion Gate. When a Titan's per-ticker per-side per-day strike counter has already incremented at least once, the next entry attempt no longer evaluates `[V560-GATE]`. Instead the **Expansion Gate** runs:

- **LONG Strike 2+:** passes iff `current_price > prior_session_HOD` (strict `>`, fresh print) AND `index_price > index_avwap` (strict `>`, same comparator as v5.6.0 G1). AVWAP `None` FAILs.
- **SHORT Strike 2+:** mirror with strict `<`.

Session HOD/LOD is **per-ticker, per-day**, seeded only from the first 9:30 ET print onward (pre-market does NOT seed). The values reset at 9:30 ET each session; the helper `_v570_update_session_hod_lod` returns the prior-tick HOD/LOD before folding in the current print, so the "fresh print" comparator is well-defined even in the first second after open.

Strike counter. `_v570_strike_counts[(ticker, side)]` is incremented on every successful ENTRY (never on SKIP). The counter is per-ticker per-side, so a LONG Strike 1 and a SHORT Strike 1 on the same ticker on the same day both fire normally. The counter resets at 9:30 ET next session along with HOD/LOD and the kill-switch latch.

Sovereign brake — `-$500` daily loss kill switch. v4.7.0 shipped `_check_daily_loss_limit` at -$500 (sourced from `DAILY_LOSS_LIMIT = float(os.getenv("DAILY_LOSS_LIMIT", "-500"))`). v5.7.0 does NOT retune the threshold; instead it layers a v5.7.0-native latch directly on top of `[TRADE_CLOSED]` emissions so realized P&L is summed lock-step with the lifecycle log, independent of the legacy halt flag. On first breach (`<= -500.00`):

- A single `[KILL_SWITCH] reason=daily_loss_limit triggered_at=<utc> realized_pnl=<f>` line is emitted (de-duped — never spammed by subsequent SKIPs).
- Every entry path (Strike 1 OR Strike 2+, all 10 Titans, both sides) returns `[SKIP] reason=daily_loss_limit_hit gate_state=null`.
- Open positions are NOT force-closed. They exit on their own normal exits (stop, target, time, eod, manual) and continue to emit `[TRADE_CLOSED]` lines — those still update `daily_realized_pnl` for forensic completeness.
- The latch resets at the next ET session boundary (along with the strike counter and HOD/LOD).

Feature flag. `ENABLE_UNLIMITED_TITAN_STRIKES = True` is the default. Setting `False` reverts every Titan to the v5.6.0 + v5.0.0 R3 path (`daily_count >= 5` cap and the `transition_re_hunt` re-arm budget on `tiger_buffalo_v5`), so emergency rollback is a single-line edit and a redeploy.

New + extended log lines:

- `[V570-STRIKE] ticker=<T> side=<L|S> ts=<utc> strike_num=<int> is_first=<bool> hod=<f|null> lod=<f|null> hod_break=<bool> lod_break=<bool> expansion_gate_pass=<bool>` — emitted on every entry-path evaluation. Replaces `[V560-GATE]` on Strike 2+; alongside `[V560-GATE]` on Strike 1.
- `[ENTRY] … strike_num=<int>` — `entry_id` schema unchanged.
- `[TRADE_CLOSED] … strike_num=<int> daily_realized_pnl=<f>` — strike echoes the entry's value; `daily_realized_pnl` is the running cumulative for the day after this close.
- `[KILL_SWITCH] reason=daily_loss_limit triggered_at=<utc> realized_pnl=<f>` — exactly once per session.

The forensic logger module (`logger`) and the v5.6.1 SKIP/GATE schema are otherwise unchanged. Every v5.7.0 helper lives in `trade_genius.py` ; `tiger_buffalo_v5.py` is byte-identical to v5.6.1.

---

23. v5.7.1 — Bison & Buffalo exit FSM (Apr 2026)

Scope. v5.7.1 rewrites the exit-logic state machine for the Ten Titans only. Non-Titan tickers (any future `[WATCHLIST_ADD]` ticker, or any ticker with `ENABLE_BISON_BUFFALO_EXITS = False`) keep the legacy `evaluate_exit` path (DI<25 hard eject + structural stop) byte-for-byte.

Three-phase FSM. Each Titan position carries an explicit `phase` field:

1. **`initial_risk`** — **v5.9.0 Recursive Forensic Stop (Maffei 1-2-3).** Replaces the v5.7.1 `hard_stop_2c` two-consec counter. Every 1-minute close that gates outside the OR boundary (LONG: close < `OR_High`; SHORT: close > `OR_Low`) runs an audit against the **prior** 1-minute candle: LONG exits iff `current_low < prior_low`; SHORT exits iff `current_high > prior_high`. Equality and a higher-low / lower-high STAY (consolidation / wick). Closes back inside OR reset the streak. On fire, `[TRADE_CLOSED] … exit_reason=forensic_stop`. **Per-trade Sovereign Brake** also runs every tick: if unrealized P&L on the single open position reaches **-$500.00** (`(current-entry)*qty` LONG; `(entry-current)*qty` SHORT), exit immediately as `exit_reason=per_trade_brake`. This is distinct from the existing portfolio-level realized -$500 brake.
2. **`house_money`** — After the close of the **2nd green 5-min candle** post-entry (LONG; `close > open`) — or 2nd red 5-min for SHORT — the stop ratchets to entry price (BE). The hard-stop counter is now inactive; the active exit is `current_price < entry_price` for LONG, mirrored for SHORT. On fire: `exit_reason=be_stop`.
3. **`sovereign_trail`** — Once the 5-minute 9-period EMA is seeded (close of the 9th 5-min bar since 9:30 ET = **10:15 ET**), a 5-min CLOSE strictly below the EMA (LONG) — or strictly above (SHORT) — fires `exit_reason=ema_trail`. Before 10:15 ET the EMA is `None` and only the hard-stop / BE exits apply.

Velocity Fuse — global override. A circuit breaker that runs every tick the bot processes, regardless of phase. Comparison base is the **OPEN of the current (in-flight) 1-min candle**, not the prior candle's close.

- **LONG fires** when `current_price < candle_1m_open * 0.99` (strict; exactly -1.00% does not trigger; -1.001% does).
- **SHORT fires** when `current_price > candle_1m_open * 1.01` (strict).
- On fire: `[V571-VELOCITY_FUSE]` emits, the position is market-exited immediately, and `[TRADE_CLOSED] … exit_reason=velocity_fuse` follows. Strike counter still increments correctly so the v5.7.0 expansion gate re-arms on the next entry.

DI deletion (Titan-only). The legacy `DI+(1m) < 25` (LONG exit) and `DI-(1m) < 25` (SHORT exit) triggers are bypassed for Titans. `tiger_buffalo_v5.py:evaluate_exit(..., is_titan=True)` is the single guard. Non-Titan tickers retain both DI exits — the v5.0.0 priority order is preserved verbatim for the legacy path.

State carried per open Titan position.

Field	Type	Purpose
`phase`	str	`initial_risk` / `house_money` / `sovereign_trail`
`forensic_consecutive_count`	int	0..N; advanced on each Phase A close-outside-OR; reset on close back inside
`entry_price` / `qty`	float / int	Cached for per-trade Sovereign Brake unrealized-P&L math
`prior_1m_low` / `prior_1m_high`	float| None	Carryover for the next-bar Forensic Stop audit
`green_5m_count` (LONG)	int	Closed green 5-min bars post-entry
`red_5m_count` (SHORT)	int	Closed red 5-min bars post-entry
`ema_5m`	float| None	Rolling 9-EMA on 5-min closes; `None` until 10:15 ET
`current_stop`	float	Active stop level (changes as phase transitions)

New + extended log lines.

- `[V571-EXIT_PHASE] ticker=<T> side=<L|S> entry_id=<id> from_phase=<…> to_phase=<…> trigger=<be_2nd_green|be_2nd_red|ema_seeded|…> current_stop=<f> ts=<utc>` — emitted **on phase transition only** (entry → `initial_risk`; BE move → `house_money`; EMA seed → `sovereign_trail`).
- `[V571-VELOCITY_FUSE] ticker=<T> side=<L|S> candle_open=<f> current_price=<f> pct_move=<f> ts=<utc>` — emitted on every fuse fire, immediately before the market exit.
- `[V571-EMA_SEED] ticker=<T> ema_value=<f> ts=<utc>` — emitted exactly once per ticker per session at 10:15 ET when the EMA is first valid.
- `[TRADE_CLOSED] … exit_reason=…` — the v5.6.1 enum (`stop|target|time|eod|manual`) gains five Titan-only values: `forensic_stop` (v5.9.0; replaces `hard_stop_2c`), `per_trade_brake` (v5.9.0), `be_stop`, `ema_trail`, `velocity_fuse`. The legacy values remain valid for non-Titan paths.
- `[V590-FORENSIC] ticker=<T> side=<L|S> close=<f> low=<f> prior_low=<f> consec=<n> ts=<utc>` (LONG; SHORT mirrors with `high` / `prior_high`) — emitted on every Phase A 1m close that gates outside OR.
- `[V590-PER-TRADE-BRAKE] ticker=<T> side=<L|S> entry=<f> current=<f> qty=<n> unrealized=<f> ts=<utc>` — emitted on per-trade brake fire.
- `[V572-REGIME] qqq_5m_close=<f> ema3=<f> ema9=<f> compass=<UP|DOWN|FLAT|None>` — emitted on every closed 5m QQQ bar.
- `[V572-REGIME-SEED] source=<archive|alpaca|prior_session> bars=<n> ema3=<f> ema9=<f> compass=<…>` — emitted once at first compass evaluation.
- `[V572-ABORT] ticker=<T> side=<L|S> prev_compass=<…> new_compass=<…> seconds_into_strike=<n>` — emitted when re-entry is blocked due to mid-strike compass flip.

Feature flag. `ENABLE_BISON_BUFFALO_EXITS = True` is the default. `VELOCITY_FUSE_PCT = 0.01` is the strict 1.0% threshold. Setting the flag `False` reverts every Titan to the v5.0.0 `evaluate_exit` path; the velocity fuse is also gated off.

Module placement. v5.7.0 carved `tiger_buffalo_v5.py` out completely (every helper landed in `trade_genius.py`). v5.7.1 carves it back IN: pure Bison/Buffalo exit-FSM helpers (`init_titan_exit_state`, `update_hard_stop_counter_long/short`, `update_green_5m_count_long`, `update_red_5m_count_short`, `update_ema_5m`, `velocity_fuse_long/short`, `evaluate_titan_exit`, plus `transition_to_house_money` / `transition_to_sovereign_trail`) live in `tiger_buffalo_v5.py` and are exercised by ~15 new smoke tests. `evaluate_exit` gains an `is_titan` kwarg for the DI deletion. The `trade_genius.py` runtime owns the log emitters, config flags, and the wiring between live ticks and these pure helpers.

---

24. v5.8.0 — Developer Velocity Bundle (Apr 2026)

Scope. Pure repo/tooling release — no algorithm logic touched, no live trading paths modified. Cuts subagent cold-start time, prevents CI-fail iteration cycles, and eliminates the universe-drift recovery class of incidents that hit v5.7.0.

Deliverables.

- **`CLAUDE.md`** at repo root — concise agent guide that subagents read on first cold-start. Lists where things live (`entry_gate_v5.py`, `tiger_buffalo_v5.py`, `bison_v5.py`, `shadow_configs.py`, `bot_version.py`, `bar_archive.py`), mandatory PR rules (BOT_VERSION + CHANGELOG lockstep, em-dash escape, forbidden-word list, Telegram 34-char rule), pre-push checklist, and PR submission flow. **`AGENTS.md`** is a thin parallel file that `@import`s `CLAUDE.md` so Codex picks up the same guide.
- **`specs/_TEMPLATE.md`** — spec scaffolding so every future release starts from a consistent shape (Decisions / Goals / Scope / Logging schema / Tests / Rollout).
- **`scripts/preflight.sh`** — local CI mirror. BLOCKS on five checks: pytest, BOT_VERSION ↔ CHANGELOG consistency, em-dash literal in `.py`, forbidden-word (`scrape|crawl|scraping|crawling`), ruff format. Em-dash and forbidden-word checks are scoped to files **changed in this PR vs `origin/main`** so the pre-v5.8.0 codebase (which has hundreds of grandfathered literal em-dashes) does not block local runs. Subagents run `bash scripts/preflight.sh` before `git push`.
- **`bot_version.py`** — canonical version constant (mirrored to `trade_genius.py.BOT_VERSION` so the existing `version-bump-check` CI workflow keeps working unchanged).
- **`[UNIVERSE_GUARD]` startup check.** New `_ensure_universe_consistency()` helper runs at boot in `trade_genius.py`, before `_init_tickers()`. Reads `/data/tickers.json` on the persistent volume, compares the on-disk list against the canonical `TICKERS_DEFAULT`, and rewrites the file (preserving the `{"tickers": [...]}` envelope) if the file is missing, corrupt, or has drifted. Tolerant of both flat-list and envelope JSON formats. Emits one of three observability lines on every boot:
- `[UNIVERSE_GUARD] universe consistent (N tickers)` — happy path
- `[UNIVERSE_GUARD] DRIFT detected: disk=… code=… — rewriting to code` — drift caught
- `[UNIVERSE_GUARD] /data/tickers.json corrupt (…), rewriting` — corrupt JSON

Smoke (post-deploy). Boot logs must contain exactly one `[UNIVERSE_GUARD]` line. If none appears, the guard didn't run.

Tests. `tests/test_universe_guard.py` covers four cases: missing file, corrupt JSON, drift detected, and consistent (no rewrite needed) — using pytest's `tmp_path` fixture and `monkeypatch` to redirect the persistent path.

---

25. v5.8.1 — Ops scripts (Infra-B post-deploy smoke + checks library)

Scope. Pure infra/tooling release — no algorithm logic touched, no live trading paths modified. Single shell library that both the post-deploy verification and the recurring weekday/Saturday crons call. Single source of truth for "what does a healthy TradeGenius deploy look like?"

Files.

- **`scripts/lib/checks.sh`** — sourceable bash library. Seven pure check functions, each prints a structured one-line stdout record and returns 0/1:
- `check_deploy_status` — Railway GraphQL `deployments(first:1)` → `DEPLOY <status> <8-char-id> v<version>`. Returns 0 on `SUCCESS`.
- `check_universe_loaded` — last-200 log scan for `[UNIVERSE_GUARD]` (the post-v5.8.0 schema; was `[UNIVERSE]` pre-v5.8.0). Parses both the `consistent (N tickers)` happy form and the `DRIFT detected: …` form. Returns 0 if `count >= 11`.
- `check_log_tags <tag…>` — variadic; counts each tag in last-500 log lines. Returns 0 if every tag occurs at least once.
- `check_no_errors` — counts `must be a coroutine`, `websocket error`, `Traceback`, `[ERROR]` in last-500 log lines. Returns 0 if all are zero.
- `check_bar_archive_today` — `railway ssh` runs `ls /data/bars/$(date -u +%Y-%m-%d)` + `du -sb`. Passes whenever the dir exists; soft-warns (stderr) when the dir is empty so market-closed days don't false-fail. Hard-fails only on a missing directory.
- `check_shadow_db_count` — `railway ssh` runs an inline `python3` block that opens `/data/shadow.db` with `sqlite3` and emits total + last-24h breakdown by `config_name`. Always returns 0 (informational).
- `check_dashboard_state` — POST `/login` with cookie jar, GET `/api/state`, parse `shadow_data_status` + `version`. Returns 0 if `status==live`.
- **`scripts/post_deploy_smoke.sh`** — orchestrator. `bash scripts/post_deploy_smoke.sh <expected_version>` sources the library, runs all 7 checks, tallies PASS/FAIL, prints a summary, and exits 0 on all-pass / 1 on any-fail. Default `EXPECTED_TAGS` set: `STARTUP SUMMARY`, `[UNIVERSE_GUARD]`, `[V560-GATE]`, `[V570-STRIKE]`, `[V571-EXIT_PHASE]`. Failures are informational — this script does NOT block automated merges; the CI smoke gate (`post-deploy-smoke.yml`) remains the blocker.
- **`tests/test_checks_lib.sh`** — 13 cases × 37 assertions. Each check has happy + sad fixture coverage. The library exposes `RAILWAY_LOGS_FIXTURE`, `RAILWAY_DEPLOY_FIXTURE`, `RAILWAY_SSH_FIXTURE`, and `DASHBOARD_STATE_FIXTURE` env-var overrides so tests run fully offline. Plain bash + manual asserts; no bats dependency.
- **`.github/workflows/scripts-lint.yml`** — soft-fail shellcheck pass over `scripts/*.sh`, `scripts/lib/*.sh`, and `tests/test_checks_lib.sh` on PRs that touch `scripts/`. `continue-on-error: true` for now since pre-existing scripts may not pass.

Caller pattern. Both `scripts/post_deploy_smoke.sh` and the recurring crons follow the same shape:

```
source "$(dirname "$0")/lib/checks.sh"
export RAILWAY_API_TOKEN=… RAILWAY_PROJECT=… RAILWAY_SERVICE=… RAILWAY_ENVIRONMENT=…
export DASHBOARD_URL=… DASHBOARD_PASSWORD=…
check_deploy_status
check_universe_loaded
check_log_tags "STARTUP SUMMARY" "[UNIVERSE_GUARD]" "[V560-GATE]" "[V570-STRIKE]" "[V571-EXIT_PHASE]"
…
```

Out of scope here. The cron task bodies (`58c883b0` weekday 8:35 CT, `873854a1` Saturday 10am CT) live outside the repo and are updated by the parent agent's `schedule_cron(action="update", …)` after this PR merges. The Saturday weekly-report directory is renamed to `backtest_v57x/` in that update. Per-exit-reason P&L breakdown belongs to the Saturday cron parser; `check_log_tags` already accepts `[V571-EXIT_PHASE]` as a recognised tag so the cron can pull the data through the shared library.

Saturday weekly report (v5.8.4)

`scripts/saturday_weekly_report.py` is the in-repo parser the Saturday cron `873854a1` invokes. It replaces the legacy parser that still reads `[V510-SHADOW]` lines from v5.5.x. The script pulls each Mon-Fri's `deploymentLogs` from Railway GraphQL (env: `RAILWAY_API_TOKEN` / `RAILWAY_PROJECT` / `RAILWAY_SERVICE` / `RAILWAY_ENVIRONMENT`), persists raw lines as `<out-dir>/week_<MONDAY>/day_YYYY-MM-DD.jsonl` , and parses the v5.6+ schema (`[V560-GATE]` , `[V570-STRIKE]` , `[V571-EXIT_PHASE]` , `[ENTRY]` , `[TRADE_CLOSED]` , `[SKIP]` , `[V510-SHADOW][CFG=…]`). Output is `report.md` plus `report.json` (the latter is read by next week's run for cumulative comparison). The 4 SHADOW_CONFIGS attribution pairs each `[ENTRY]` 's `entry_id` to the most recent `[V510-SHADOW][CFG=<name>]` verdict for that ticker. Offline mode (`--logs-dir <path>`) reads existing `day_*.jsonl` files instead of pulling from Railway and is the recommended dry-run path before each Saturday's cron firing.

---

v6.1.0 — P&L recovery bundle (May 2026)

Three algo upgrades shipped together. Each lives behind its own feature flag so any one of them can be reverted without redeploying.

Why

The v6.0.8.1 TRUE backtest against 2026-05-01 bars (full RTH, 11 universe tickers, gap-patched via Alpaca-IEX backfill) closed at −$8.32 on 7 entries (3W/3L, 1 still open). Hindsight max available swing across the universe was ~$143/share. Three structural failures:

1. Entry coverage — 4 tickers (META $20.65/sh, AVGO $14.28, AMZN $15.47, ORCL $10.22) had double-digit per-share swings and the algo never fired a single entry.
2. Premature exits — TSLA long 11:50 exited at +$1.00/sh, missed continuation to +$3.12/sh ($74.88 on 24 sh). NVDA short held 1 minute. Three trades killed by `sentinel_b_ema_cross` on minor pullbacks.
3. Win/loss asymmetry — avg win +$35, avg loss −$38; 50% W/L = guaranteed bleed because the trailing stop is fixed-cents and indifferent to volatility.

#1 ATR-scaled trailing stop with profit-protect ratchet

`engine/sentinel.py:check_alarm_a_stop_price` + `indicators.py:atr5_1m`.

Replaces the fixed-cents trail with a 3-stage ATR-distance trail keyed off the position's open P&L per share:

Stage	Condition	Trail distance
1 (initial)	`pnl_per_share < 1× ATR(5,1m)`	`1.0 × ATR`
2 (widen)	`1× ≤ pnl < 3× ATR`	`1.5 × ATR`
3 (lock-in)	`pnl ≥ 3× ATR`	`0.5 × peak_open_profit_per_share`
Floor (all)	always	`≥ 0.3 × ATR`

New kwargs on `check_alarm_a_stop_price`: `atr_value`, `position_pnl_per_share`, `peak_open_profit_per_share`, `entry_price`. All default `None`; when `None`, the legacy fixed-cents path runs unchanged. Feature flag: `_V610_ATR_TRAIL_ENABLED = True`.

#2 Two-bar EMA-cross confirmation + lunch-chop suppression

`engine/sentinel.py:check_alarm_b`.

Per-position counter `_ema_cross_pending: dict[str, int]` keyed by `position_id`. On each call: if the cross-against-position condition is True, increment; if False, reset to 0. Exit fires only at count ≥ 2.

Lunch-chop suppression: between 11:30 and 13:00 ET (uses `engine.timing.ET`), the exit is blocked even if the counter has reached 2. The counter still advances during suppression so state stays coherent when the window closes.

Feature flags: `_V610_EMA_CONFIRM_ENABLED = True` (master), `_V610_LUNCH_SUPPRESSION_ENABLED = True` (sub-flag). New `reset_ema_cross_pending()` helper for position-close and test cleanup, exported in `__all__`. When master flag is False, behavior reverts to the legacy single-bar `confirm_bars` path. Callers without `position_id` also take the legacy path (backward compat).

#3 ATR-normalized OR-break entry gate + late-OR window

`trade_genius.py`, `broker/orders.py`, `indicators.py:pre_market_range_atr`.

OR-break threshold replaced from fixed-cents to `k × ATR_pre_market` where `k = V610_OR_BREAK_K = 0.25` (defensive starting point; awaiting calibration). `pre_market_range_atr(bars, window_minutes=15, period=5)` filters 08:30–09:25 ET 1-min bars and computes Wilder ATR(5). Falls back to ATR(5) of the first 5 RTH bars if pre-market is sparse. Symmetric for short side.

Late-OR window: if the standard 9:30–10:30 OR-break never triggered for that ticker, the late-window opening range (first 30 min of 11:00–12:00 ET) becomes eligible during 11:00–12:00 ET. New module-level state `_v610_pm_atr`, `_v610_or_break_fired`, `_v610_late_or_high`, `_v610_late_or_low` — all cleared by `reset_daily_state`. `broker/orders.py:price_break` now consults `_v610_or_break_long/short`; Strike-1 `boundary_high/low` shifts by `k×ATR`.

Feature flags: `_V610_ATR_OR_BREAK_ENABLED = False` (ships dormant in v6.1.0; flipping on awaits calibration of `V610_OR_BREAK_K` against the Apr 27–May 1 weekly shadow data due tomorrow morning),

`V610_LATE_OR_ENABLED = True`. When the master flag is False, routes back through `_tiger_two_bar_long/short` (existing fixed-cents path) and the late-OR window logic does not contribute new entries either.

Validation

5 + 6 + 11 new tests: `tests/test_v610_atr_trail.py`, `tests/test_v610_ema_confirm.py`, `tests/test_v610_atr_or_break.py`. Full suite 655 passed, 0 failures, 0 regressions (up from 638 on v6.0.8.1).

Rollback playbook

If a flag-on flips a single day worse than expected, set the relevant module-level constant to `False` in a follow-up patch release — no schema/data migration required. The 5 flags (`_V610_ATR_TRAIL_ENABLED`, `_V610_EMA_CONFIRM_ENABLED`, `_V610_LUNCH_SUPPRESSION_ENABLED`, `_V610_ATR_OR_BREAK_ENABLED`, `V610_LATE_OR_ENABLED`) can be flipped independently. Recommended pre-prod validation: run each fix in shadow against the Apr 27–May 1 weekly shadow data (Saturday cron output) and the Apr 20–24 baseline before flipping default-off.

---

v6.2.0 — Entry-gate loosening (May 2026)

Three forensics-backed relaxations of the entry matrix, each behind its own feature flag.

Why

The 7,113-row entry-gate forensics sweep (`v611_roi_research/entry_gate_forensics.md`) and the TSLA 5/1 case study identified three gates rejecting entries that would have profited:

1. **Local override (divergence permit)** — 1,798 rejections at `evaluate_local_override` when the QQQ permit closed even though the per-ticker tape had cleared OR by a meaningful ATR multiple. 49.4% would-have-profited; long subset 64% wins, mean +$0.79/share.
2. **Boundary hold pre-10:30 ET** — AMZN 09:36–09:51 had 16 boundary-hold rejections, 100% would-have-profited, mean +$7.30/share. The 2-bar requirement punishes the highest-edge window of the session.
3. **ENTRY_1 DI threshold (25.0)** — TSLA 5/1 had 44 di_5m rejections, 100% would-have-profited; same gate blocked AVGO/GOOG/AMZN. Marginal entries 22 ≤ DI < 25 retain the same edge as DI ≥ 25.

Anti-recommendations (held the line, did NOT loosen): 5m ADX > 20, `anchor_misaligned`, STRIKE-CAP-3.

#A Local override OR-break leg

`engine/local_weather.py`, `broker/orders.py`.

New constants `V620_LOCAL_OR_BREAK_ENABLED = True`, `V620_LOCAL_OR_BREAK_K = 0.25`. New helper `_or_break_leg(side, last, or_high, or_low, atr_pm)` returns True iff the ticker has cleared OR by k×ATR_pm. `classify_local_weather` / `_check_direction` / `evaluate_local_override` threaded with `or_high` / `or_low` / `atr_pm` kwargs. Result struct gains `or_break_aligned`. Reason `"open_or_break"` when only the new leg fires.

`broker/orders.py:285` plumbs the OR-break params from existing locals. Strict ADDITION — never blocks a trade that would have fired under the legacy 3-leg path; only widens the divergence-override admit set.

#B Time-conditional 1-bar boundary hold pre-10:30 ET

`v5_10_1_integration.py`, `broker/orders.py`.

New constants `V620_FAST_BOUNDARY_ENABLED = True`, `V620_FAST_BOUNDARY_CUTOFF_HHMM_ET = "12:00"` (default updated to 12:00 in v6.10.0; was "10:30" at original v6.2.0 ship). New helper `_v620_fast_boundary_active(now_et)` returns True when wall-clock ET is before cutoff. `evaluate_boundary_hold_gate()` signature gained `now_et=None`; uses `required_closes=1` when fast-boundary active, else falls back to `eot.BOUNDARY_HOLD_REQUIRED_CLOSES = 2`.

`broker/orders.py:499` passes `now_et=now_et` (already defined at line 102) into the call. The spec constant is unchanged — only the gate caller relaxes — so `tests/test_spec_v15_conformance.py:96` (`== 2` assertion) still passes.

#D Generic ENTRY_1 DI threshold 25 → 22

`eye_of_tiger.py:30`.

`ENTRY_1_DI_THRESHOLD: 25.0 → 22.0`. Generic, NOT symbol-specific. Comment block cites TSLA 5/1 forensics (44 di_5m rejections, 100% would-have-profited).

Dashboard surfacing

`dashboard_server.py`, `dashboard_static/app.js`.

New `/api/state.v620_flags` block: `{local_or_break_enabled, local_or_break_k, fast_boundary_enabled, fast_boundary_cutoff_et, entry1_di_threshold}`. Defensively read from `engine.local_weather`, `v5_10_1_integration`, `eye_of_tiger`; safe defaults if any module is missing.

Three card-text suffixes appended to existing card vals (mirrors v6.1.1 plumbing):

- **Local Weather card** (Phase 1): `· OR+0.25×ATR leg` when `local_or_break_enabled` and `k > 0`.
- **Boundary card** (Phase 2): `· 1-bar pre-10:30` when fast-boundary active, `· 2-bar hold` when off.
- **Momentum card** (Phase 3, ENTRY_1): `· DI≥22` (or whatever threshold is live).

`s.v620_flags → v620Flags` parsed once in `renderPermitMatrix`, threaded through `_pmtxBuildRow.visibilityOpts`, into the card data object. Zero new DOM, zero new endpoints, zero new function signatures.

Rollback playbook

Three independent module-level flags: `V620_LOCAL_OR_BREAK_ENABLED`, `V620_FAST_BOUNDARY_ENABLED`, plus the literal `ENTRY_1_DI_THRESHOLD` constant. Setting any to its v6.1.x value fully reverts that path — the new kwargs default to `None` so legacy logic re-engages cleanly. The OR-break leg is strictly additive (never narrows admits); the fast-boundary path requires `now_et` to relax (legacy callers without it get spec-strict 2-bar).

Last refresh: May 2026, against `BOT_VERSION = "6.5.0"`.

---

§18b. v6.6.0 — Ingest Hardening (May 2026)

Product spec: `v6_6_0_product_spec.md` (Priya). **Architecture doc:** `v6_6_0_architecture.md` (Aria). All 9 decisions (P1–P4, A1–A5) are locked and ratified by Val.

Overview

v6.6.0 hardens the always-on Algo Plus ingest layer introduced in v6.5.0 across three pillars:

- **Pillar A — SLA Monitoring:** Formal SLA metrics (last_bar_age_s, open_gaps_today, backfill_queue_depth, backfill_lag_s) with GREEN/YELLOW/RED thresholds. RTH-only enforcement (09:30–16:00 ET). Surfaced on `/api/state.ingest_status.ingest_health`.
- **Pillar B — Gap-Fill Verification:** Every `RestBackfillWorker._backfill()` call triggers an inline re-scan (Decision A3) immediately after writing bars. Gap lifecycle (open → backfilling → closed/missing) is persisted in `/data/ingest_audit.db`.
- **Pillar C — Trading Gate:** Per-ticker two-state gate (ALLOW/BLOCK) with hysteresis (5 min RED→BLOCK, 2 min GREEN→ALLOW). Fires in `execute_breakout()` after post-loss cooldown. Ships as `dry_run` only in v6.6.0; `enforce` flip requires Val+Devi+Reese sign-off (Decision A5).

New Files

File	Lines	Role
`ingest_config.py`	~100	All SLA + gate tunable constants; env-var overrides at import time (Decision A1)
`ingest/sla.py`	~311	SLAThreshold, SLAMetric, IngestHealthState; SLA classifier; get_health_state(); RTH check
`ingest/audit.py`	~406	AuditLog; ingest_audit.db schema; gap lifecycle + gate decision persistence (Decision A2)
`engine/ingest_gate.py`	~288	GateDecision, _TickerGateState; evaluate_gate(); two-state hysteresis (Decision A4)
`tests/test_ingest_sla.py`	—	SLA threshold + color derivation unit tests
`tests/test_ingest_gate.py`	—	Gate hysteresis + dry_run unit tests
`tests/test_gap_audit.py`	—	AuditLog DB lifecycle unit tests
`tests/test_gate_smoke.py`	—	Gate smoke: enforce mode, green ingest → allow

Modified Files

File	Change
`ingest/algo_plus.py`	`_backfill()` : adds `_backfill_start` timer, AuditLog calls, `_verify_gap_closed()` inline; `_update_ingest_stats()` : propagates to SLA collector; `_ingest_health_snapshot()` : adds `ingest_health` , `gap_audit` , `gate_override_active` keys
`broker/orders.py`	`execute_breakout()` : adds ingest gate check after post-loss cooldown (fails open; dry_run never returns early)
`trade_genius.py`	`gap_detect_task()` : adds AuditLog.record_gap_detected/enqueued calls, SLA gap count update, daily retention prune
`bot_version.py`	`BOT_VERSION = "6.5.2" → "6.6.0"`
`CHANGELOG.md`	v6.6.0 entry

Storage

- **`/data/ingest_audit.db`** (new, Decision A2): two tables: `ingest_gap_audit` (gap lifecycle, 180-day retention) and `ingest_gate_decisions` (gate allow/block log, 180-day retention). WAL mode. `state.db` is untouched.
- **Retention:** 180 calendar days for both audit tables (Decision P4). Bar archive retention unchanged at 90 days.

Locked Decisions

#	Decision	Implementation
P1	Gate ceiling 20 min	`GATE_REST_ONLY_CEILING_S=1200` in `ingest_config.py`
P2	Two-state only (no Yellow action)	`engine/ingest_gate.py` : only `"allow"` and `"block"` decisions; no half-size logic
P3	RTH-only enforcement (09:30–16:00 ET)	`_is_rth()` in `ingest/sla.py` ; gate returns green outside RTH
P4	180-day audit retention	`AUDIT_RETENTION_DAYS=180` ; prune in `gap_detect_task()` via `AuditLog.prune_old_rows()`
A1	SLA defaults: last_bar_age_s>300→RED, open_gaps>2→RED	`ingest_config.py` constants
A2	Separate `/data/ingest_audit.db`	`ingest/audit.py` opens its own SQLite connection
A3	Inline verification (one extra read per gap)	`_verify_gap_closed()` called from `_backfill()`
A4	Hysteresis: 5 min RED→BLOCK, 2 min GREEN→ALLOW	`_update_ticker_gate_state()` in `engine/ingest_gate.py`
A5	dry_run in v6.6.0; enforce needs Val+Devi+Reese	`SSM_INGEST_GATE_MODE=dry_run` default; enforce path gated in code

Rollout

- **v6.6.0 (this PR):** `SSM_INGEST_GATE_MODE=dry_run`. Gate evaluates and logs BLOCK decisions but never prevents a trade. Collects data for Monday calibration.
- **v6.6.1 (future):** Flip `SSM_INGEST_GATE_MODE=enforce` in Railway after written sign-off from Devi (data validation), Reese (cost ceiling), and Val (ratification). No code change required.

Last refresh: May 2026, against `BOT_VERSION = "6.6.0"`.

System Health Test (v6.7.0)

Purpose: Provides a single sub-15-second signal covering every component that can silently break production trading. Triggered manually via `/test` in Telegram or automatically by the scheduler at 8:20 CT (pre-open check) and 8:31 CT (post-open confirmation). A clean run at 8:20 CT is the green light for RTH. CRITICAL failures emit `[ERROR] [SYS-TEST]` log lines picked up by the 8:35 CT morning cron log-grep. WARN failures emit `[WARNING] [SYS-TEST]`. INFO results are Telegram-only (no log noise).

The `/test` Telegram command and the 8:20/8:31 CT scheduler calls run 15 checks covering every component that can silently break production trading.

15 Checks

#	Name	Block	Severity	RTH-gated?
1	Alpaca account reachability	A	CRITICAL	No
2	Alpaca positions parity	A	CRITICAL (RTH) / WARN (non-RTH) / skip (paper)	Partial
3	Order round-trip (SPY IOC)	A	CRITICAL	No
4	WS connection state	B	CRITICAL (RTH) / INFO (non-RTH)	Partial
5	Bar archive today	B	CRITICAL/WARN (RTH) / INFO (non-RTH)	Partial
6	AlgoPlus ingest liveness	B	CRITICAL (RTH) / INFO (non-RTH)	Partial
7	Ingest gate state	B	INFO always	No
8	SQLite reachability	C	CRITICAL	No
9	paper_state JSON parity	C	CRITICAL	No
10	Disk space on /data	C	CRITICAL/WARN	No
11	Kill-switch posture	D	CRITICAL / INFO	No
12	Trading mode	D	INFO	No
13	Dashboard /api/state	E	WARN	No
14	Telegram config sanity	E	CRITICAL	No
15	Version parity	E	CRITICAL	No

Orchestrator Design

- `CheckResult` dataclass: `name, block, severity, message, duration_ms`.
- `_safe_check(name, block, fn, timeout_s)` : timeout enforcement + exception catch.
- `_run_system_test_sync_v2(label)` : 5 block-runners submitted to `ThreadPoolExecutor(max_workers=5)`. Checks within a block run sequentially. RTH computed once at entry; all checks share the same RTH bool.
- Single logging emission point in the orchestrator: `[ERROR] [SYS-TEST]` for critical, `[WARNING] [SYS-TEST]` for warn, silence for ok/info/skip (D2).
- Concurrency guard: `_system_test_running` + `_system_test_lock`. Concurrent call returns cached result with staleness note.
- `_run_system_test_sync(label)` preserved as one-line shim for scheduler call sites.

RTH Window

08:30:00 \u2013 15:00:00 US/Central (America/Chicago), inclusive. Computed via `CDT` ZoneInfo in `_is_rth_ct()`. Outside RTH: Checks 4/5/6 \u2192 INFO; Check 2 \u2192 WARN.

Thresholds (locked, PRODUCT_SPEC.md)

Check	Threshold
Disk CRITICAL	< 1 GB free
Disk WARN	< 5 GB free
WS WARN	last bar 30\u201390s (RTH)
WS CRITICAL	last bar > 90s or disconnected (RTH)
AlgoPlus CRITICAL	last tick > 60s (RTH)
paper_state parity	delta > $0.01
Order round-trip limit	bid x 0.90, floor-cent, min $1.00

Sample Passing Output

```
System Test [8:20-pre-RTH] v6.7.0
Block A -- Broker
  Alpaca account: OK buying_power $25,341.22
  Alpaca positions: OK parity (3=3)
  Order round-trip: OK 287ms
Block B -- Streaming
  WS: OK connected, last bar 2s ago
  Bars today: OK /data/bars/2026-05-03 -- 28 files, 4.2MB
  AlgoPlus: OK tick 1s ago
  Ingest gate: INFO dry_run=True
Block C -- State
  SQLite: OK shadow_positions=1,247
  paper_state parity: OK $24,512.18
  Disk /data: OK 8.4GB free
Block D -- Risk
  Kill-switch: INFO limit=-$1,500, realized=+$0.00, halted=False
  Mode: INFO paper
Block E -- Obs
  Dashboard: OK shadow_data_status=live
  Telegram: OK owner_id set
  Version: OK 6.7.0 parity
All systems GO  (took 8.3s)
```

On CRITICAL failure the footer shows: `N CRITICAL, M WARN -- see logs` and each failing check emits exactly one `[ERROR] [SYS-TEST] Block X: ...` log line.

Last refresh: May 2026, against `BOT_VERSION = "6.7.0"`.

---

§19. v6.9.0 -- Backtest Cache Layer

Goal: cut Wave 2 sweep wall from ~15 min to ~2 min by caching the 84-day SIP corpus once per sweep series.

L1 -- Parquet bar cache (`backtest/bar_cache.py`)

- **Cache root:** `<bars_dir>/.cache_v1/`
- **One Parquet per ticker** covering the full date range: `<cache_root>/<TICKER>.parquet`
- **Schema:** `ts (str ISO-8601)` , `date (str)` , `session (str: pre|rth)` , `open/high/low/close (float64)` , `volume (int64)` , `vw (float64)` , `n (int64)` , `_extras (str JSON passthrough)`
- **Compression:** ZSTD level 3.
- **Cache key:** SHA-256 of (path, mtime_ns, size) for all source JSONL files for that ticker; stored in sidecar `<TICKER>.parquet.meta.json` .
- **Public API:** `get_bars(bars_dir, ticker, date) -> list[dict]` -- drop-in for `replay_v511_full.load_day_bars` ; `build_all(bars_dir)` for CLI pre-build.
- **Integration:** `load_day_bars` delegates to `get_bars` ; fallback to direct JSONL parse if `pyarrow` not installed.

L2 -- Indicator precompute cache (`backtest/indicator_cache.py`)

- **Cache root:** `<bars_dir>/.indcache_v1/`
- **One Parquet per (ticker, params_hash):** `<cache_root>/<TICKER>__<params_hash>.parquet`
- **Indicators covered (pure functions of bars + static params):** ATR(14), ATR(20), EMA9, EMA20, EMA50, VWAP rolling, OR high/low (9:30-9:35, 9:30-10:00 ET), premarket high/low/range, session boundary markers.
- **Cache key:** SHA-256 of (bar_cache_key, indicator_set_version, canonical params JSON). Invalidates if bars, indicator logic, or params change.
- **Public API:** `get_indicators(bars_dir, ticker, date, indicators, params) -> dict[str, list[float]]`
- **Sweep reuse:** variants changing only entry/exit logic (C5, C7, C10) get full L2 reuse; variants changing indicator params get a cache miss only on the changed indicator.

Engine module annotations

`engine/scan.py` , `engine/sma_stack.py` , `engine/seeders.py` carry docstring annotations directing backtest callers to `indicator_cache.get_indicators` . Live engine code paths are unchanged.

---

§20. v6.11.0 -- C25: SPY Regime-B Short Amplification

Goal: harvest the regime-B short-side edge (+60.78% WR / +$14.43/pair across 232 short pairs in the 84d v6.10.0 SIP corpus) by scaling short-side position size 1.5x on SPY regime-B days within the [10:00, 11:00) ET window.

Regime classification (`spy_regime.py`)

- Mirrors `qqq_regime.py` pattern. Class `SpyRegime` . No external I/O.
- Two price anchors captured via `tick(now_et, spy_price)` :
- `spy_open_930` : first bar in `[09:30:00, 09:31:00)` ET.
- `spy_close_1000` : first bar in `[10:00:00, 10:01:00)` ET.
- At 10:00 ET, computes `ret_pct = (close_1000 - open_930) / open_930 * 100` .
- Five bands (strict-both-sides on B per spec test-11):
- A: `ret <= -0.50`
- B: `-0.50 < ret < -0.15` (moderately down)
- C: `-0.15 <= ret <= +0.15` (flat)
- D: `+0.15 < ret <= +0.50`
- E: `ret > +0.50`
- Feed gap (missing 09:30 anchor at 10:00): regime stays `None` , C25 fails closed.
- `daily_reset()` called from `reset_daily_state()` in `trade_genius.py` .
- Log tag: `[V611-REGIME-B]` at classification time.

Env-var contract (`eye_of_tiger.py`)

Variable	Default	Purpose
`V611_REGIME_B_ENABLED`	`"1"`	Feature kill-switch. Set `"0"` to no-op.
`V611_REGIME_B_SHORT_SCALE_MULT`	`"1.5"`	Position-size multiplier on amp.
`V611_REGIME_B_SHORT_ARM_HHMM_ET`	`"10:00"`	Amp window start (inclusive).
`V611_REGIME_B_SHORT_DISARM_HHMM_ET`	`"11:00"`	Amp window end (exclusive).
`V611_REGIME_B_LOWER_PCT`	`"-0.50"`	Band-B lower boundary (strict).
`V611_REGIME_B_UPPER_PCT`	`"-0.15"`	Band-B upper boundary (strict).

Sizing helper (`broker/orders.py`)

Helper `_maybe_apply_regime_b_short_amp(*, cfg, shares, ticker, now_et, regime, scale, arm_hhmm_et, disarm_hhmm_et)` :

- Called AFTER the v15-sizing tier decision, BEFORE `notional = current_price * shares` .
- Window `[arm, disarm)` is half-open: arm-inclusive, disarm-exclusive.
- Returns unmodified shares for: long side, outside window, non-regime-B, disabled.
- Log tag: `[V611-AMP]` on every amplified entry.

Wire points

- `trade_genius.py` line ~1100: `import spy_regime; _SPY_REGIME = spy_regime.SpyRegime()`
- `trade_genius.py` `_qqq_weather_tick()` : calls `_SPY_REGIME.tick(_now_et(), spy_price)` inside the existing SPY price fetch block (fail-safe try/except).
- `trade_genius.py` `reset_daily_state()` : calls `_SPY_REGIME.daily_reset()` .
- `trade_genius.py` STARTUP SUMMARY: logs `regime_b_amp` , `mult` , `arm` , `disarm` .

Dashboard fields (`/api/state`)

```
"spy_regime_today": {
  "regime": "B",
  "spy_open_930": 685.71,
  "spy_close_1000": 684.27,
  "ret_pct": -0.21,
  "classified_at": "2026-05-04T14:00:00Z"
},
"v611_regime_b_active": true,
"v611_window": {"arm": "10:00", "disarm": "11:00"}
```

`v611_regime_b_active` is `true` only when regime==B AND current ET time is inside `[arm, disarm)` . Frontend (`app.js`) surfaces SPY 9:30, SPY 10:00, SPY 30m %, regime letter, and C25 amp state in the Phase 1 Weather block.

Validation

- 84d SIP backtest (v2 [10:00, 11:00), 1.5x): **+$683 vs $12,145 baseline (+5.6%)**
- Amp pool: 108 pairs / 61.11% WR / +$12.65 mean $/pair
- Bootstrap P(delta<0)=0.10%; 95% CI [+$151, +$1,274]
- Walk-forward H1/H2: +$511 / +$173 (same sign, per-pair drift 23%)
- Sources: `c25_v2_results.json`, `c25_intraday_stability.json`

Rollback

Set `V611_REGIME_B_ENABLED=0` in Railway. The feature becomes a complete no-op without code revert.

---

§21. Pre-market Readiness Check (v6.11.1)

Goal: validate that the live production bot is healthy 5 hours before market open, giving Val time to intervene if a check fails before the 09:30 ET bell.

Cron schedule

Daily at **04:30 ET** on weekdays, invoked via `railway ssh`:

```
python3 /app/scripts/premarket_check.py --in-container --json
```

The cron itself is created separately via `pplx-tool schedule_cron` after the v6.11.1 Railway deploy completes (out of scope for this PR).

Script: `scripts/premarket_check.py`

Standalone Python script (stdlib only). Two execution modes: - `--in-container` (default): direct filesystem + module-import access inside Railway. - `--remote`: Phase 1 stub -- errors with "use --in-container via railway ssh".

14 checks performed (in order)

Hard failures (FAIL -> overall FAIL, exit 1; fail-fast through check 7):

1. **process_alive** -- `bot_version` importable; `state.db` last write < 24h.
2. **version_parity** -- `bot_version.BOT_VERSION == trade_genius.BOT_VERSION`.
3. **module_imports** -- all 17 critical modules import cleanly.
4. **persistence_reachable** -- `state.db` opens; expected tables present.
5. **bar_archive_yesterday** -- last-trading-day bar directory non-empty (>= 5 files).
6. **alpaca_auth_paper** -- paper API credentials validate (200 + `account_number`).
7. **alpaca_auth_live** -- live API credentials validate (SKIP if not configured).

Soft failures (WARN -> overall WARN if no FAILs; always run):

1. **alpaca_data_feed_recent_trade** -- AAPL latest trade < 1h old.
2. **classifier_smoke** -- `SpyRegime` + `QQQRegime` classify synthetic bars.
3. **sizing_helper_smoke** -- `_maybe_apply_regime_b_short_amp` all 6 branches correct.
4. **dashboard_state** -- `/api/state` reachable; `spy_regime_today` + `v611_window` present.
5. **disk_space** -- `>= 1 GB` free on `/data` (WARN); `>= 100 MB` (FAIL).
6. **time_sync** -- local clock vs Alpaca `Date` header drift `<= 5s` (WARN `> 5s`, FAIL `> 30s`).
7. **cron_introspection** -- SKIP in-container (no Railway API token available).
8. **replay_smoke** -- SKIP (Phase 2 candidate).

Output artifact

Written to `/data/preflight/<UTC_date>.json` after each cron run. JSON shape:

```
{
  "version": "1",
  "timestamp_utc": "2026-05-04T08:30:00Z",
  "bot_version": "6.11.1",
  "overall_status": "PASS",
  "n_pass": 12, "n_warn": 1, "n_fail": 0, "n_skip": 2,
  "elapsed_total_ms": 1234,
  "checks": [...]
}
```

On-demand invocation: the Telegram `/test` command runs the same checks (with `write_artifact=False` so the daily artifact is preserved).

Exit codes

Code	Meaning
0	PASS
1	FAIL
2	WARN

Future hook

v6.12.0 will gate live-session actions on PASS: the bot reads `/data/preflight/<today>.json` at startup and refuses to arm if `overall_status != "PASS"`. (Controlled by a `V612_STARTUP_GATE` env var.)

Callable Python API

```
from scripts.premarket_check import run_all_checks, format_for_telegram
result = run_all_checks(in_container=True, write_artifact=False)
text = format_for_telegram(result)   # compact Telegram-ready string (< 3500 chars)
```